



हां जी तो करें स्टार्ट। वेलकम बै...  
TXT 164.01KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

## LLD: Dating App (Tinder/Hinge/Bumble Clone) — Complete Notes

### 1. Introduction

This lecture builds a **Low-Level Design (LLD)** for a **Dating Application** (like Tinder, Hinge, Bumble). Such questions are increasingly asked in interviews (including matrimonial apps) because they involve interesting algorithms like **left/right swipe** and **match-making scoring**.

The design follows a **Top-Down Approach** (unlike most previous LLDs which used Bottom-Up), because there are **many small objects** and understanding their integration is easier if we start with the biggest object (User) and then drill down.

### 2. Functional Requirements

1. **User can swipe Left or Right** on a profile.
2. **User can set up their own profile** — name, bio, interests, hobbies (add/remove).
3. **User can set their preferences** — interested gender, max distance, interests, etc.
4. **Once there is a match, they can chat** in a chat room.
5. **User can see all profiles near their location** (within e.g. 5 km). Near-by strategy should be **scalable** (different strategies possible — blocked users, validations, etc.).
6. **User should be notified** whenever there is a match or a new message is received.
7. **User matching should be based on several factors** — interests, location, preferences, etc., producing a **match score**.

### Non-Functional Requirements

- Scalable system
- Use design principles (SOLID, loose coupling)
- Modular code

### 3. Happy Flow

1. A **User** opens the app.
2. User has a **Profile** (photo, bio, interests, preferences).
3. User sees profiles of other users and **swipes Left/Right**.
4. If **both swipe Right** → **Match** happens (based on match score).
5. A **Chat Room** becomes available.
6. User gets **Notifications** for matches & new messages.

### 4. Why Top-Down Approach?

- Many small objects would be created (Preferences, Profile, Interest, Gender, Location, Messages, ChatRoom, Matchers, etc.).
- If we started Bottom-Up, we might not realize we need separate classes for `Interest` , `Gender` , `Location` — we might just use `String` .
- Starting with the **biggest object (User)** helps identify **what small objects are needed**.

### 5. Design Patterns Used

| Pattern                            | Where Used                                                    |
|------------------------------------|---------------------------------------------------------------|
| Observer                           | Notification Service                                          |
| Singleton                          | NotificationService, LocationService                          |
| Strategy                           | Near-By Location strategies, Matching strategies              |
| Chain of Responsibility (modified) | Match score calculation (Location → Interest → Basic Details) |

| Pattern | Where Used     |
|---------|----------------|
| Factory | MatcherFactory |

## 6. UML Class Design (Explained Step-by-Step)

### 6.1 Notification Service (Observer Pattern)

#### NotificationObservable (Subject / Observable)

- Holds `Map<String, NotificationObserver>` — key = `userId`, value = `observer`.
- Methods:
  - `registerObserver(userId, observer)`
  - `removeObserver(userId)` — only needs `userId` since map is keyed by it.
  - `notifyUser(userId, message)` — notifies a specific user.
  - `notifyAll(message)` — notifies everyone.
- **Singleton** class.

#### NotificationObserver (abstract)

- `update(message)` — abstract.

#### UserNotificationObserver (concrete)

- `update(message)` — prints the notification.

**Relationship:** `NotificationService (Observable)`  `NotificationObserver` (1-to-many).

### 6.2 User (Main Class)

#### Fields:

- `userId` : String
- `profile` : UserProfile (Composition)
- `preferences` : UserPreferences (Composition)

- `notificationObserver` : `UserNotificationObserver`
- `swipeHistory` : `Map<String, Swipe>` (`userId` → `Swipe` enum)
- `location` : `Location`

### Constructor:

- Creates its own `UserNotificationObserver` (tightly coupled — always the same type).
- Registers itself with `NotificationService` singleton via `registerObserver(userId, observer)` .

### Destructor:

- Removes itself from `NotificationService`.
- Destroys profile, preferences, observer (Composition).

### Methods:

- `swipe(userId, Swipe action)` — saves to `swipeHistory`.
- `hasLiked(userId)` → `boolean`
- `hasDisliked(userId)` → `boolean`
- `hasInteractedWith(userId)` → `boolean`
- `isInterestedIn(userId)` → `boolean`
- `displayProfile()` — delegates to `profile.display()` (Principle of Least Knowledge).

## 6.3 UserProfile

### Fields:

- `name` : `String`
- `age` : `int`
- `gender` : `Gender` (enum)
- `bio` : `String`
- `pictures` : `List<String>` (paths — actual files in S3)
- `interests` : `List<Interest>`
- `location` : `Location`

**Methods:** `display()` , getters/setters, `addInterest()` , `removeInterest()` .

## Relationships:

- User ◆—— UserProfile (Composition — profile can't exist without user).
- UserProfile ◆—— Location (Composition).

## 6.4 Interest

### Fields:

- name : String
- category : String

## 6.5 Gender (Enum)

text

MALE, FEMALE, NON\_BINARY, OTHER

## 6.6 Location

### Fields:

- latitude : double
- longitude : double

### Method:

- distanceInKm(Location other) — uses **Haversine Formula**.

## 6.7 UserPreferences

### Fields:

- minAge : int
- maxAge : int

- `maxDistance : int (km)`
- `interestedIn : List<Gender>`
- `interests : List<String>`

**Relationship:** User  UserPreferences (Composition).

## 6.8 Swipe (Enum)

`text`

`LEFT, RIGHT`

## 6.9 LocationService (Singleton + Strategy)

**Fields:**

- `nearByStrategy : NearByStrategy`

**Methods:**

- `getInstance()`
- `setStrategy(NearByStrategy)`
- `findNearByUsers(location, maxDistance, allUsers)` — **delegates** to strategy.

**Relationship:** LocationService has-a NearByStrategy .

## 6.10 NearByStrategy (Interface)

- `findNearByUsers(Location loc, double maxDistance, List<User> allUsers) → List<User>`

**BasicNearByStrategy (Concrete)**

- Loops through all users; computes `loc.distanceInKm(user.getLocation())` ; if  $\leq$  `maxDistance` → add to result.

*(Can add more strategies later — e.g., filtering blocked users.)*

## 6.11 ChatRoom

### Fields:

- `chatRoomId` : String
- `participantIds` : List<String> (kept as list for future group chat)
- `messages` : List<Message>

### Methods:

- `addMessage(senderId, content)`
- `removeMessage(messageId)`
- `displayChat()`
- `hasParticipant(userId) → boolean`

**Relationship:** ChatRoom ◆—— Message (1-to-many).

## 6.12 Message

### Fields:

- `senderId` : String
- `content` : String
- `timestamp` : time\_t (or LocalDateTime / Date )

**Methods:** getters, `getFormattedTime()` .

## 6.13 Matcher (Base Class) + Chain of Responsibility

### Matcher (abstract)

- `calculateMatchScore(User u1, User u2) → double`

### Concrete Matchers:

- `BasicDetailsMatcher` — matches bios, pictures, hobbies.
- `InterestMatcher` — matches interests.
- `LocationMatcher` — matches location proximity.

**Chain setup:**

```
text
```

```
LocationMatcher → InterestMatcher → BasicDetailsMatcher → returns avg score
```

Each matcher holds a **hard reference** to the next matcher (fixed chain, not dynamic).

**Flow:**

1. `LocationMatcher.calculateMatchScore(u1,u2)` calls `InterestMatcher.calculateMatchScore(u1,u2)` first.
2. `InterestMatcher` calls `BasicDetailsMatcher` first.
3. `BasicDetailsMatcher` computes score (e.g., 50).
4. `InterestMatcher` computes score (e.g., 60) and **averages** with returned 50.
5. `LocationMatcher` computes score (e.g., 70) and averages all → final score.

**Note:** This looks like Decorator but is **NOT** — intent is Chain of Responsibility (fixed chain, not dynamic decoration).

**6.14 MatcherType (Enum)**

```
text
```

```
LOCATION, INTEREST, BASIC
```

**6.15 MatcherFactory**

- `createMatcher(MatcherType)` → returns appropriate Matcher (with chain pre-wired).

**6.16 DatingApp (Orchestrator)****Fields:**

- `users : List<User>`
- `chatRooms : List<ChatRoom>`

**Methods:**

- `swipe(userId, SwipeType)`
- (Other orchestration methods.)

**7. Full UML Relationships Summary**

```
text
```

```

NotificationService ◆— NotificationObserver (1..*)
UserNotificationObserver ▷— NotificationObserver
User ◆— UserProfile
User ◆— UserPreferences
User ◆— UserNotificationObserver
User ◇— Swipe (enum, in map)
UserProfile ◆— Location
UserProfile ◆— Interest (1..*)
UserPreferences ◆— Gender (list)
LocationService ◇— NearByStrategy
BasicNearByStrategy ▷— NearByStrategy
Matcher <— BasicDetailsMatcher, InterestMatcher, LocationMatcher
MatcherFactory ..> MatcherType, Matcher
ChatRoom ◆— Message (1..*)
DatingApp ◆— User (1..*), ChatRoom (1..*)

```

**8. Code — Java****8.1 Notification (Observer Pattern)**

```
java
```

```

// Observer
abstract class NotificationObserver {
    public abstract void update(String message);
}

class UserNotificationObserver extends NotificationObserver {
    @Override
    public void update(String message) {
        System.out.println("Notification for user: " + message);
    }
}

```

```

    }

    // Observable (Singleton)
    class NotificationService {
        private static NotificationService instance;
        private Map<String, NotificationObserver> observers = new HashMap<>();

        private NotificationService() {}

        public static synchronized NotificationService getInstance() {
            if (instance == null) instance = new NotificationService();
            return instance;
        }

        public void registerObserver(String userId, NotificationObserver observer) {
            observers.put(userId, observer);
        }

        public void removeObserver(String userId) {
            observers.remove(userId);
        }

        public void notifyUser(String userId, String message) {
            NotificationObserver obs = observers.get(userId);
            if (obs != null) obs.update(message);
        }

        public void notifyAll(String message) {
            for (NotificationObserver obs : observers.values()) obs.update(message);
        }
    }
}

```

## 8.2 Enums & Basic Models

java

```

enum Gender { MALE, FEMALE, NON_BINARY, OTHER }
enum Swipe { LEFT, RIGHT }
enum MatcherType { LOCATION, INTEREST, BASIC }

class Interest {
    private String name, category;
    public Interest(String name, String category) { this.name = name; this.category = category; }
    // getters...
}

```

```

class Location {
    private double latitude, longitude;
    public Location(double lat, double lon) { this.latitude = lat; this.longitude = lon;
}

    public double distanceInKm(Location other) {
        double R = 6371; // Earth radius km
        double dLat = Math.toRadians(other.latitude - this.latitude);
        double dLon = Math.toRadians(other.longitude - this.longitude);
        double a = Math.sin(dLat/2) * Math.sin(dLat/2)
            + Math.cos(Math.toRadians(this.latitude)) * Math.cos(Math.toRadians(oth
r.latitude))
            * Math.sin(dLon/2) * Math.sin(dLon/2);
        double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
        return R * c;
    }
}

```

## 8.3 UserProfile & Preferences

java

```

class UserProfile {
    private String name, bio;
    private int age;
    private Gender gender;
    private List<String> pictures = new ArrayList<>();
    private List<Interest> interests = new ArrayList<>();
    private Location location;

    public UserProfile(String name, int age, Gender gender, Location location) {
        this.name = name; this.age = age; this.gender = gender; this.location = location;
    }

    public void addInterest(Interest i) { interests.add(i); }
    public void removeInterest(Interest i) { interests.remove(i); }
    public Location getLocation() { return location; }
    public int getAge() { return age; }
    public Gender getGender() { return gender; }
    public List<Interest> getInterests() { return interests; }

    public void display() {
        System.out.println(name + ", " + age + ", " + gender + " - " + bio);
        for (Interest i : interests) System.out.println(" Interest: " + i.name);
        System.out.println(" Location: " + location.latitude + "," + location.longitud
e);
    }
}

```

```
}

class UserPreferences {
    private int minAge, maxAge, maxDistance;
    private List<Gender> interestedIn = new ArrayList<>();
    private List<String> interests = new ArrayList<>();

    public boolean isInterestedIn(Gender g) { return interestedIn.contains(g); }
    // getters/setters...
}
```

## 8.4 User (Main Class)

java

```
class User {
    private String userId;
    private UserProfile profile;
    private UserPreferences preferences;
    private UserNotificationObserver notificationObserver;
    private Map<String, Swipe> swipeHistory = new HashMap<>();

    public User(String userId, UserProfile profile, UserPreferences preferences) {
        this.userId = userId;
        this.profile = profile;
        this.preferences = preferences;
        this.notificationObserver = new UserNotificationObserver();
        NotificationService.getInstance().registerObserver(userId, notificationObserver);
    }

    public void swipe(String targetUserId, Swipe action) {
        swipeHistory.put(targetUserId, action);
    }

    public boolean hasLiked(String targetUserId) {
        return swipeHistory.getOrDefault(targetUserId, Swipe.LEFT) == Swipe.RIGHT;
    }

    public boolean hasDisliked(String targetUserId) {
        return swipeHistory.getOrDefault(targetUserId, Swipe.RIGHT) == Swipe.LEFT;
    }

    public boolean hasInteractedWith(String targetUserId) {
        return swipeHistory.containsKey(targetUserId);
    }

    public boolean isInterestedIn(String targetUserId) {
```

```

        return swipeHistory.containsKey(targetUserId);
    }

    public void displayProfile() { profile.display(); }
    public String getUserId() { return userId; }
    public UserProfile getProfile() { return profile; }
    public UserPreferences getPreferences() { return preferences; }
    public Location getLocation() { return profile.getLocation(); }
}

```

## 8.5 Location Service + Strategy

java

```

interface NearByStrategy {
    List<User> findNearByUsers(Location loc, double maxDistance, List<User> allUsers);
}

class BasicNearByStrategy implements NearByStrategy {
    @Override
    public List<User> findNearByUsers(Location loc, double maxDistance, List<User> allUsers) {
        List<User> result = new ArrayList<>();
        for (User u : allUsers) {
            if (loc.distanceInKm(u.getLocation()) <= maxDistance) result.add(u);
        }
        return result;
    }
}

class LocationService {
    private static LocationService instance;
    private NearByStrategy nearByStrategy;

    private LocationService() { nearByStrategy = new BasicNearByStrategy(); }

    public static synchronized LocationService getInstance() {
        if (instance == null) instance = new LocationService();
        return instance;
    }

    public void setStrategy(NearByStrategy s) { this.nearByStrategy = s; }

    public List<User> findNearByUsers(Location loc, double maxDistance, List<User> allUsers) {
        return nearByStrategy.findNearByUsers(loc, maxDistance, allUsers);
    }
}

```

```

    }
}

```

## 8.6 ChatRoom & Message

java

```

class Message {
    private String senderId, content;
    private java.time.LocalDateTime timestamp;

    public Message(String senderId, String content) {
        this.senderId = senderId; this.content = content; this.timestamp = java.time.Loca
lDateTime.now();
    }

    public String getFormattedTime() { return timestamp.toString(); }
    public String getContent() { return content; }
    public String getSenderId() { return senderId; }
}

class ChatRoom {
    private String chatRoomId;
    private List<String> participantIds = new ArrayList<>();
    private List<Message> messages = new ArrayList<>();

    public ChatRoom(String id, List<String> participantIds) {
        this.chatRoomId = id; this.participantIds = participantIds;
    }

    public void addMessage(String senderId, String content) {
        messages.add(new Message(senderId, content));
    }

    public boolean hasParticipant(String userId) { return participantIds.contains(userI
d); }

    public void displayChat() {
        System.out.println("--- Chat Room " + chatRoomId + " ---");
        for (Message m : messages)
            System.out.println(m.getFormattedTime() + " | " + m.getSenderId() + ": " + m.
getContent());
    }
}

```

## 8.7 Matcher (Chain of Responsibility)

java

```

abstract class Matcher {
    public abstract double calculateMatchScore(User u1, User u2);
}

class BasicDetailsMatcher extends Matcher {
    @Override
    public double calculateMatchScore(User u1, User u2) {
        // Simple example: score based on shared interests count
        int shared = 0;
        for (Interest i : u1.getProfile().getInterests())
            if (u2.getProfile().getInterests().contains(i)) shared++;
        int total = Math.max(1, u1.getProfile().getInterests().size());
        return (100.0 * shared) / total;
    }
}

class InterestMatcher extends Matcher {
    private Matcher next;
    public InterestMatcher(Matcher next) { this.next = next; }

    @Override
    public double calculateMatchScore(User u1, User u2) {
        double nextScore = next.calculateMatchScore(u1, u2);
        // Interest-based score
        int shared = 0;
        for (String i : u1.getPreferences().getInterests())
            if (u2.getPreferences().getInterests().contains(i)) shared++;
        double myScore = (shared * 100.0) / Math.max(1, u1.getPreferences().getInterests
().size());
        return (nextScore + myScore) / 2.0;
    }
}

class LocationMatcher extends Matcher {
    private Matcher next;
    public LocationMatcher(Matcher next) { this.next = next; }

    @Override
    public double calculateMatchScore(User u1, User u2) {
        double nextScore = next.calculateMatchScore(u1, u2);
        double dist = u1.getLocation().distanceInKm(u2.getLocation());
        double myScore = Math.max(0, 100 - dist * 10); // closer → higher score
        return (nextScore + myScore) / 2.0;
    }
}

```

```

    }

    class MatcherFactory {
        public static Matcher createMatcher(MatcherType type) {
            switch (type) {
                case LOCATION:
                    return new LocationMatcher(new InterestMatcher(new BasicDetailsMatcher
                    ()));
                case INTEREST:
                    return new InterestMatcher(new BasicDetailsMatcher());
                case BASIC:
                default:
                    return new BasicDetailsMatcher();
            }
        }
    }
}

```

## 8.8 DatingApp (Orchestrator)

java

```

class DatingApp {
    private List<User> users = new ArrayList<>();
    private List<ChatRoom> chatRooms = new ArrayList<>();

    public void addUser(User u) { users.add(u); }

    public void swipe(String userId, String targetUserId, Swipe action) {
        User user = findUser(userId);
        User target = findUser(targetUserId);
        if (user == null || target == null) return;

        user.swipe(targetUserId, action);

        if (action == Swipe.RIGHT && target.hasLiked(userId)) {
            // Match! Compute score
            Matcher matcher = MatcherFactory.createMatcher(MatcherType.LOCATION);
            double score = matcher.calculateMatchScore(user, target);

            // Create chat room
            List<String> participants = Arrays.asList(userId, targetUserId);
            ChatRoom room = new ChatRoom("room_" + userId + "_" + targetUserId, participa
            nts);

            chatRooms.add(room);

            String msg = "You matched with " + target.getProfile().getName() + "! Score:
            " + score;

```

```

        NotificationService.getInstance().notifyUser(userId, msg);
        NotificationService.getInstance().notifyUser(targetUserId, "New match! Score:
" + score);
    }
}

private User findUser(String id) {
    return users.stream().filter(u -> u.getUserId().equals(id)).findFirst().orElse(nu
11);
}
}

```

## 9. Code — Python

python

```

from __future__ import annotations
from enum import Enum
from abc import ABC, abstractmethod
from math import radians, sin, cos, sqrt, atan2
from datetime import datetime
from typing import List, Dict
import threading

```

# ----- Enums -----

```

class Gender(Enum):
    MALE = "MALE"
    FEMALE = "FEMALE"
    NON_BINARY = "NON_BINARY"
    OTHER = "OTHER"

```

```

class Swipe(Enum):
    LEFT = "LEFT"
    RIGHT = "RIGHT"

```

```

class MatcherType(Enum):
    LOCATION = "LOCATION"
    INTEREST = "INTEREST"
    BASIC = "BASIC"

```

# ----- Observer Pattern -----

```

class NotificationObserver(ABC):
    @abstractmethod

```

```

def update(self, message: str): ...

class UserNotificationObserver(NotificationObserver):
    def update(self, message: str):
        print(f"Notification for user: {message}")

class NotificationService:
    _instance = None
    _lock = threading.Lock()

    def __init__(self):
        self._observers: Dict[str, NotificationObserver] = {}

    @classmethod
    def get_instance(cls):
        with cls._lock:
            if cls._instance is None:
                cls._instance = NotificationService()
            return cls._instance

    def register_observer(self, user_id: str, observer: NotificationObserver):
        self._observers[user_id] = observer

    def remove_observer(self, user_id: str):
        self._observers.pop(user_id, None)

    def notify_user(self, user_id: str, message: str):
        obs = self._observers.get(user_id)
        if obs: obs.update(message)

    def notify_all(self, message: str):
        for obs in self._observers.values(): obs.update(message)

# ----- Basic Models -----
class Interest:
    def __init__(self, name: str, category: str):
        self.name, self.category = name, category

class Location:
    def __init__(self, lat: float, lon: float):
        self.latitude, self.longitude = lat, lon

    def distance_in_km(self, other: "Location") -> float:
        R = 6371.0
        dlat = radians(other.latitude - self.latitude)
        dlon = radians(other.longitude - self.longitude)
        a = sin(dlat/2)**2 + cos(radians(self.latitude)) * cos(radians(other.latitude)) *

```

```

sin(dlon/2)**2
    c = 2 * atan2(sqrt(a), sqrt(1-a))
    return R * c

```

```

class UserProfile:
    def __init__(self, name: str, age: int, gender: Gender, location: Location):
        self.name, self.age, self.gender, self.location = name, age, gender, location
        self.bio = ""
        self.pictures: List[str] = []
        self.interests: List[Interest] = []

    def add_interest(self, i: Interest): self.interests.append(i)
    def remove_interest(self, i: Interest):
        if i in self.interests: self.interests.remove(i)

    def display(self):
        print(f"{self.name}, {self.age}, {self.gender.value} - {self.bio}")
        for i in self.interests: print(f"  Interest: {i.name}")
        print(f"  Location: {self.location.latitude},{self.location.longitude}")

```

```

class UserPreferences:
    def __init__(self):
        self.min_age, self.max_age, self.max_distance = 18, 100, 5
        self.interested_in: List[Gender] = []
        self.interests: List[str] = []

    def is_interested_in(self, g: Gender) -> bool:
        return g in self.interested_in

```

# ----- User -----

```

class User:
    def __init__(self, user_id: str, profile: UserProfile, prefs: UserPreferences):
        self.user_id = user_id
        self.profile = profile
        self.preferences = prefs
        self.notification_observer = UserNotificationObserver()
        self.swipe_history: Dict[str, Swipe] = {}
        NotificationService.get_instance().register_observer(user_id, self.notification_o
bserver)

    def swipe(self, target_id: str, action: Swipe):
        self.swipe_history[target_id] = action

    def has_liked(self, target_id: str) -> bool:
        return self.swipe_history.get(target_id) == Swipe.RIGHT

```

```

def has_disliked(self, target_id: str) -> bool:
    return self.swipe_history.get(target_id) == Swipe.LEFT

def has_interacted_with(self, target_id: str) -> bool:
    return target_id in self.swipe_history

def is_interested_in(self, target_id: str) -> bool:
    return target_id in self.swipe_history

def display_profile(self): self.profile.display()

def get_location(self) -> Location: return self.profile.location

# ----- Location Strategy -----
class NearByStrategy(ABC):
    @abstractmethod
    def find_near_by_users(self, loc: Location, max_dist: float, all_users: List[User]) -
    > List[User]: ...

class BasicNearByStrategy(NearByStrategy):
    def find_near_by_users(self, loc, max_dist, all_users):
        return [u for u in all_users if loc.distance_in_km(u.get_location()) <= max_dist]

class LocationService:
    _instance = None
    _lock = threading.Lock()

    def __init__(self):
        self.near_by_strategy: NearByStrategy = BasicNearByStrategy()

    @classmethod
    def get_instance(cls):
        with cls._lock:
            if cls._instance is None:
                cls._instance = LocationService()
            return cls._instance

    def set_strategy(self, s: NearByStrategy): self.near_by_strategy = s

    def find_near_by_users(self, loc, max_dist, all_users):
        return self.near_by_strategy.find_near_by_users(loc, max_dist, all_users)

# ----- Chat -----
class Message:
    def __init__(self, sender_id: str, content: str):

```

```

        self.sender_id, self.content = sender_id, content
        self.timestamp = datetime.now()

    def get_formatted_time(self): return self.timestamp.strftime("%Y-%m-%d %H:%M:%S")

class ChatRoom:
    def __init__(self, chat_id: str, participant_ids: List[str]):
        self.chat_room_id = chat_id
        self.participant_ids = participant_ids
        self.messages: List[Message] = []

    def add_message(self, sender_id: str, content: str):
        self.messages.append(Message(sender_id, content))

    def has_participant(self, user_id: str) -> bool:
        return user_id in self.participant_ids

    def display_chat(self):
        print(f"--- Chat Room {self.chat_room_id} ---")
        for m in self.messages:
            print(f"{m.get_formatted_time()} | {m.sender_id}: {m.content}")

# ----- Matcher (Chain of Responsibility) -----
class Matcher(ABC):
    @abstractmethod
    def calculate_match_score(self, u1: User, u2: User) -> float: ...

class BasicDetailsMatcher(Matcher):
    def calculate_match_score(self, u1, u2):
        shared = sum(1 for i in u1.profile.interests if i in u2.profile.interests)
        total = max(1, len(u1.profile.interests))
        return (100.0 * shared) / total

class InterestMatcher(Matcher):
    def __init__(self, next_matcher: Matcher):
        self.next = next_matcher

    def calculate_match_score(self, u1, u2):
        next_score = self.next.calculate_match_score(u1, u2)
        shared = sum(1 for i in u1.preferences.interests if i in u2.preferences.interests)
        my_score = (shared * 100.0) / max(1, len(u1.preferences.interests))
        return (next_score + my_score) / 2.0

class LocationMatcher(Matcher):
    def __init__(self, next_matcher: Matcher):
        self.next = next_matcher

```

```

def calculate_match_score(self, u1, u2):
    next_score = self.next.calculate_match_score(u1, u2)
    dist = u1.get_location().distance_in_km(u2.get_location())
    my_score = max(0, 100 - dist * 10)
    return (next_score + my_score) / 2.0

class MatcherFactory:
    @staticmethod
    def create_matcher(t: MatcherType) -> Matcher:
        if t == MatcherType.LOCATION:
            return LocationMatcher(InterestMatcher(BasicDetailsMatcher()))
        elif t == MatcherType.INTEREST:
            return InterestMatcher(BasicDetailsMatcher())
        else:
            return BasicDetailsMatcher()

# ----- Orchestrator -----
class DatingApp:
    def __init__(self):
        self.users: List[User] = []
        self.chat_rooms: List[ChatRoom] = []

    def add_user(self, u: User): self.users.append(u)

    def _find_user(self, uid: str) -> User | None:
        return next((u for u in self.users if u.user_id == uid), None)

    def swipe(self, user_id: str, target_id: str, action: Swipe):
        user = self._find_user(user_id)
        target = self._find_user(target_id)
        if not user or not target: return

        user.swipe(target_id, action)

        if action == Swipe.RIGHT and target.has_liked(user_id):
            matcher = MatcherFactory.create_matcher(MatcherType.LOCATION)
            score = matcher.calculate_match_score(user, target)

            room = ChatRoom(f"room_{user_id}_{target_id}", [user_id, target_id])
            self.chat_rooms.append(room)

            NotificationService.get_instance().notify_user(
                user_id, f"You matched with {target.profile.name}! Score: {score:.2f}")
            NotificationService.get_instance().notify_user(
                target_id, f"New match! Score: {score:.2f}")

```

```
# ----- Demo -----
if __name__ == "__main__":
    loc1 = Location(28.6139, 77.2090) # Delhi
    loc2 = Location(28.6200, 77.2100) # Nearby
    loc3 = Location(19.0760, 72.8777) # Mumbai (far)

    p1 = UserProfile("Alice", 25, Gender.FEMALE, loc1)
    p1.add_interest(Interest("Coding", "Tech"))
    p2 = UserProfile("Bob", 27, Gender.MALE, loc2)
    p2.add_interest(Interest("Coding", "Tech"))
    p3 = UserProfile("Carol", 30, Gender.FEMALE, loc3)

    prefs = UserPreferences()
    prefs.interested_in = [Gender.MALE, Gender.FEMALE]
    prefs.interests = ["Coding"]

    alice = User("u1", p1, prefs)
    bob = User("u2", p2, prefs)
    carol = User("u3", p3, prefs)

    app = DatingApp()
    app.add_user(alice); app.add_user(bob); app.add_user(carol)

    app.swipe("u1", "u2", Swipe.RIGHT)
    app.swipe("u2", "u1", Swipe.RIGHT) # Match!

    # Near-by search
    nearby = LocationService.get_instance().find_near_by_users(loc1, 5.0, app.users)
    print("Nearby users:", [u.profile.name for u in nearby])
```

## 10. Key Design Takeaways

1. **Top-Down works when many small objects exist** — start with `User`, then discover `Interest`, `Gender`, `Location`, `Preferences`, etc.
2. **Observer Pattern** for notifications — modified to support `notifyUser(userId)` (not just `notifyAll`).
3. **Singleton** for `NotificationService` and `LocationService`.
4. **Strategy Pattern** for both near-by search and matching (scalable).
5. **Chain of Responsibility (modified)** for match scoring — fixed chain `Location` → `Interest` → `Basic`, each averaging with next.
6. **Factory Pattern** for `MatcherFactory` to encapsulate chain construction.

- 7. **Composition (◆)** used for User-Profile , User-Preferences , ChatRoom-Message , UserProfile-Location .
- 8. **Principle of Least Knowledge** — User.displayProfile() delegates to profile.display() .
- 9. **Tight coupling is acceptable** when the type will never change (e.g., User → UserNotificationObserver ).
- 10. **Haversine Formula** used for real distance in km.

11. Quick Revision Table

| Requirement      | Solved By                                                      |
|------------------|----------------------------------------------------------------|
| Swipe L/R        | User.swipe() , Swipe enum                                      |
| Profile setup    | UserProfile , Interest , Gender , Location                     |
| Preferences      | UserPreferences                                                |
| Chat after match | ChatRoom , Message                                             |
| Near-by profiles | LocationService + NearByStrategy                               |
| Notifications    | Observer Pattern ( NotificationService )                       |
| Match scoring    | Chain of Responsibility ( Matcher subclasses) + MatcherFactory |
| Orchestration    | DatingApp                                                      |

Would you like me to also produce a **clean UML diagram in Mermaid syntax** or a **modular multi-file project structure** for this design?



हां जी। सो वेलकम बैक कोडर ...  
TXT 294.74KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

# LLD: Builder Design Pattern — Complete Deep Notes

## 1. Introduction

The **Builder Design Pattern** is one of the most used patterns in real-world applications. Whenever **object creation** is involved — especially for **complex objects with many fields** — the industry prefers Builder over the `new` keyword.

It is heavily asked in interviews because:

- It solves **multiple real problems** at once.
- It has **three variants** (Simple Builder, Builder with Director, Step Builder), each adding more power.

**One-line definition:**

Builder separates the **construction** of a complex object from its **representation**, so the same construction process can create different representations.

## 2. The Motivating Example — HTTP Request

Every HTTP request (client → server, or microservice → microservice) contains many fields:

| Field                              | Required?  |
|------------------------------------|------------|
| url                                | ✔ Required |
| method (GET/POST/PUT/DELETE/PATCH) | ✔ Required |
| headers (Map<String,String>)       | Optional   |
| queryParams (Map<String,String>)   | Optional   |
| body (JSON)                        | Optional   |

| Field                         | Required? |
|-------------------------------|-----------|
| timeout (seconds, default 60) | Optional  |

We want a class `HttpRequest` with an `execute()` method that fires the actual HTTP call.

## Naive approach (without Builder)

```
java
```

```
HttpRequest req = new HttpRequest(url, method, headers, queryParams, body, timeout);
```

This leads to **four major problems** discussed below.

## 3. The Four Problems (Why Builder is Needed)

### Problem 1: Constructor Overloading / Telescoping Constructor

When a class has many fields, we end up writing **multiple overloaded constructors**:

```
java
```

```
HttpRequest(String url)
HttpRequest(String url, String method)
HttpRequest(String url, String method, Map headers)
HttpRequest(String url, String method, Map headers, Map queryParams)
HttpRequest(String url, String method, Map headers, Map queryParams, String body)
HttpRequest(String url, String method, Map headers, Map queryParams, String body, int timeout)
```

This is called **Telescoping Constructor** problem (a.k.a. Constructor Overloading).

**Why bad?** Explosion of constructors; unreadable call sites; hard to maintain.

### Problem 2: Mutability (Setters break immutability)

To avoid overloaded constructors, we add **setters**:

```
java
```

```
req.setUrl(...);  
req.setMethod(...);  
req.setBody(...);
```

But now the object is **mutable** — anyone can change it after creation. For objects like `HttpRequest`, we often want **immutability** (once built, cannot change).

**Why bad?** Thread-safety issues, unexpected side-effects, breaks immutability guarantee.

### Problem 3: Inconsistent State

If we keep a **minimal constructor** (only required fields) + **setters** for optional fields, nothing stops the client from calling `execute()` **before setting all required fields**.

```
java  
  
HttpRequest req = new HttpRequest(url, method); // required fields set  
req.execute(); // body missing → runtime error
```

**Why bad?** No compile-time check. Runtime failures are the worst kind.

### Problem 4: Scattered Validation

Validation (e.g., `url != null`, `body != null`) has to be duplicated **at every place** the object is used.

**Why bad?** Code duplication, easy to forget, hard to maintain.

### Summary of Problems

| # | Problem                 | Cause                        |
|---|-------------------------|------------------------------|
| 1 | Constructor Telescoping | Many overloaded constructors |
| 2 | Mutability              | Public setters               |

| # | Problem              | Cause                             |
|---|----------------------|-----------------------------------|
| 3 | Inconsistent State   | No enforcement of required fields |
| 4 | Scattered Validation | Validation at every call site     |

Note: These problems **contradict** each other. Fixing one introduces another — which is exactly why we need a **new pattern**.

## 4. The `this` Keyword (Prerequisite)

`this` refers to the **current object's reference**. In Builder, we use it heavily:

```
java
```

```
return this; // returns the builder itself, enabling method chaining
```

Also used in constructors:

```
java
```

```
this.url = url; // disambiguate field vs parameter
```

In Java, unlike C++, we don't need the `friend` keyword — **package-private** members already allow access within the same package.

## 5. Pattern 1: Simple Builder

### 5.1 Structure

1. Make **constructor of target class private**.
2. Make **Builder a friend class (C++)** or **package-private access (Java)** so it can access private members.
3. Builder holds a **reference to the target object**.
4. Builder exposes `withXxx()` **methods** that:
  - Set the field on the target.

- Return `this` (the builder) → enables **fluent chaining**.

#### 5. Final `build()` method:

- Runs validations.
- Returns the fully-constructed target object.

## 5.2 Terminology

| Term                 | Meaning                                                                                                              |
|----------------------|----------------------------------------------------------------------------------------------------------------------|
| Intermediate methods | <code>withUrl()</code> , <code>withMethod()</code> , <code>withHeader()</code> , etc. —<br>return builder, chainable |
| Terminating method   | <code>build()</code> — returns the target object, ends the chain                                                     |

## 5.3 Benefits

- ✓ Removes constructor overloading
- ✓ Centralizes validation
- ✓ Enables immutability (no setters on target)
- ✓ Removes inconsistent state (`build()` enforces required fields)
- ✓ Highly readable, fluent API

## 5.4 Real-Life Analogy

Building a house step-by-step: foundation → walls → doors → roof → windows.

Or ordering a pizza: crust → sauce → toppings → cheese.

## 5.5 Java Code — Simple Builder

```
java

import java.util.*;

// ===== Target =====
public class HttpRequest {
    private final String url;           // required
    private final String method;       // required
    private final Map<String, String> headers; // optional
    private final Map<String, String> queryParams; // optional
}
```

```
private final String body; // optional
private final int timeout; // optional

// Private constructor - only Builder can create
private HttpRequest(Builder b) {
    this.url = b.url;
    this.method = b.method;
    this.headers = b.headers;
    this.queryParams = b.queryParams;
    this.body = b.body;
    this.timeout = b.timeout;
}

public void execute() {
    System.out.println("Executing " + method + " request to " + url);
    if (queryParams != null && !queryParams.isEmpty())
        System.out.println(" Query Params: " + queryParams);
    if (headers != null && !headers.isEmpty())
        System.out.println(" Headers: " + headers);
    if (body != null && !body.isEmpty())
        System.out.println(" Body: " + body);
    System.out.println(" Timeout: " + timeout + "s");
    System.out.println("Request Executed Successfully");
}

// ===== Builder =====
public static class Builder {
    private String url;
    private String method = "GET";
    private Map<String, String> headers = new HashMap<>();
    private Map<String, String> queryParams = new HashMap<>();
    private String body;
    private int timeout = 30;

    public Builder withUrl(String url) {
        this.url = url;
        return this;
    }

    public Builder withMethod(String method) {
        this.method = method;
        return this;
    }

    public Builder withHeader(String key, String value) {
        this.headers.put(key, value);
        return this;
    }
}
```

```

public Builder withQueryParam(String key, String value) {
    this.queryParams.put(key, value);
    return this;
}

public Builder withBody(String body) {
    this.body = body;
    return this;
}

public Builder withTimeout(int timeout) {
    this.timeout = timeout;
    return this;
}

// Terminating method
public HttpRequest build() {
    // Centralized validation
    if (url == null || url.isEmpty())
        throw new RuntimeException("URL is empty");
    if (method == null || method.isEmpty())
        throw new RuntimeException("Method is empty");
    return new HttpRequest(this);
}
}

// ===== Client =====
class Demo {
    public static void main(String[] args) {
        HttpRequest req = new HttpRequest.Builder()
            .withUrl("https://api.example.com/users")
            .withMethod("POST")
            .withHeader("Content-Type", "application/json")
            .withQueryParam("page", "1")
            .withBody("{\"name\":\"Alice\"}")
            .withTimeout(60)
            .build();

        req.execute();
    }
}

```

## 5.6 Python Code — Simple Builder

python

```
from typing import Dict, Optional
```

```
class HttpRequest:
```

```
    """Target class – fields are effectively immutable after build()."""
```

```
    def __init__(self, builder: "HttpRequest.Builder"):
```

```
        self.url = builder.url
```

```
        self.method = builder.method
```

```
        self.headers = builder.headers
```

```
        self.query_params = builder.query_params
```

```
        self.body = builder.body
```

```
        self.timeout = builder.timeout
```

```
    def execute(self):
```

```
        print(f"Executing {self.method} request to {self.url}")
```

```
        if self.query_params:
```

```
            print(f"  Query Params: {self.query_params}")
```

```
        if self.headers:
```

```
            print(f"  Headers: {self.headers}")
```

```
        if self.body:
```

```
            print(f"  Body: {self.body}")
```

```
        print(f"  Timeout: {self.timeout}s")
```

```
        print("Request Executed Successfully")
```

```
# ===== Builder =====
```

```
class Builder:
```

```
    def __init__(self):
```

```
        self.url = None
```

```
        self.method = "GET"
```

```
        self.headers = {}
```

```
        self.query_params = {}
```

```
        self.body = None
```

```
        self.timeout = 30
```

```
    def with_url(self, url: str) -> "HttpRequest.Builder":
```

```
        self.url = url
```

```
        return self
```

```
    def with_method(self, method: str) -> "HttpRequest.Builder":
```

```
        self.method = method
```

```
        return self
```

```
    def with_header(self, key: str, value: str) -> "HttpRequest.Builder":
```

```
        self.headers[key] = value
```

```
        return self
```

```

def with_query_param(self, key: str, value: str) -> "HttpRequest.Builder":
    self.query_params[key] = value
    return self

def with_body(self, body: str) -> "HttpRequest.Builder":
    self.body = body
    return self

def with_timeout(self, timeout: int) -> "HttpRequest.Builder":
    self.timeout = timeout
    return self

def build(self) -> "HttpRequest":
    if not self.url:
        raise RuntimeError("URL is empty")
    if not self.method:
        raise RuntimeError("Method is empty")
    return HttpRequest(self)

# ===== Client =====
if __name__ == "__main__":
    req = (HttpRequest.Builder()
           .with_url("https://api.example.com/users")
           .with_method("POST")
           .with_header("Content-Type", "application/json")
           .with_query_param("page", "1")
           .with_body('{"name": "Alice"}')
           .with_timeout(60)
           .build())

    req.execute()

```

## 6. Pattern 2: Builder with Director

### 6.1 Idea

Many clients **repeatedly** want the same **default configuration** — e.g., "give me a GET request" or "give me a JSON POST request". Instead of forcing every client to chain `withUrl().withMethod("GET")...`, we introduce a **Director** that encapsulates **pre-built templates**.

## 6.2 Director's Role

- Holds **static methods** (so no instance needed).
- Each method knows the **recipe** for a common template.
- Returns a fully-built target directly.

## 6.3 Java Code — Director

java

```
public class HttpRequestDirector {

    public static HttpRequest createGetRequest(String url) {
        return new HttpRequest.Builder()
            .withUrl(url)
            .withMethod("GET")
            .build();
    }

    public static HttpRequest createJsonPostRequest(String url, String body) {
        return new HttpRequest.Builder()
            .withUrl(url)
            .withMethod("POST")
            .withHeader("Content-Type", "application/json")
            .withHeader("Accept", "application/json")
            .withBody(body)
            .build();
    }
}

// Client
HttpRequest getReq = HttpRequestDirector.createGetRequest("https://api.example.com/health");
HttpRequest postReq = HttpRequestDirector.createJsonPostRequest("https://api.example.com/users", "{ \"n\": 1 }");
```

## 6.4 Python Code — Director

python

```
class HttpRequestDirector:

    @staticmethod
    def create_get_request(url: str) -> HttpRequest:
        return HttpRequest.Builder().with_url(url).with_method("GET").build()
```

```
@staticmethod
def create_json_post_request(url: str, body: str) -> HttpRequest:
    return (HttpRequest.Builder()
            .with_url(url)
            .with_method("POST")
            .with_header("Content-Type", "application/json")
            .with_header("Accept", "application/json")
            .with_body(body)
            .build())
```

## 6.5 Benefit

- ✓ **Reusability** — one method call replaces a long chain.
- ✓ **Consistent templates** across the codebase.

## 7. Pattern 3: Step Builder

### 7.1 Motivation

Simple Builder lets the client call **any** `withXxx()` **in any order** — which is flexible but **error-prone**. Sometimes we want:

- **Strict order:** `url` → `method` → `body` → `headers`
- **No skipping of required fields**

Real-life examples:

- **Pizza shop:** `crust` → `sauce` → `toppings` → `cheese` (fixed order).
- **House construction:** `foundation` → `walls` → `roof` → `doors` → `windows`.

### 7.2 Core Idea

1. Split each required field into its **own interface**:

- `UrlStep` (only `withUrl` )
- `MethodStep` (only `withMethod` )
- `BodyStep` (only `withBody` )
- `OptionalStep` (all remaining optional methods + `build` )

## 2. Chain them via **return types**:

- `UrlStep.withUrl()` returns `MethodStep`
- `MethodStep.withMethod()` returns `BodyStep`
- `BodyStep.withBody()` returns `OptionalStep`
- `OptionalStep.withHeader()` returns `OptionalStep`
- `OptionalStep.build()` returns `HttpRequest`

3. **StepBuilder** implements **all** these interfaces (multiple inheritance in C++, implements in Java).

4. Entry point: `HttpRequest.getBuilder()` returns a `UrlStep`.

## 7.3 Why Multiple Inheritance?

Only Step Builder uses multiple inheritance in the entire design pattern catalog. It's a **trade-off**: we sacrifice simplicity for **compile-time enforcement** of the build order.

## 7.4 Compile-Time Errors

If a user tries:

```
java
```

```
HttpRequest.getBuilder().withUrl("...").withBody("...") // ❌
```

They get "**Method withBody not found in MethodStep**" — a **compile-time error**. This is far better than a **runtime error**.

## 7.5 Java Code — Step Builder

```
java
```

```
// ===== Step Interfaces =====
interface UrlStep { MethodStep withUrl(String url); }
interface MethodStep { BodyStep withMethod(String method); }
interface BodyStep { OptionalStep withBody(String body); }

interface OptionalStep {
    OptionalStep withHeader(String key, String value);
    OptionalStep withQueryParam(String key, String value);
    OptionalStep withTimeout(int timeout);
    HttpRequest build();
}
```

```

    }

// ===== Target =====
class HttpRequest {
    private final String url, method, body;
    private final Map<String,String> headers, queryParams;
    private final int timeout;

    private HttpRequest(StepBuilder b) {
        this.url      = b.url;
        this.method    = b.method;
        this.body      = b.body;
        this.headers   = b.headers;
        this.queryParams = b.queryParams;
        this.timeout   = b.timeout;
    }

    public static UrlStep getBuilder() {
        return new StepBuilder();
    }

    public void execute() {
        System.out.println("Executing " + method + " " + url);
        if (!queryParams.isEmpty()) System.out.println("  Query: " + queryParams);
        if (!headers.isEmpty())     System.out.println("  Headers: " + headers);
        if (body != null)           System.out.println("  Body: " + body);
        System.out.println("  Timeout: " + timeout + "s");
        System.out.println("Request Executed Successfully");
    }

// ===== StepBuilder =====
    static class StepBuilder implements UrlStep, MethodStep, BodyStep, OptionalStep {
        private String url, method, body;
        private Map<String,String> headers      = new HashMap<>();
        private Map<String,String> queryParams = new HashMap<>();
        private int timeout = 30;

        // UrlStep
        @Override
        public MethodStep withUrl(String url) {
            this.url = url;
            return this;
        }

        // MethodStep
        @Override
        public BodyStep withMethod(String method) {
            this.method = method;
        }
    }

```

```

        return this;
    }

    // BodyStep
    @Override
    public OptionalStep withBody(String body) {
        this.body = body;
        return this;
    }

    // OptionalStep
    @Override
    public OptionalStep withHeader(String key, String value) {
        this.headers.put(key, value);
        return this;
    }

    @Override
    public OptionalStep withQueryParam(String key, String value) {
        this.queryParams.put(key, value);
        return this;
    }

    @Override
    public OptionalStep withTimeout(int timeout) {
        this.timeout = timeout;
        return this;
    }

    // Terminating method
    @Override
    public HttpRequest build() {
        if (url == null || url.isEmpty())
            throw new RuntimeException("URL is empty");
        if (method == null || method.isEmpty())
            throw new RuntimeException("Method is empty");
        return new HttpRequest(this);
    }
}

// ===== Client =====
class DemoStep {
    public static void main(String[] args) {
        HttpRequest req = HttpRequest.getBuilder()
            .withUrl("https://api.example.com/users")
            .withMethod("POST")
            .withBody("{\"name\":\"Alice\"}")

```

```

        .withHeader("Content-Type", "application/json")
        .withTimeout(60)
        .build();

req.execute();

// ❌ Compile-time error if you try:
// HttpRequest.getBuilder().withUrl("x").withBody("y");
//    -> withBody not found in MethodStep
    }
}

```

## 7.6 Python Code — Step Builder (Simulated)

Python doesn't have interfaces, but we can **simulate** step builder using **separate wrapper classes** that only expose the allowed next method.

python

```

from typing import Dict

class HttpRequest:
    def __init__(self, builder):
        self.url = builder.url
        self.method = builder.method
        self.body = builder.body
        self.headers = builder.headers
        self.query_params = builder.query_params
        self.timeout = builder.timeout

    @staticmethod
    def get_builder() -> "UrlStep":
        return UrlStep()

    def execute(self):
        print(f"Executing {self.method} {self.url}")
        if self.query_params: print(f"  Query: {self.query_params}")
        if self.headers:      print(f"  Headers: {self.headers}")
        if self.body:         print(f"  Body: {self.body}")
        print(f"  Timeout: {self.timeout}s")
        print("Request Executed Successfully")

# ---- Shared storage ----
class _Storage:
    def __init__(self):

```

```
self.url = None
self.method = None
self.body = None
self.headers: Dict[str, str] = {}
self.query_params: Dict[str, str] = {}
self.timeout = 30

# ---- Step 1: URL (entry) ----
class UrlStep:
    def __init__(self):
        self._s = _Storage()

    def with_url(self, url: str) -> "MethodStep":
        self._s.url = url
        return MethodStep(self._s)

# ---- Step 2: Method ----
class MethodStep:
    def __init__(self, storage: _Storage):
        self._s = storage

    def with_method(self, method: str) -> "BodyStep":
        self._s.method = method
        return BodyStep(self._s)

# ---- Step 3: Body ----
class BodyStep:
    def __init__(self, storage: _Storage):
        self._s = storage

    def with_body(self, body: str) -> "OptionalStep":
        self._s.body = body
        return OptionalStep(self._s)

# ---- Step 4: Optional fields + build ----
class OptionalStep:
    def __init__(self, storage: _Storage):
        self._s = storage

    def with_header(self, key: str, value: str) -> "OptionalStep":
        self._s.headers[key] = value
        return self

    def with_query_param(self, key: str, value: str) -> "OptionalStep":
```

```

        self._s.query_params[key] = value
    return self

def with_timeout(self, timeout: int) -> "OptionalStep":
    self._s.timeout = timeout
    return self

def build(self) -> HttpRequest:
    if not self._s.url:
        raise RuntimeError("URL is empty")
    if not self._s.method:
        raise RuntimeError("Method is empty")
    return HttpRequest(self._s)

# ---- Client ----
if __name__ == "__main__":
    req = (HttpRequest.get_builder()
           .with_url("https://api.example.com/users")
           .with_method("POST")
           .with_body('{"name":"Alice"}')
           .with_header("Content-Type", "application/json")
           .with_timeout(60)
           .build())

    req.execute()

# ❌ Python can't enforce at compile time, but the API surface
#    simply won't expose `with_body` on MethodStep, so calling
#    `.with_url(...).with_body(...)` raises AttributeError at runtime.

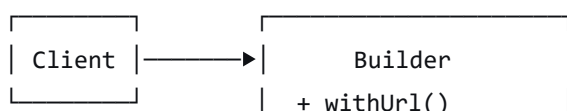
```

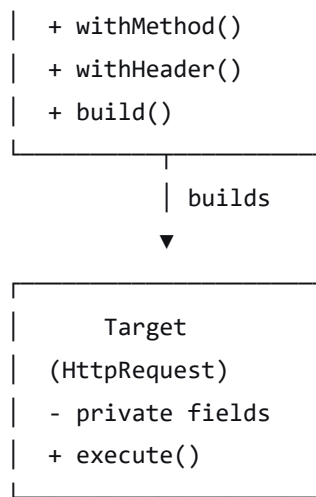
In Python, the "compile-time" guarantee becomes an **AttributeError at runtime**, because the method simply doesn't exist on the returned object. This is still a big improvement over silently allowing wrong usage.

## 8. UML Diagrams

### 8.1 Simple Builder UML

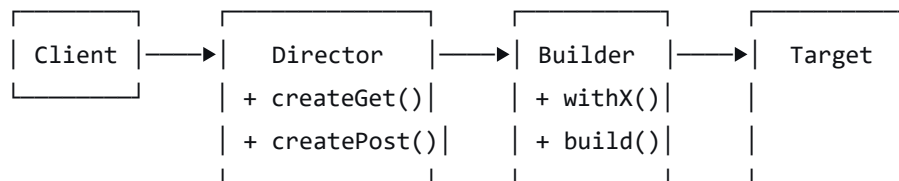
text





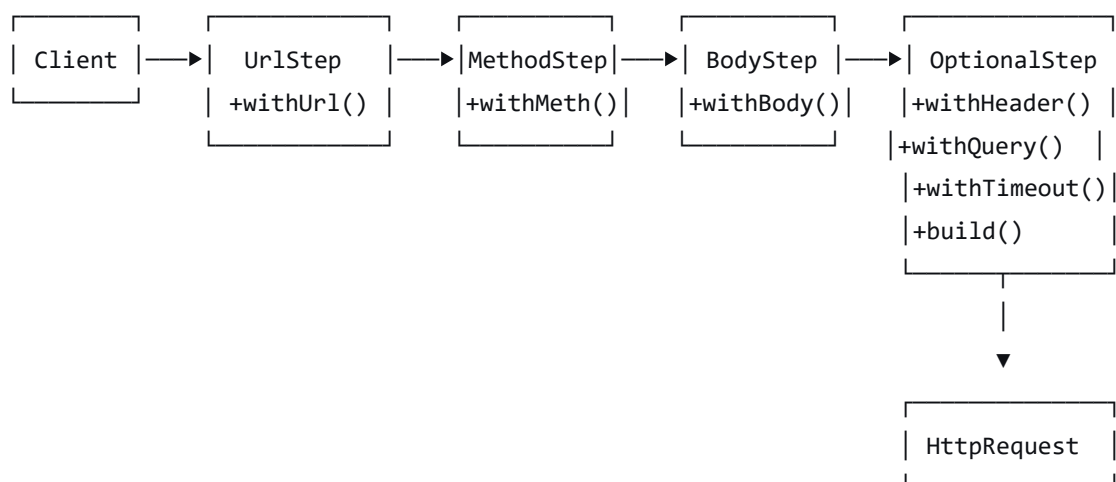
## 8.2 Builder with Director UML

text



## 8.3 Step Builder UML

text



StepBuilder implements ALL four step interfaces (multiple inheritance).

## 9. Comparison Table

| Aspect                          | Simple Builder                      | Builder + Director                  | Step Builder                            |
|---------------------------------|-------------------------------------|-------------------------------------|-----------------------------------------|
| Enforces order                  | ✗                                   | ✗                                   | ✓ (compile-time)                        |
| Enforces all required set       | Via <code>build()</code> validation | Via <code>build()</code> validation | Via compile-time + <code>build()</code> |
| Reusable templates              | ✗                                   | ✓                                   | ✗                                       |
| Flexibility for optional fields | ✓                                   | ✓                                   | ✓ (in <code>OptionalStep</code> )       |
| Complexity                      | Low                                 | Medium                              | High                                    |
| Uses multiple inheritance       | ✗                                   | ✗                                   | ✓ (only pattern that does)              |
| Best for                        | General complex objects             | Repeated configurations             | Strict construction workflows           |

## 10. Key Takeaways

1. **Builder solves 4 problems simultaneously:** telescoping constructors, mutability, inconsistent state, scattered validation.
2. **Simple Builder** — private constructor + fluent `withXxx()` chain + `build()` terminating method.
3. **Director** — pre-built templates for common configurations; purely a convenience layer.
4. **Step Builder** — strict step-by-step construction; uses **multiple inheritance** (unique in design patterns); converts runtime errors into **compile-time errors**.
5. **Terminating method** = `build()` . **Intermediate methods** = all `withXxx()` .
6. `return this` is the secret behind fluent chaining.
7. Builder is the **industry default** for creating complex objects — prefer it over `new` in production code.
8. Best practice: **make fields** `final` **in the target** and only set them in the constructor invoked by the Builder.

## 11. Quick Revision Cheatsheet

| Question                                 | Answer                                                                         |
|------------------------------------------|--------------------------------------------------------------------------------|
| Definition                               | Separate construction from representation                                      |
| Entry method                             | <code>Builder()</code> OR <code>getBuilder()</code>                            |
| Terminating method                       | <code>build()</code>                                                           |
| Intermediate methods                     | <code>withUrl</code> , <code>withMethod</code> , <code>withHeader</code> , ... |
| Why <code>return this</code> ?           | Enable method chaining                                                         |
| Why private constructor?                 | Force use of Builder                                                           |
| Why friend/package-private?              | Builder needs access to private fields                                         |
| Which pattern uses multiple inheritance? | Step Builder (only one)                                                        |
| Benefit of Step Builder?                 | Compile-time enforcement of build order                                        |
| Benefit of Director?                     | Reusable templates for common configs                                          |

Would you like me to also produce:

- A **side-by-side comparison** of Builder vs Factory vs Prototype?
- A **real-world Spring Boot / Lombok** `@Builder` example?
- A **modular multi-file Java project structure** for the above code?



हां जी तो करें स्टार्ट। वेलकम बै...  
TXT 112.61KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also



## LLD: Iterator Design Pattern — Complete Deep Notes

# 1. Introduction

The **Iterator Pattern** is one of the most widely used design patterns — often used *without realizing* it's a design pattern.

If you've used `for (auto x : list)` in C++, `Iterator` in Java, or `for x in list` in Python, you've used iterators. But **why** do we need them? Why not just use a `for` loop and traverse any data structure directly?

This lecture answers:

- **Why** iterators exist
- **What** problem they solve (Single Responsibility Principle)
- **How** to design your own iterator (a common LLD interview question)

## 2. The Core Problem — Why Iterators?

### 2.1 Different data structures, different traversals

Consider traversing:

- **Linked List** → need a temp pointer, loop `while (temp != null) temp = temp->next`
- **Binary Tree** → in-order, pre-order, post-order, level-order traversal
- **Playlist (Vector/Map)** → simple `for` loop or map iteration
- **Map** → `for (auto& [k,v] : map)`

Each data structure has a **different traversal logic**.

### 2.2 The trap: mixing storage logic with traversal logic

Imagine a `Playlist` class:

java

```
class Playlist {  
    private Vector<Song> songs;  
  
    public void playEntirePlaylist() {  
        for (int i = 0; i < songs.size(); i++) {  
            play(songs.get(i));  
        }  
    }  
}
```

```
}  
}  
}
```

Now if you change `Vector`  $\rightarrow$  `LinkedList` :

- `songs.get(i)` no longer works (no random access).
- You must rewrite `playEntirePlaylist` with a temp pointer.
- **Every method** that relies on traversal (e.g., `getSongById` , `getSongByName` ) must change.
- This **violates Open/Closed Principle** and **Single Responsibility Principle**.

The Playlist's job is to **manage songs** (add, remove, get-by-name). It should **not** know **how** to traverse its internal storage.

## 2.3 The solution: separate traversal into an Iterator

- The **Iterator** knows **how** to traverse a specific data structure.
- The **Aggregate** (Playlist, LinkedList, Tree) just exposes a `getIterator()` method.
- The client uses `hasNext()` / `next()` — **never** touches the underlying storage.

### Benefits:

- ✓ Follows **Single Responsibility Principle** — storage vs traversal.
- ✓ Follows **Open/Closed Principle** — change storage, swap iterator.
- ✓ Client is decoupled from internal representation (encapsulation).
- ✓ Works uniformly across LinkedList, Tree, Playlist, etc.

## 3. The Pattern Structure

Two hierarchies:

| Hierarchy                   | Role                           | Example                                                                                                                         |
|-----------------------------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Iterator</b>             | Abstract + Concrete iterators  | <code>Iterator</code> , <code>LinkedListIterator</code> ,<br><code>BinaryTreeIterator</code> ,<br><code>PlaylistIterator</code> |
| <b>Iterable (Aggregate)</b> | Abstract + Concrete aggregates | <code>Iterable</code> , <code>LinkedList</code> ,<br><code>BinaryTree</code> , <code>Playlist</code>                            |

### 3.1 Iterator interface

java

```
public interface Iterator<T> {
    boolean hasNext();
    T next();
    // Optional (homework):
    // boolean hasPrevious();
    // T previous();
}
```

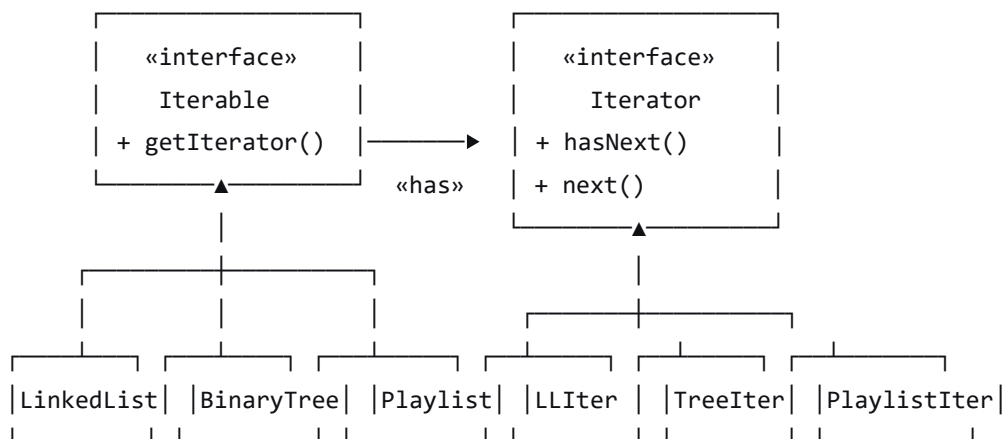
### 3.2 Iterable interface

java

```
public interface Iterable<T> {
    Iterator<T> getIterator();
}
```

### 3.3 UML relationships

text



Each concrete `Iterable` **has-a** concrete `Iterator` (or many), and each concrete `Iterator` **has-a** reference back to its aggregate (to access the underlying data).

## 4. Concrete Data Structures

## 4.1 Song (model)

java

```
public class Song {
    private final String title;
    private final String artist;

    public Song(String title, String artist) {
        this.title = title;
        this.artist = artist;
    }

    public String getTitle() { return title; }
    public String getArtist() { return artist; }

    @Override
    public String toString() {
        return title + " by " + artist;
    }
}
```

## 4.2 LinkedList

java

```
public class LinkedList implements Iterable<Integer> {
    int data;
    LinkedList next;

    public LinkedList(int data) {
        this.data = data;
        this.next = null;
    }

    @Override
    public Iterator<Integer> getIterator() {
        return new LinkedListIterator(this);    // pass 'this' as head
    }
}
```

## 4.3 BinaryTree

java

```

public class BinaryTree implements Iterable<Integer> {
    int data;
    BinaryTree left, right;

    public BinaryTree(int data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }

    @Override
    public Iterator<Integer> getIterator() {
        return new BinaryTreeInorderIterator(this);
    }
}

```

## 4.4 Playlist

java

```

public class Playlist implements Iterable<Song> {
    private final List<Song> songs = new ArrayList<>();

    public void addSong(Song s) { songs.add(s); }
    public void removeSong(Song s) { songs.remove(s); }

    // This method no longer knows HOW the List is traversed!
    public Song getSngByName(String name) {
        Iterator<Song> it = getIterator();
        while (it.hasNext()) {
            Song s = it.next();
            if (s.getTitle().equals(name)) return s;
        }
        return null;
    }

    @Override
    public Iterator<Song> getIterator() {
        return new PlaylistIterator(songs);
    }
}

```

## 5. Concrete Iterators

## 5.1 LinkedListIterator

### Logic:

- Holds current reference (starts at head).
- hasNext() → current != null .
- next() → return current.data , then current = current.next .

### Java:

java

```
public class LinkedListIterator implements Iterator<Integer> {
    private LinkedList current;

    public LinkedListIterator(LinkedList head) {
        this.current = head;
    }

    @Override
    public boolean hasNext() {
        return current != null;
    }

    @Override
    public Integer next() {
        if (!hasNext()) throw new NoSuchElementException();
        int value = current.data;
        current = current.next;
        return value;
    }
}
```

### Python:

python

```
class LinkedListIterator:
    def __init__(self, head):
        self.current = head

    def has_next(self):
        return self.current is not None

    def next(self):
        if not self.has_next():
```

```

        raise StopIteration
    value = self.current.data
    self.current = self.current.next
    return value

```

## 5.2 BinaryTreeInorderIterator

Iterative in-order traversal using an explicit stack.

**Logic:**

- Push all left descendants of current onto a stack.
- hasNext() → !stack.isEmpty() .
- next() → pop top, record its value, push all left descendants of its right child.

**Java:**

java

```

public class BinaryTreeInorderIterator implements Iterator<Integer> {
    private final Deque<BinaryTree> stack = new ArrayDeque<>();

    public BinaryTreeInorderIterator(BinaryTree root) {
        pushLeft(root);
    }

    private void pushLeft(BinaryTree node) {
        while (node != null) {
            stack.push(node);
            node = node.left;
        }
    }

    @Override
    public boolean hasNext() {
        return !stack.isEmpty();
    }

    @Override
    public Integer next() {
        if (!hasNext()) throw new NoSuchElementException();
        BinaryTree node = stack.pop();
        pushLeft(node.right);
        return node.data;
    }
}

```

```
    }
}
```

## Python:

python

```
class BinaryTreeInorderIterator:
    def __init__(self, root):
        self.stack = []
        self._push_left(root)

    def _push_left(self, node):
        while node:
            self.stack.append(node)
            node = node.left

    def has_next(self):
        return bool(self.stack)

    def next(self):
        if not self.has_next():
            raise StopIteration
        node = self.stack.pop()
        self._push_left(node.right)
        return node.data
```

## 5.3 PlaylistIterator

### Logic:

- Holds a reference to the `List<Song>` and an index.
- `hasNext()` → `index < songs.size()` .
- `next()` → `return songs.get(index++)` .

### Java:

java

```
public class PlaylistIterator implements Iterator<Song> {
    private final List<Song> songs;
    private int index = 0;

    public PlaylistIterator(List<Song> songs) {
        this.songs = songs;
    }
}
```

```

    }

    @Override
    public boolean hasNext() {
        return index < songs.size();
    }

    @Override
    public Song next() {
        if (!hasNext()) throw new NoSuchElementException();
        return songs.get(index++);
    }
}

```

## Python:

python

```

class PlaylistIterator:
    def __init__(self, songs):
        self.songs = songs
        self.index = 0

    def has_next(self):
        return self.index < len(self.songs)

    def next(self):
        if not self.has_next():
            raise StopIteration
        song = self.songs[self.index]
        self.index += 1
        return song

```

## 6. Client Code

### 6.1 Java

java

```

public class Main {
    public static void main(String[] args) {
        // ---- LinkedList ----
        LinkedList head = new LinkedList(1);
        head.next = new LinkedList(2);
    }
}

```

```

head.next.next = new LinkedList(3);

Iterator<Integer> llIt = head.getIterator();
System.out.print("LinkedList: ");
while (llIt.hasNext()) System.out.print(llIt.next() + " ");
System.out.println();

// ---- BinaryTree (In-order) ----
BinaryTree root = new BinaryTree(2);
root.left = new BinaryTree(1);
root.right = new BinaryTree(3);

Iterator<Integer> btIt = root.getIterator();
System.out.print("BinaryTree Inorder: ");
while (btIt.hasNext()) System.out.print(btIt.next() + " ");
System.out.println();

// ---- Playlist ----
Playlist playlist = new Playlist();
playlist.addSong(new Song("Admiring You", "Karan Aujla"));
playlist.addSong(new Song("Hustle", "Anuj Jain"));

Iterator<Song> plIt = playlist.getIterator();
System.out.println("Playlist:");
while (plIt.hasNext()) System.out.println(" " + plIt.next());
}
}

```

## Output:

text

```

LinkedList: 1 2 3
BinaryTree Inorder: 1 2 3
Playlist:
  Admiring You by Karan Aujla
  Hustle by Anuj Jain

```

## 6.2 Python

python

```

if __name__ == "__main__":
    # ---- LinkedList ----
    head = LinkedList(1)
    head.next = LinkedList(2)

```

```

head.next.next = LinkedList(3)

it = head.get_iterator()
print("LinkedList:", end=" ")
while it.has_next():
    print(it.next(), end=" ")
print()

# ---- BinaryTree Inorder ----
root = BinaryTree(2)
root.left = BinaryTree(1)
root.right = BinaryTree(3)

it = root.get_iterator()
print("BinaryTree Inorder:", end=" ")
while it.has_next():
    print(it.next(), end=" ")
print()

# ---- Playlist ----
playlist = Playlist()
playlist.add_song(Song("Admiring You", "Karan Aujla"))
playlist.add_song(Song("Hustle", "Anuj Jain"))

it = playlist.get_iterator()
print("Playlist:")
while it.has_next():
    print(" ", it.next())

```

## 7. Full Runnable Python Demo

python

```

from abc import ABC, abstractmethod
from typing import List

# ===== Iterator / Iterable =====
class Iterator(ABC):
    @abstractmethod
    def has_next(self) -> bool: ...
    @abstractmethod
    def next(self): ...

```

```

class Iterable(ABC):
    @abstractmethod
    def get_iterator(self) -> Iterator: ...

# ===== Models =====
class Song:
    def __init__(self, title: str, artist: str):
        self.title = title
        self.artist = artist

    def __str__(self):
        return f"{self.title} by {self.artist}"

# ===== LinkedList =====
class LinkedList(Iterable):
    def __init__(self, data: int):
        self.data = data
        self.next: LinkedList | None = None

    def get_iterator(self) -> Iterator:
        return LinkedListIterator(self)

class LinkedListIterator(Iterator):
    def __init__(self, head: LinkedList):
        self.current = head

    def has_next(self) -> bool:
        return self.current is not None

    def next(self) -> int:
        if not self.has_next():
            raise StopIteration
        val = self.current.data
        self.current = self.current.next
        return val

# ===== BinaryTree =====
class BinaryTree(Iterable):
    def __init__(self, data: int):
        self.data = data
        self.left: BinaryTree | None = None
        self.right: BinaryTree | None = None

    def get_iterator(self) -> Iterator:

```

```
return BinaryTreeInorderIterator(self)
```

```
class BinaryTreeInorderIterator(Iterator):
```

```
    def __init__(self, root: BinaryTree):
```

```
        self.stack: List[BinaryTree] = []
```

```
        self._push_left(root)
```

```
    def _push_left(self, node):
```

```
        while node:
```

```
            self.stack.append(node)
```

```
            node = node.left
```

```
    def has_next(self) -> bool:
```

```
        return bool(self.stack)
```

```
    def next(self) -> int:
```

```
        if not self.has_next():
```

```
            raise StopIteration
```

```
        node = self.stack.pop()
```

```
        self._push_left(node.right)
```

```
        return node.data
```

```
# ===== Playlist =====
```

```
class Playlist(Iterable):
```

```
    def __init__(self):
```

```
        self.songs: List[Song] = []
```

```
    def add_song(self, song: Song):
```

```
        self.songs.append(song)
```

```
    def remove_song(self, song: Song):
```

```
        self.songs.remove(song)
```

```
    def get_song_by_name(self, name: str) -> Song | None:
```

```
        it = self.get_iterator()
```

```
        while it.has_next():
```

```
            s = it.next()
```

```
            if s.title == name:
```

```
                return s
```

```
        return None
```

```
    def get_iterator(self) -> Iterator:
```

```
        return PlaylistIterator(self.songs)
```

```
class PlaylistIterator(Iterator):
```

```

def __init__(self, songs: List[Song]):
    self.songs = songs
    self.index = 0

def has_next(self) -> bool:
    return self.index < len(self.songs)

def next(self) -> Song:
    if not self.has_next():
        raise StopIteration
    s = self.songs[self.index]
    self.index += 1
    return s

# ===== Client =====
if __name__ == "__main__":
    head = LinkedList(1); head.next = LinkedList(2); head.next.next = LinkedList(3)
    it = head.get_iterator()
    print("LinkedList:", end=" ")
    while it.has_next(): print(it.next(), end=" ")
    print()

    root = BinaryTree(2); root.left = BinaryTree(1); root.right = BinaryTree(3)
    it = root.get_iterator()
    print("BinaryTree Inorder:", end=" ")
    while it.has_next(): print(it.next(), end=" ")
    print()

    pl = Playlist()
    pl.add_song(Song("Admiring You", "Karan Aujla"))
    pl.add_song(Song("Hustle", "Anuj Jain"))
    it = pl.get_iterator()
    print("Playlist:")
    while it.has_next(): print(" ", it.next())

```

## 8. Standard Definition

**Iterator Pattern** provides a way to access the elements of an **aggregate object** sequentially **without exposing its underlying representation**.

Key words:

- **Aggregate object** — an object that stores a collection (LinkedList, Tree, Playlist, Vector).

- **Sequentially** — one element at a time (not all at once).
- **Without exposing** — the client doesn't know if it's a LinkedList, Tree, Vector, or Map.

## 9. Real-World Use Cases

| Where                 | How                                                                                         |
|-----------------------|---------------------------------------------------------------------------------------------|
| C++ STL               | <code>vector::begin()/end()</code> , <code>list::iterator</code>                            |
| Java Collections      | <code>Iterator&lt;T&gt;</code> , <code>ListIterator&lt;T&gt;</code> , <code>for-each</code> |
| Python                | <code>__iter__</code> , <code>__next__</code> , <code>generators</code>                     |
| Database cursors      | Row-by-row fetching                                                                         |
| File readers          | Line-by-line reading                                                                        |
| Tree/Graph traversals | Custom iterators (preorder, inorder, postorder)                                             |

## 10. Homework (from the lecture)

Extend the iterator to support **backward traversal**:

java

```
public interface Iterator<T> {  
    boolean hasNext();  
    T next();  
    boolean hasPrevious();  
    T previous();  
}
```

Implementation hints:

- **LinkedListIterator**: needs a **doubly-linked list** (or a stack of visited nodes). Easiest: use a doubly-linked list with `prev` pointer.
- **BinaryTreeInorderIterator**: maintain a second stack for reverse in-order, or a `List<Integer>` of visited values and an index pointer.
- **PlaylistIterator**: trivial — just decrement `index` .

Alternative design: expose `getSequentialIterator()` and `getReverseIterator()` on `Iterable` .

Doubly-linked list hint (Java):

```
java

class DNode {
    int data;
    DNode next, prev;
    DNode(int d) { data = d; }
}

class DoublyLinkedListIterator implements Iterator<Integer> {
    private DNode current;
    private DNode head, tail;
    // hasNext/next go forward; hasPrevious/previous go backward
}
```

11. Comparison of Two Hierarchies

| Aspect                         | Iterator Hierarchy                           | Iterable Hierarchy            |
|--------------------------------|----------------------------------------------|-------------------------------|
| Role                           | Knows <b>how</b> to traverse                 | Knows <b>what</b> it stores   |
| Methods                        | <code>hasNext()</code> , <code>next()</code> | <code>getIterator()</code>    |
| Changes when storage changes   | Yes                                          | No                            |
| Changes when traversal changes | No (usually)                                 | No                            |
| Number of concrete classes     | Multiple (LL, Tree, Playlist)                | Multiple (LL, Tree, Playlist) |

The **two hierarchies evolve independently** — this is the beauty of the pattern.

12. Key Takeaways

- 1. Iterator **separates traversal** from **storage** — honors **SRP**.

2. Client uses only `hasNext()` / `next()` — never touches underlying structure.
3. Change `Vector` → `LinkedList` → **only** the iterator changes; other methods stay intact.
4. Only two methods needed for forward iteration: `hasNext()` and `next()`.
5. For **Tree iterators**, you need a stack (or recursion in a co-routine style) — the lecture uses **iterative in-order with an explicit stack**.
6. For **LinkedList iterators**, a single `current` pointer suffices.
7. For **Playlist iterators**, just hold the list + index.
8. Standard definition: **"Access elements of an aggregate object sequentially without exposing its underlying representation."**
9. Used **everywhere** — C++ STL, Java Collections, Python iterators, DB cursors.
10. **Homework**: add `hasPrevious()` / `previous()` to enable reverse traversal.

## 13. Quick Cheatsheet

| Question                                           | Answer                                                                                                         |
|----------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Why iterators?                                     | Separate traversal logic from storage logic (SRP)                                                              |
| Two core methods                                   | <code>hasNext()</code> , <code>next()</code>                                                                   |
| Which pattern does <code>for x in list</code> use? | Iterator                                                                                                       |
| Abstract classes/interfaces                        | <code>Iterator</code> , <code>Iterable</code> (Aggregate)                                                      |
| Concrete iterators in the example                  | <code>LinkedListIterator</code> ,<br><code>BinaryTreeInorderIterator</code> ,<br><code>PlaylistIterator</code> |
| Storage change impact                              | Only the iterator changes                                                                                      |
| Tree traversal implemented                         | Iterative in-order using explicit stack                                                                        |
| What to add for reverse iteration?                 | <code>hasPrevious()</code> + <code>previous()</code>                                                           |
| Standard definition                                | Access aggregate sequentially without exposing representation                                                  |

Would you like me to also produce:

- A **Mermaid UML diagram** for this pattern?
- A **generic Java version** using `Iterable<T>` / `Iterator<T>` with the `for-each` loop?
- A **comparison of Iterator vs Composite vs Visitor patterns**?



हां जी तो करें स्टार्ट। वेलकम बै...  
TXT 149.74KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

## LLD: Flyweight Design Pattern — Complete Deep Notes

### 1. Introduction

The **Flyweight Pattern** is a **memory-optimization** design pattern. It's used when you need to create **millions of similar objects** but don't want to exhaust RAM.

**Core idea:** Instead of creating a new object every time, **reuse** one object for all objects that share the same **intrinsic** (shared, unchanging) state, while storing **extrinsic** (unique, varying) state externally.

**When to use:** Whenever RAM is an important resource, and you have many objects with mostly-shared properties.

This is asked in LLD interviews as: *"Design a system where millions of objects are created but memory must be saved."*

### 2. The Problem — Millions of Objects Consuming RAM

Imagine a **Space Shooter** game:

- A spaceship shoots asteroids.
- Asteroids continuously spawn on screen — easy level = few, hard level = millions.
- Each asteroid is an object with properties:

- **Length, Width, Weight, Color, Texture, Material**
- **Position X, Position Y, Velocity X, Velocity Y**

If we create a **new asteroid object** every time one appears:

- 10 lakh (1 million) asteroids = 10 lakh objects.
- Each asteroid  $\approx$  196 bytes.
- Total RAM  $\approx$  **187 MB** (just for 1 million).

For a real game with 10 million asteroids, this becomes **gigabytes** → crash.

## Example: Without Flyweight

java

```
class Asteroid {
    int length, width, weight;
    String color, texture, material;
    int posX, posY, velX, velY;

    public Asteroid(int length, int width, int weight, String color,
                   String texture, String material,
                   int posX, int posY, int velX, int velY) {
        // assign all
    }

    public void render() {
        System.out.println("Rendering " + color + " " + texture + " " +
                           material + " asteroid of " + length + "x" + width +
                           " at (" + posX + "," + posY + ")");
    }
}
```

Creating 10 lakh of these → RAM explodes.

## 3. The Solution — Intrinsic vs Extrinsic State

The Flyweight pattern says:

| Type      | Meaning                                | Example                                         | Shared? |
|-----------|----------------------------------------|-------------------------------------------------|---------|
| Intrinsic | Shared, unchanging across many objects | length, width, weight, color, texture, material | ✔ Yes   |
| Extrinsic | Unique per object, varies              | posX, posY, velX, velY                          | ✗ No    |

Idea:

1. Split the object into two parts.
2. Keep only the intrinsic state in a **Flyweight** object (shared).
3. Move extrinsic state into a **Context** class (unique).
4. Use a **Factory + HashMap (Pool)** to cache and reuse flyweights.

### Why intrinsic state can be shared

Look at the asteroid:

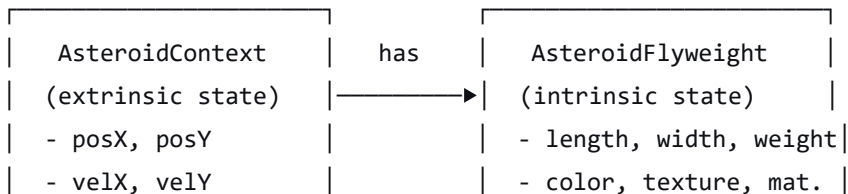
- Length can only be **10, 20, or 30**.
- Width can only be **10, 20, or 30**.
- Weight can only be **1, 2, or 3**.
- Color can only be **Red, Green, Blue**.
- Texture can only be **Soft, Hard, Semi-Hard**.
- Material can only be **Iron, Stone, Ice**.

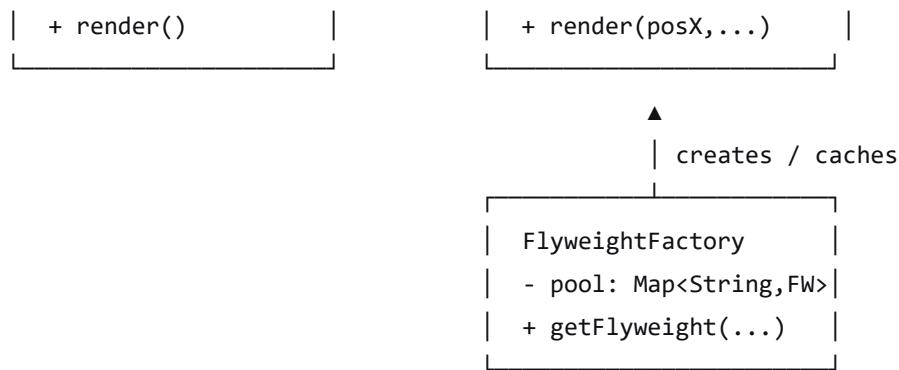
So there are only  $3 \times 3 \times 3 \times 3 \times 3 \times 3 = 729$  possible combinations. In practice (with limited configs), even fewer — like **3 unique types**.

**Conclusion:** Instead of 10 lakh objects, we only need **3 flyweight objects**.

### 4. UML Diagram

text





- **Context** → has-a **Flyweight** (holds extrinsic state + reference to intrinsic).
- **Factory** → caches flyweights in a `Map<key, Flyweight>`.

## 5. Key Principle: Immutability

Since a flyweight is **shared** across thousands of contexts, its state must be **immutable** (or at least never mutated after creation).

If you mutate a flyweight's `length`, all 3 lakh asteroids using that flyweight change at once. The client (game player) expects each asteroid to look independent.

So:

- Make fields `final` (Java) or use frozen dataclasses (Python).
- Provide only getters, no setters.

## 6. Java Code — Full Example

### 6.1 Flyweight (Intrinsic state)

java

```

public final class AsteroidFlyweight {
    private final int length;
    private final int width;
    private final int weight;
    private final String color;
    private final String texture;
    private final String material;
  }
  
```

```

public AsteroidFlyweight(int length, int width, int weight,
                        String color, String texture, String material) {
    this.length = length;
    this.width = width;
    this.weight = weight;
    this.color = color;
    this.texture = texture;
    this.material = material;
}

// Extrinsic state passed as parameters – this object never stores it
public void render(int posX, int posY, int velX, int velY) {
    System.out.println("Rendering " + color + " " + texture + " " +
        material + " asteroid of size " + length + "x" + width +
        " at (" + posX + "," + posY + ") velocity (" + velX + "," + velY + ")");
}

public int getMemoryUsage() {
    // 3 ints + 3 strings (approx)
    return Integer.BYTES * 3 + 32 * 3;
}
}

```

## 6.2 Context (Extrinsic state)

java

```

public class AsteroidContext {
    private final int posX, posY, velX, velY;
    private final AsteroidFlyweight flyweight; // reference, not copy

    public AsteroidContext(int posX, int posY, int velX, int velY,
                          AsteroidFlyweight flyweight) {
        this.posX = posX;
        this.posY = posY;
        this.velX = velX;
        this.velY = velY;
        this.flyweight = flyweight;
    }

    public void render() {
        flyweight.render(posX, posY, velX, velY);
    }

    public int getMemoryUsage() {
        return Integer.BYTES * 4 + 8; // 4 ints + reference (approx)
    }
}

```

```
    }
}
```

## 6.3 Flyweight Factory (with cache)

```
java
```

```
import java.util.*;

public class AsteroidFactory {
    private static final Map<String, AsteroidFlyweight> pool = new HashMap<>();

    public static AsteroidFlyweight getAsteroid(int length, int width, int weight,
                                                String color, String texture, String material) {
        String key = length + "_" + width + "_" + weight + "_" +
                    color + "_" + texture + "_" + material;

        if (!pool.containsKey(key)) {
            pool.put(key, new AsteroidFlyweight(length, width, weight, color, texture, material));
        }
        return pool.get(key);
    }

    public static int getFlyweightCount() { return pool.size(); }

    public static int getTotalFlyweightMemory() {
        int total = 0;
        for (AsteroidFlyweight fw : pool.values()) total += fw.getMemoryUsage();
        return total;
    }

    public static void clear() { pool.clear(); }
}
```

## 6.4 Space Game (Client)

```
java
```

```
import java.util.*;

public class SpaceGame {
    private final List<AsteroidContext> asteroids = new ArrayList<>();
}
```

```

private static final int[] SIZES      = {25, 35, 45};
private static final String[] COLORS  = {"RED", "BLUE", "GRAY"};
private static final String[] TEXTURES = {"ROCKY", "METALLIC", "ICY"};
private static final String[] MATERIALS = {"IRON", "STONE", "ICE"};

public void spawnAsteroids(int count) {
    for (int i = 0; i < count; i++) {
        int type = i % 3;

        AsteroidFlyweight fw = AsteroidFactory.getAsteroid(
            SIZES[type], SIZES[type], SIZES[type] * 10,
            COLORS[type], TEXTURES[type], MATERIALS[type]);

        int posX = 100 + i * 50;
        int posY = 200 + i * 30;
        int velX = 1;
        int velY = 2;

        asteroids.add(new AsteroidContext(posX, posY, velX, velY, fw));
    }
}

public void renderAll(int limit) {
    int n = Math.min(limit, asteroids.size());
    for (int i = 0; i < n; i++) asteroids.get(i).render();
}

public long getTotalMemory() {
    long contextMem = (long) asteroids.size() * new AsteroidContext(0,0,0,0,
        AsteroidFactory.getAsteroid(1,1,1,"x","x","x")).getMemoryUsage();
    long flyweightMem = AsteroidFactory.getTotalFlyweightMemory();
    return contextMem + flyweightMem;
}

public int getAsteroidCount() { return asteroids.size(); }
}

```

## 6.5 Main (Comparison)

java

```

public class Main {
    public static void main(String[] args) {
        int count = 1_000_000;

        // ---- WITHOUT Flyweight ----
        System.out.println("=== WITHOUT FLYWEIGHT ===");
    }
}

```

```

List<AsteroidWithoutFlyweight> list = new ArrayList<>();
long start = System.currentTimeMillis();
for (int i = 0; i < count; i++) {
    int t = i % 3;
    list.add(new AsteroidWithoutFlyweight(
        25 + t*10, 25 + t*10, (25 + t*10)*10,
        "C"+t, "T"+t, "M"+t, 100 + i*50, 200 + i*30, 1, 2));
}
long memBytes = (long) list.size() * 196;    // ~196 bytes/object
System.out.println("Objects: " + list.size());
System.out.println("Total memory: " + memBytes + " bytes = " +
    (memBytes / (1024*1024)) + " MB");
System.out.println("Time: " + (System.currentTimeMillis()-start) + " ms\n");

// ---- WITH Flyweight ----
System.out.println("=== WITH FLYWEIGHT ===");
SpaceGame game = new SpaceGame();
start = System.currentTimeMillis();
game.spawnAsteroids(count);
long withFwMem = game.getTotalMemory();
System.out.println("AsteroidContexts: " + game.getAsteroidCount());
System.out.println("Unique Flyweights: " + AsteroidFactory.getFlyweightCount());
System.out.println("Total memory: " + withFwMem + " bytes = " +
    (withFwMem / (1024*1024)) + " MB");
System.out.println("Time: " + (System.currentTimeMillis()-start) + " ms");
}
}

```

### Typical output:

text

=== WITHOUT FLYWEIGHT ===

Objects: 1000000

Total memory: 186 MB

=== WITH FLYWEIGHT ===

AsteroidContexts: 1000000

Unique Flyweights: 3

Total memory: 22 MB

**Savings ≈ 88% memory!**

## 7. Python Code — Full Example

python

```

from __future__ import annotations
from typing import Dict, List

# ----- Flyweight (intrinsic state, immutable) -----
class AsteroidFlyweight:
    def __init__(self, length, width, weight, color, texture, material):
        self.length = length
        self.width = width
        self.weight = weight
        self.color = color
        self.texture = texture
        self.material = material

    def render(self, pos_x, pos_y, vel_x, vel_y):
        print(f"Rendering {self.color} {self.texture} {self.material} "
              f"asteroid of size {self.length}x{self.width} at "
              f"({pos_x},{pos_y}) velocity ({vel_x},{vel_y})")

    def memory_usage(self) -> int:
        # 3 ints (28 bytes each in CPython) + 3 strings (approx 50 each)
        return 28 * 3 + 50 * 3

# ----- Context (extrinsic state) -----
class AsteroidContext:
    def __init__(self, pos_x, pos_y, vel_x, vel_y, flyweight: AsteroidFlyweight):
        self.pos_x = pos_x
        self.pos_y = pos_y
        self.vel_x = vel_x
        self.vel_y = vel_y
        self.flyweight = flyweight

    def render(self):
        self.flyweight.render(self.pos_x, self.pos_y, self.vel_x, self.vel_y)

    def memory_usage(self) -> int:
        # 4 ints + 1 reference (~28*4 + 8)
        return 28 * 4 + 8

# ----- Flyweight Factory (with pool) -----
class AsteroidFactory:
    _pool: Dict[str, AsteroidFlyweight] = {}

    @staticmethod

```

```

def get_asteroid(length, width, weight, color, texture, material) -> AsteroidFlyweight:
    key = f"{length}_{width}_{weight}_{color}_{texture}_{material}"
    if key not in AsteroidFactory._pool:
        AsteroidFactory._pool[key] = AsteroidFlyweight(
            length, width, weight, color, texture, material)
    return AsteroidFactory._pool[key]

@staticmethod
def flyweight_count() -> int:
    return len(AsteroidFactory._pool)

@staticmethod
def total_flyweight_memory() -> int:
    return sum(fw.memory_usage() for fw in AsteroidFactory._pool.values())

@staticmethod
def clear():
    AsteroidFactory._pool.clear()

# ----- Space Game (client) -----
class SpaceGame:
    SIZES      = [25, 35, 45]
    COLORS     = ["RED", "BLUE", "GRAY"]
    TEXTURES   = ["ROCKY", "METALLIC", "ICY"]
    MATERIALS  = ["IRON", "STONE", "ICE"]

    def __init__(self):
        self.asteroids: List[AsteroidContext] = []

    def spawn_asteroids(self, count: int):
        for i in range(count):
            t = i % 3
            fw = AsteroidFactory.get_asteroid(
                self.SIZES[t], self.SIZES[t], self.SIZES[t] * 10,
                self.COLORS[t], self.TEXTURES[t], self.MATERIALS[t])
            pos_x = 100 + i * 50
            pos_y = 200 + i * 30
            self.asteroids.append(AsteroidContext(pos_x, pos_y, 1, 2, fw))

    def render_all(self, limit: int):
        for c in self.asteroids[:limit]:
            c.render()

    def total_memory(self) -> int:
        ctx_mem = len(self.asteroids) * AsteroidContext(0, 0, 0, 0,
            AsteroidFactory.get_asteroid(1, 1, 1, "x", "x", "x")).memory_usage()

```

```
fw_mem = AsteroidFactory.total_flyweight_memory()
return ctx_mem + fw_mem

def asteroid_count(self) -> int:
    return len(self.asteroids)

# ----- Main (comparison) -----
if __name__ == "__main__":
    COUNT = 1_000_000

    print("=== WITH FLYWEIGHT ===")
    game = SpaceGame()
    game.spawn_asteroids(COUNT)
    game.render_all(5)
    mem = game.total_memory()
    print(f"AsteroidContexts: {game.asteroid_count()}")
    print(f"Unique Flyweights: {AsteroidFactory.flyweight_count()}")
    print(f"Total memory: {mem} bytes = {mem / (1024*1024):.2f} MB")
```

## 8. Memory Analysis

### Without Flyweight (10 lakh asteroids)

| Component                              | Size     | Count     | Total    |
|----------------------------------------|----------|-----------|----------|
| Int fields (length, width, weight)     | 4 B × 3  | 10,00,000 | 12 MB    |
| String refs (color, texture, material) | 4 B × 3  | 10,00,000 | 12 MB    |
| Extrinsic int fields                   | 4 B × 4  | 10,00,000 | 16 MB    |
| String data (~32 B each)               | 32 B × 3 | 10,00,000 | 96 MB    |
| Object overhead                        | ~50 B    | 10,00,000 | 50 MB    |
| Total                                  |          |           | ≈ 187 MB |

### With Flyweight

| Component             | Size   | Count     | Total   |
|-----------------------|--------|-----------|---------|
| Flyweights (3 unique) | ~196 B | 3         | ~600 B  |
| Contexts              | 24 B   | 10,00,000 | ~22 MB  |
| Total                 |        |           | ≈ 22 MB |

Savings ≈ 165 MB (88% reduction!)

## 9. Real-World Use Cases

| Domain                      | Flyweight                                                                    |
|-----------------------------|------------------------------------------------------------------------------|
| Games (GTA V, open-world)   | Trees, NPCs, cars, boats — millions of similar objects                       |
| Text editors                | Each character = intrinsic (glyph) + extrinsic (font, size, style, position) |
| Browsers                    | Repeated HTML elements / icons                                               |
| Java String pool            | Interned strings                                                             |
| Integer cache (-128 to 127) | Autoboxing reuses cached Integer objects                                     |
| Chess / Board games         | 64 squares, 32 pieces (only types shared)                                    |
| Forest simulation           | Billions of trees with shared sprites                                        |
| Databases                   | Shared metadata objects                                                      |

### Text Editor Example — Deep Dive

In a text editor like Word or Notepad:

- If user types "Hello World", that's 11 characters.
- If each char is a full object with font, size, style, position → **11 objects** (not many yet).
- But a book has ~5,00,000 characters → **5 lakh objects** per page-turn!
- Better:

- **Intrinsic:** the character glyph ('a', 'b', ...), font, size
- **Extrinsic:** position, color, style per instance
- There are only ~100 unique (char + font + size) combos.
- So we need only ~**100 flyweights** and millions of lightweight contexts.

## 10. Standard Definition

**Flyweight pattern** uses **sharing** to support a large number of **fine-grained objects** efficiently.

It separates an object into:

- **Intrinsic state** — shared, stored inside the flyweight.
- **Extrinsic state** — not shared, supplied by the client at call time.

## 11. Key Takeaways

1. **When to use:** Millions of similar objects consuming RAM.
2. **Two-part split:** Intrinsic (shared) + Extrinsic (unique).
3. **Flyweight** = intrinsic state, **immutable**, reusable.
4. **Context** = extrinsic state + reference to flyweight.
5. **Factory + HashMap pool** = reuse mechanism. Key = concatenation of all intrinsic fields.
6. **Memory saving is huge** — the lecture example went from 187 MB → 22 MB.
7. **Immutability is mandatory** — else all contexts sharing the flyweight get corrupted.
8. **Trade-off:** Small CPU overhead for map lookups; big memory savings.
9. **Three participants:** Flyweight, Context, FlyweightFactory.
10. **Java String pool and Integer cache are built-in Flyweight implementations.**

## 12. Quick Cheatsheet

| Question                 | Answer                                                       |
|--------------------------|--------------------------------------------------------------|
| Purpose                  | Reduce memory when many similar objects                      |
| Two states               | Intrinsic (shared), Extrinsic (unique)                       |
| Key participant          | Flyweight + Context + Factory                                |
| Cache data structure     | <code>HashMap&lt;Key, Flyweight&gt;</code>                   |
| Key formula              | Concatenation of intrinsic fields                            |
| Immutability required?   | Yes — shared state must not mutate                           |
| Real examples            | GTA V NPCs, text editors, Java String pool, Integer cache    |
| Memory saving in example | 187 MB → 22 MB (~88%)                                        |
| Standard definition      | Use sharing to support many fine-grained objects efficiently |
| Where used in real life  | Games, editors, browsers, DBs                                |

## 13. When NOT to Use Flyweight

- ✗ Few objects (< 1000) — overhead of pool & lookup outweighs savings.
- ✗ Objects don't share much state — no intrinsic state to share.
- ✗ Objects need to be mutated independently — breaks the sharing model.
- ✗ Hard to identify intrinsic vs extrinsic — wrong split causes bugs.

## 14. Interview Tip (from the lecture)

Whenever solving an LLD problem, ask:

*"Is RAM an important resource here? Do we have millions of similar objects? Can we identify intrinsic vs extrinsic state?"*

If yes → **propose Flyweight**. This impresses interviewers because it shows awareness of memory optimization, not just correctness.

Would you like me to also produce:

- A **Mermaid UML diagram** for the Flyweight pattern?
- A **comparison of Flyweight vs Prototype vs Singleton**?
- A **Runnable Java project structure** with separate files?



हां जी तो करें स्टार्ट। वेलकम बै...  
TXT 266.57KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also



## LLD: Splitwise Clone — Complete Deep Notes

### 1. Introduction

We're building a **Splitwise clone** — an **expense-sharing application** where friends in a group split bills, track balances, and settle up.

**Why it's a popular LLD interview question:**

- Multiple interacting entities (User, Group, Expense, Split)
- Need to handle **group membership lifecycle** (can't leave with pending dues)
- **Settlement tracking** between users
- **Multiple split strategies** (equal, exact, percentage)
- **Transaction simplification** via greedy algorithm — a classic DSA+LLD combo question
- Notifications on every expense/settlement

### 2. Functional Requirements

| # | Requirement                           |
|---|---------------------------------------|
| 1 | User can <b>join or leave a group</b> |

| # | Requirement                                                                               |
|---|-------------------------------------------------------------------------------------------|
| 2 | User can <b>add an expense</b> in a group                                                 |
| 3 | User can <b>settle an expense</b> in a group                                              |
| 4 | Adding an expense must support <b>several split strategies</b> — Equal, Percentage, Exact |
| 5 | User <b>cannot leave a group without settling all expenses</b>                            |
| 6 | User can also add <b>individual expenses</b> (outside groups — 1-on-1)                    |
| 7 | User gets <b>notified</b> when an expense is added or settled in a group                  |

**Non-Functional Requirements** (same as always):

- Scalable, loosely coupled, SOLID principles, modular.

### 3. Happy Flow

1. User opens the Splitwise app.
2. Sees groups G1, G2, G3 they belong to.
3. Opens a group → sees members U1, U2, U3, ...
4. **Adds an expense** (e.g., "U1 paid ₹800 for Lunch, split equally among U1, U2, U3, U4").
5. **Settles an expense** (e.g., "U2 paid U1 back ₹200").
6. All group members get a **notification**.
7. User can also do **individual 1-on-1 transactions** with another user.
8. Optionally, **simplify transactions** to minimize the number of transfers.

### 4. Design Patterns Used

| Pattern  | Where                          |
|----------|--------------------------------|
| Observer | Group notifies all its members |

| Pattern   | Where                                       |
|-----------|---------------------------------------------|
| Strategy  | Split strategies (Equal, Percentage, Exact) |
| Factory   | Create the right split strategy             |
| Facade    | Splitwise orchestrator class                |
| Singleton | Splitwise manager                           |

## 5. Core Classes — Detailed

### 5.1 User (Observer)

#### Fields:

- `userId` : String (auto-generated `user1` , `user2` , ...)
- `name` : String
- `email` : String
- `balances` : Map<String, Double> — key = other `userId`, value = amount
  - **Positive** → that user owes me
  - **Negative** → I owe that user

#### Methods:

- `update(message)` — from Observer interface; prints notification
- `updateBalance(otherUserId, amount)` — adds amount; if  $|\text{balance}| < 0.01$ , remove entry
- `getTotalOwed()` — sum of negative balances
- `getTotalOwing()` — sum of positive balances

### 5.2 Expense

#### Fields:

- `expenseId` : String (auto `expense1` , `expense2` , ...)
- `description` : String
- `amount` : double

- `paidByUserId : String`
- `splits : List<Split>`
- `groupId : String` (null / empty for individual expense)

## 5.3 Split

Instead of `Pair<String, Double>` (unclear semantics), define a class:

### Fields:

- `userId : String`
- `amount : double`

**Design Tip:** Never use `Pair<First, Second>` in public APIs. Wrap it in a named class to convey intent.

## 5.4 SplitStrategy (Interface)

java

```
List<Split> calculateSplit(double amount, List<String> userIds, List<Double> values);
```

### Concrete strategies:

#### EqualSplitStrategy

text

```
perUser = amount / userIds.size()
```

No values needed.

#### PercentageSplitStrategy

text

```
userAmount = amount * (percentage / 100)
```

Requires `values = percentages`.

#### ExactSplitStrategy (a.k.a. Unequal)

Uses `values` directly as exact amounts.

## 5.5 SplitFactory

java

```
static SplitStrategy getSplitStrategy(SplitType type) {
    switch (type) {
        case EQUAL:      return new EqualSplitStrategy();
        case PERCENTAGE: return new PercentageSplitStrategy();
        case EXACT:
        default:          return new ExactSplitStrategy();
    }
}
```

## 5.6 SplitType (Enum)

text

EQUAL, PERCENTAGE, EXACT

## 5.7 Group (Observable)

### Fields:

- groupId : String (auto group1 , group2 , ...)
- name : String
- users : List<User> (this is the **observer list**)
- expenses : Map<String, Expense> — history by expense ID
- balances : Map<String, Map<String, Double>> — **Map of Maps**
  - Outer key = userId (whose balance sheet)
  - Inner key = other userId
  - Value = amount (+/-)

**Why Map of Maps?** Each user has their own "mini balance sheet" of who they owe / are owed from.

### Methods:

- addUser(user) — add to list; also create an empty balance map entry for them
- removeUser(userId) — check canUserLeaveGroup() first; remove from users list + balances map

- `notifyMembers(message)` — iterate users; call `update()`
- `isMember(userId)`
- `addExpense(...)` — create `Expense`; update balances ; notify all
- `settlePayment(fromUserId, toUserId, amount)` — update balances ; notify all
- `updateGroupBalance(fromUserId, toUserId, amount)` — mutate the balance maps
- `canUserLeaveGroup(userId)` — return false if any pending balance exists
- `getUserGroupBalance(userId)` — return the inner map
- `showGroupBalances()` — pretty-print
- `simplifyGroupDebts()` — call `DebtSimplifier` ; replace balances
- `hasMember(userId)` — used by facade
- `getUserByUserId(userId)` — loop through users

## 5.8 DebtSimplifier (Greedy Algorithm)

### Field:

- `simplifyData(Map<String, Map<String, Double>> groupBalances)` → returns simplified map

### Algorithm (Greedy):

1. **Compute net balance per user** — for each (A,B) pair, if A→B is +X (B owes A), then A += X and B -= X. Only process each pair **once** (positive amounts only) to avoid double-counting.
2. Split into:
  - **Creditors** (net > 0, will receive)
  - **Debtors** (net < 0, will pay)
3. Sort both by absolute value descending (largest first).
4. Greedy matching:
  - Match largest debtor with largest creditor.
  - Settlement amount = `min(creditor.net, -debtor.net)` .
  - Apply and reduce both; move on.
5. Build the final simplified map.

### Example:

text

Before (net):

Aditya: +400

Manish: +200

Rohit: -200

Saurav: -400

Creditors (sorted): Aditya (400), Manish (200)

Debtors (sorted): Saurav (-400), Rohit (-200)

Greedy:

Saurav → Aditya:  $\min(400, 400) = 400 \rightarrow$  Aditya: 0, Saurav: 0

Rohit → Manish:  $\min(200, 200) = 200 \rightarrow$  Manish: 0, Rohit: 0

Result: 2 transactions instead of 4!

## 5.9 Splitwise (Facade / Orchestrator / Singleton)

**Fields:**

- users : Map<String, User>
- groups : Map<String, Group>
- expenses : Map<String, Expense>

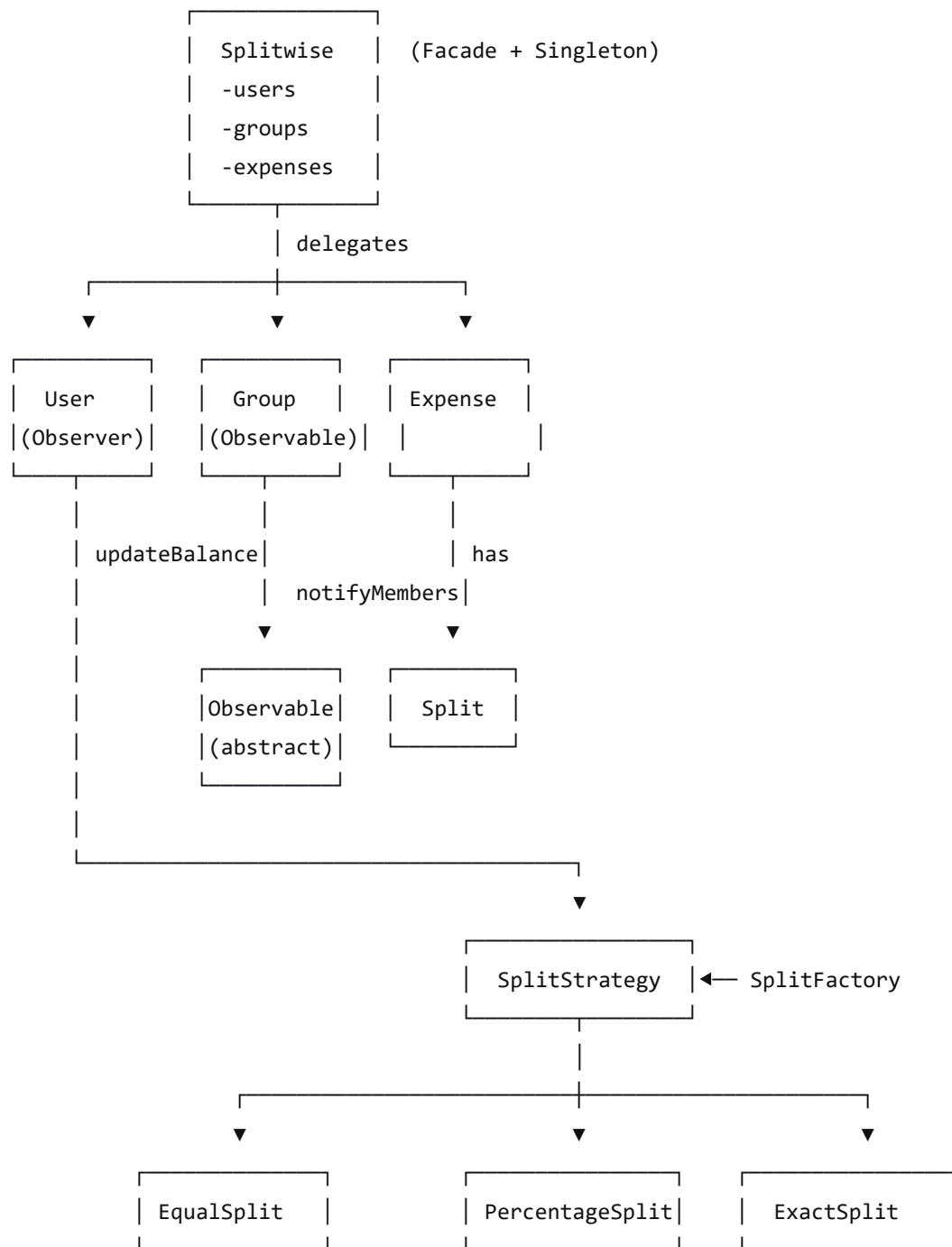
**Methods (all delegate):**

- createUser(name, email) , getUser(userId)
- createGroup(name) , getGroup(groupId)
- addUserToGroup(groupId, userId)
- removeUserFromGroup(groupId, userId) — returns success/fail
- addExpenseToGroup(groupId, desc, amount, paidBy, involvedUsers, splitType, values)
- settlePaymentInGroup(groupId, fromUserId, toUserId, amount)
- addIndividualExpense(desc, amount, paidBy, toUserId, splitType, values)
- settleIndividualPayment(fromUserId, toUserId, amount)
- showUserBalance(userId) / showGroupBalances(groupId)
- simplifyGroupDebts(groupId)

**Design note:** Facade breaks SRP — that's an **accepted trade-off** because its job is delegation.

## 6. UML Diagram (Text Form)

text



Group ◆— DebtSimplifier (composition – only Group uses it)

## 7. Java Code — Full Example

### 7.1 SplitType Enum

```
java
```

```
public enum SplitType { EQUAL, PERCENTAGE, EXACT }
```

## 7.2 Observer (abstract)

```
java
```

```
public interface Observer {  
    void update(String message);  
}
```

## 7.3 Split

```
java
```

```
public class Split {  
    private final String userId;  
    private final double amount;  
  
    public Split(String userId, double amount) {  
        this.userId = userId;  
        this.amount = amount;  
    }  
  
    public String getUserId() { return userId; }  
    public double getAmount() { return amount; }  
}
```

## 7.4 SplitStrategy + Implementations

```
java
```

```
import java.util.*;  
  
public interface SplitStrategy {  
    List<Split> calculateSplit(double amount, List<String> userIds, List<Double> values);  
}  
  
class EqualSplitStrategy implements SplitStrategy {  
    @Override  
    public List<Split> calculateSplit(double amount, List<String> userIds, List<Double> values) {  
        List<Split> splits = new ArrayList<>();  

```

```

        double perUser = amount / userIds.size();
        for (String userId : userIds) splits.add(new Split(userId, perUser));
        return splits;
    }
}

class PercentageSplitStrategy implements SplitStrategy {
    @Override
    public List<Split> calculateSplit(double amount, List<String> userIds, List<Double> values) {
        if (userIds.size() != values.size())
            throw new IllegalArgumentException("Size mismatch");
        List<Split> splits = new ArrayList<>();
        for (int i = 0; i < userIds.size(); i++) {
            double amt = amount * values.get(i) / 100.0;
            splits.add(new Split(userIds.get(i), amt));
        }
        return splits;
    }
}

class ExactSplitStrategy implements SplitStrategy {
    @Override
    public List<Split> calculateSplit(double amount, List<String> userIds, List<Double> values) {
        if (userIds.size() != values.size())
            throw new IllegalArgumentException("Size mismatch");
        List<Split> splits = new ArrayList<>();
        for (int i = 0; i < userIds.size(); i++) {
            splits.add(new Split(userIds.get(i), values.get(i)));
        }
        return splits;
    }
}

```

## 7.5 SplitFactory

java

```

public class SplitFactory {
    public static SplitStrategy getSplitStrategy(SplitType type) {
        switch (type) {
            case EQUAL:      return new EqualSplitStrategy();
            case PERCENTAGE: return new PercentageSplitStrategy();
            case EXACT:
            default:         return new ExactSplitStrategy();
        }
    }
}

```

```

    }
}

```

## 7.6 User (Observer implementation)

java

```

import java.util.*;

public class User implements Observer {
    private static int nextUserId = 0;
    private final String userId;
    private final String name;
    private final String email;
    private final Map<String, Double> balances = new HashMap<>();

    public User(String name, String email) {
        this.userId = "user" + (++nextUserId);
        this.name = name;
        this.email = email;
    }

    public String getUserId() { return userId; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public Map<String, Double> getBalances() { return balances; }

    @Override
    public void update(String message) {
        System.out.println("[Notification for " + name + "] " + message);
    }

    public void updateBalance(String otherUserId, double amount) {
        balances.merge(otherUserId, amount, Double::sum);
        if (Math.abs(balances.get(otherUserId)) < 0.01) balances.remove(otherUserId);
    }

    public double getTotalOwed() {
        double total = 0;
        for (double amt : balances.values()) if (amt < 0) total += amt;
        return total;
    }

    public double getTotalOwing() {
        double total = 0;
        for (double amt : balances.values()) if (amt > 0) total += amt;
        return total;
    }
}

```

```
}  
}
```

## 7.7 Expense

java

```
import java.util.*;  
  
public class Expense {  
    private static int nextId = 0;  
    private final String expenseId;  
    private final String description;  
    private final double amount;  
    private final String paidByUserId;  
    private final List<Split> splits;  
    private final String groupId; // null/empty for individual expense  
  
    public Expense(String description, double amount, String paidByUserId,  
                   List<Split> splits, String groupId) {  
        this.expenseId = "expense" + (++nextId);  
        this.description = description;  
        this.amount = amount;  
        this.paidByUserId = paidByUserId;  
        this.splits = splits;  
        this.groupId = groupId;  
    }  
  
    public String getExpenseId() { return expenseId; }  
    public String getDescription() { return description; }  
    public double getAmount() { return amount; }  
    public String getPaidByUserId() { return paidByUserId; }  
    public List<Split> getSplits() { return splits; }  
    public String getGroupId() { return groupId; }  
}
```

## 7.8 DebtSimplifier (Greedy)

java

```
import java.util.*;  
  
public class DebtSimplifier {  
  
    public static Map<String, Map<String, Double>> simplifyData(  

```

```

    Map<String, Map<String, Double>> groupBalances) {

    // Step 1: compute net balance for each user (process each pair once)
    Map<String, Double> net = new HashMap<>();
    for (var outer : groupBalances.entrySet()) {
        String creditor = outer.getKey();
        for (var inner : outer.getValue().entrySet()) {
            String debtor = inner.getKey();
            double amount = inner.getValue();
            if (amount > 0) { // process only positive → avoid double count
                net.merge(creditor, amount, Double::sum);
                net.merge(debtor, -amount, Double::sum);
            }
        }
    }

    // Step 2: split into creditors and debtors
    List<double[]> creditors = new ArrayList<>(); // [index, amount]
    List<double[]> debtors = new ArrayList<>();
    List<String> ids = new ArrayList<>(net.keySet());

    for (int i = 0; i < ids.size(); i++) {
        double v = net.get(ids.get(i));
        if (v > 0) creditors.add(new double[]{i, v});
        else if (v < 0) debtors.add(new double[]{i, -v});
    }

    // Step 3: sort descending
    creditors.sort((a, b) -> Double.compare(b[1], a[1]));
    debtors.sort((a, b) -> Double.compare(b[1], a[1]));

    // Step 4: greedy matching
    Map<String, Map<String, Double>> simplified = new HashMap<>();
    for (String id : ids) simplified.put(id, new HashMap<>());

    int i = 0, j = 0;
    while (i < debtors.size() && j < creditors.size()) {
        int dIdx = (int) debtors.get(i)[0];
        int cIdx = (int) creditors.get(j)[0];
        double dAmt = debtors.get(i)[1];
        double cAmt = creditors.get(j)[1];

        double settled = Math.min(dAmt, cAmt);
        String debtor = ids.get(dIdx);
        String creditor = ids.get(cIdx);

        // debtor owes creditor `settled`
        simplified.get(creditor).merge(debtor, settled, Double::sum);
    }
}

```

```

simplified.get(debtor).merge(creditor, -settled, Double::sum);

debtors.get(i)[1] -= settled;
creditors.get(j)[1] -= settled;

if (debtors.get(i)[1] < 0.01) i++;
if (creditors.get(j)[1] < 0.01) j++;
}

// Clean up zeros
simplified.values().forEach(m -> m.entrySet().removeIf(e -> Math.abs(e.getValue
()) < 0.01));
return simplified;
}
}

```

## 7.9 Group (Observable)

java

```

import java.util.*;

public class Group {
    private static int nextGroupId = 0;
    private final String groupId;
    private final String name;
    private final List<User> users = new ArrayList<>();
    private final Map<String, Expense> expenses = new HashMap<>();
    // userId → (otherUserId → amount)
    private final Map<String, Map<String, Double>> balances = new HashMap<>();

    public Group(String name) {
        this.groupId = "group" + (++nextGroupId);
        this.name = name;
    }

    public String getGroupId() { return groupId; }
    public String getName() { return name; }
    public List<User> getUsers() { return users; }
    public Map<String, Map<String, Double>> getBalances() { return balances; }

    // ----- Observers -----
    public void addUser(User user) {
        users.add(user);
        balances.putIfAbsent(user.getUserId(), new HashMap<>());
        System.out.println(user.getName() + " added to group " + name);
    }
}

```

```

public boolean removeUser(String userId) {
    if (!canUserLeaveGroup(userId)) {
        System.out.println("User is not allowed to leave the group without clearing e
xpenses.");
        return false;
    }
    users.removeIf(u -> u.getUserId().equals(userId));
    balances.remove(userId);
    for (Map<String, Double> inner : balances.values()) inner.remove(userId);
    System.out.println("User " + userId + " removed from group " + name);
    return true;
}

public boolean isMember(String userId) {
    return balances.containsKey(userId);
}

public boolean hasMember(String userId) { return isMember(userId); }

public User getUserByUserId(String userId) {
    for (User u : users) if (u.getUserId().equals(userId)) return u;
    return null;
}

public void notifyMembers(String message) {
    for (User u : users) u.update(message);
}

// ----- Balance management -----
public void updateGroupBalance(String fromUserId, String toUserId, double amount) {
    balances.get(fromUserId).merge(toUserId, amount, Double::sum);
    balances.get(toUserId).merge(fromUserId, -amount, Double::sum);

    // cleanup zeros
    if (Math.abs(balances.get(fromUserId).getOrDefault(toUserId, 0.0)) < 0.01)
        balances.get(fromUserId).remove(toUserId);
    if (Math.abs(balances.get(toUserId).getOrDefault(fromUserId, 0.0)) < 0.01)
        balances.get(toUserId).remove(fromUserId);
}

public boolean canUserLeaveGroup(String userId) {
    if (!isMember(userId)) return false;
    for (double amt : balances.get(userId).values()) {
        if (Math.abs(amt) > 0.01) return false;
    }
    return true;
}

```

```

public Map<String, Double> getUserGroupBalance(String userId) {
    return balances.getDefault(userId, new HashMap<>());
}

// ----- Add expense -----
public void addExpense(String description, double amount, String paidByUserId,
    List<String> involvedUserIds, SplitType splitType,
    List<Double> values) {

    if (!isMember(paidByUserId))
        throw new RuntimeException("Payer is not a member");
    for (String uid : involvedUserIds)
        if (!isMember(uid)) throw new RuntimeException("User " + uid + " not in group");

    SplitStrategy strategy = SplitFactory.getSplitStrategy(splitType);
    List<Split> splits = strategy.calculateSplit(amount, involvedUserIds, values);

    Expense expense = new Expense(description, amount, paidByUserId, splits, groupId);
    expenses.put(expense.getExpenseId(), expense);

    // update balances
    for (Split s : splits) {
        if (s.getUserId().equals(paidByUserId)) continue;
        updateGroupBalance(s.getUserId(), paidByUserId, s.getAmount());
    }

    notifyMembers("New expense added: " + description + " of ₹" + amount +
        " paid by " + getUserByUserId(paidByUserId).getName());
}

// ----- Settle payment -----
public void settlePayment(String fromUserId, String toUserId, double amount) {
    if (!isMember(fromUserId) || !isMember(toUserId))
        throw new RuntimeException("User not in group");

    updateGroupBalance(toUserId, fromUserId, amount); // reverses existing balance

    String fromName = getUserByUserId(fromUserId).getName();
    String toName = getUserByUserId(toUserId).getName();

    notifyMembers("Settlement: " + fromName + " paid ₹" + amount + " to " + toName);
    System.out.println(fromName + " settled ₹" + amount + " with " + toName);
}

// ----- Display -----

```

```

public void showGroupBalances() {
    System.out.println("----- Group: " + name + " -----");
    for (var outer : balances.entrySet()) {
        String uid = outer.getKey();
        User u = getUserById(uid);
        String uname = (u != null) ? u.getName() : uid;
        System.out.println("  " + uname + "'s balance sheet:");
        for (var inner : outer.getValue().entrySet()) {
            String otherUid = inner.getKey();
            User ou = getUserById(otherUid);
            String otherName = (ou != null) ? ou.getName() : otherUid;
            System.out.println("    " + otherName + ": ₹" + inner.getValue());
        }
    }
}

// ----- Simplify -----
public void simplifyGroupDebts() {
    Map<String, Map<String, Double>> simplified = DebtSimplifier.simplifyData(balances);
    balances.clear();
    balances.putAll(simplified);
    System.out.println("Group debts have been simplified.");
}
}

```

## 7.10 Splitwise (Facade + Singleton)

```

java

import java.util.*;

public class Splitwise {
    private static Splitwise instance;

    private final Map<String, User> users = new HashMap<>();
    private final Map<String, Group> groups = new HashMap<>();
    private final Map<String, Expense> expenses = new HashMap<>();

    private Splitwise() {}

    public static synchronized Splitwise getInstance() {
        if (instance == null) instance = new Splitwise();
        return instance;
    }

    public User createUser(String name, String email) {

```

```
User u = new User(name, email);
users.put(u.getId(), u);
return u;
}

public User getUser(String userId) { return users.get(userId); }

public Group createGroup(String name) {
    Group g = new Group(name);
    groups.put(g.getId(), g);
    return g;
}

public Group getGroup(String groupId) { return groups.get(groupId); }

public void addUserToGroup(String groupId, String userId) {
    Group g = groups.get(groupId);
    User u = users.get(userId);
    if (g != null && u != null) g.addUser(u);
}

public boolean removeUserFromGroup(String groupId, String userId) {
    Group g = groups.get(groupId);
    return g != null && g.removeUser(userId);
}

public void addExpenseToGroup(String groupId, String description, double amount,
                             String paidBy, List<String> involvedUsers,
                             SplitType splitType, List<Double> values) {
    Group g = groups.get(groupId);
    if (g == null) { System.out.println("Group not found"); return; }
    g.addExpense(description, amount, paidBy, involvedUsers, splitType, values);
}

public void settlePaymentInGroup(String groupId, String fromUserId,
                                 String toUserId, double amount) {
    Group g = groups.get(groupId);
    if (g != null) g.settlePayment(fromUserId, toUserId, amount);
}

public void addIndividualExpense(String description, double amount,
                                 String paidBy, String toUserId,
                                 SplitType splitType, List<Double> values) {
    SplitStrategy strategy = SplitFactory.getSplitStrategy(splitType);
    List<Split> splits = strategy.calculateSplit(amount, List.of(paidBy, toUserId), v
alues);

    Expense exp = new Expense(description, amount, paidBy, splits, null);
```

```

        expenses.put(exp.getExpenseId(), exp);

        User payer = users.get(paidBy);
        User other = users.get(toUserId);

        double half = amount / 2.0;
        payer.updateBalance(toUserId, +half);
        other.updateBalance(paidBy, -half);

        System.out.println(payer.getName() + " added individual expense with " + other.ge
tName());
    }

    public void settleIndividualPayment(String fromUserId, String toUserId, double amoun
t) {
        User from = users.get(fromUserId);
        User to = users.get(toUserId);
        from.updateBalance(toUserId, amount);
        to.updateBalance(fromUserId, -amount);
        System.out.println(from.getName() + " settled ₹" + amount + " with " + to.getName
());
    }

    public void showUserBalance(String userId) {
        User u = users.get(userId);
        if (u == null) return;
        System.out.println("----- Balance for " + u.getName() + " -----");
        u.getBalances().forEach((otherId, amt) -> {
            User other = users.get(otherId);
            String otherName = other != null ? other.getName() : otherId;
            System.out.println("    " + otherName + ": ₹" + amt);
        });
    }

    public void showGroupBalances(String groupId) {
        Group g = groups.get(groupId);
        if (g != null) g.showGroupBalances();
    }

    public void simplifyGroupDebts(String groupId) {
        Group g = groups.get(groupId);
        if (g != null) g.simplifyGroupDebts();
    }
}

```

## 7.11 Main (Demo)

java

```
public class Main {
    public static void main(String[] args) {
        Splitwise app = Splitwise.getInstance();

        User aditya = app.createUser("Aditya", "aditya@x.com");
        User rohit = app.createUser("Rohit", "rohit@x.com");
        User manish = app.createUser("Manish", "manish@x.com");
        User saurav = app.createUser("Saurav", "saurav@x.com");

        Group hostel = app.createGroup("Hostel Expenses");
        app.addUserToGroup(hostel.getId(), aditya.getId());
        app.addUserToGroup(hostel.getId(), rohit.getId());
        app.addUserToGroup(hostel.getId(), manish.getId());
        app.addUserToGroup(hostel.getId(), saurav.getId());

        List<String> allUsers = List.of(aditya.getId(), rohit.getId(),
                                      manish.getId(), saurav.getId());

        // Expense 1: Aditya paid ₹800 for Lunch – Equal split
        app.addExpenseToGroup(hostel.getId(), "Lunch", 800, aditya.getId(),
                              allUsers, SplitType.EQUAL, List.of());

        // Expense 2: Manish paid ₹700 for Dinner – Exact split
        List<String> dinnerUsers = List.of(aditya.getId(), manish.getId(), saurav.getId());
        List<Double> dinnerAmounts = List.of(200.0, 300.0, 200.0);
        app.addExpenseToGroup(hostel.getId(), "Dinner", 700, manish.getId(),
                              dinnerUsers, SplitType.EXACT, dinnerAmounts);

        System.out.println("\n--- Balances BEFORE simplification ---");
        app.showGroupBalances(hostel.getId());

        app.simplifyGroupDebts(hostel.getId());

        System.out.println("\n--- Balances AFTER simplification ---");
        app.showGroupBalances(hostel.getId());

        // Try removing Rohit – should fail (pending dues)
        System.out.println();
        app.removeUserFromGroup(hostel.getId(), rohit.getId());

        // Settle and retry
        app.settlePaymentInGroup(hostel.getId(), rohit.getId(), manish.getId(), 200);
        app.removeUserFromGroup(hostel.getId(), rohit.getId());
    }
}
```

```

// Individual expense: Rohit paid ₹40 coffee for Saurav
app.addIndividualExpense("Coffee", 40, rohit.getUserId(), saurav.getUserId(),
                        SplitType.EQUAL, List.of());
app.showUserBalance(rohit.getUserId());
}
}

```

## 8. Python Code — Full Example

python

```

from __future__ import annotations
from abc import ABC, abstractmethod
from enum import Enum
from typing import List, Dict

# ----- Enums -----
class SplitType(Enum):
    EQUAL = "EQUAL"
    PERCENTAGE = "PERCENTAGE"
    EXACT = "EXACT"

# ----- Observer -----
class Observer(ABC):
    @abstractmethod
    def update(self, message: str): ...

# ----- Split -----
class Split:
    def __init__(self, user_id: str, amount: float):
        self.user_id = user_id
        self.amount = amount

# ----- Split Strategies -----
class SplitStrategy(ABC):
    @abstractmethod
    def calculate_split(self, amount: float, user_ids: List[str],
                       values: List[float]) -> List[Split]: ...

class EqualSplitStrategy(SplitStrategy):

```

```

def calculate_split(self, amount, user_ids, values):
    per_user = amount / len(user_ids)
    return [Split(uid, per_user) for uid in user_ids]

class PercentageSplitStrategy(SplitStrategy):
    def calculate_split(self, amount, user_ids, values):
        if len(user_ids) != len(values):
            raise ValueError("Size mismatch")
        return [Split(user_ids[i], amount * values[i] / 100.0) for i in range(len(user_ids))]

class ExactSplitStrategy(SplitStrategy):
    def calculate_split(self, amount, user_ids, values):
        if len(user_ids) != len(values):
            raise ValueError("Size mismatch")
        return [Split(user_ids[i], values[i]) for i in range(len(user_ids))]

class SplitFactory:
    @staticmethod
    def get_split_strategy(t: SplitType) -> SplitStrategy:
        if t == SplitType.EQUAL: return EqualSplitStrategy()
        if t == SplitType.PERCENTAGE: return PercentageSplitStrategy()
        return ExactSplitStrategy()

# ----- User (Observer) -----
class User(Observer):
    _next_id = 0

    def __init__(self, name: str, email: str):
        User._next_id += 1
        self.user_id = f"user{User._next_id}"
        self.name = name
        self.email = email
        self.balances: Dict[str, float] = {}

    def update(self, message: str):
        print(f"[Notification for {self.name}] {message}")

    def update_balance(self, other_user_id: str, amount: float):
        self.balances[other_user_id] = self.balances.get(other_user_id, 0.0) + amount
        if abs(self.balances[other_user_id]) < 0.01:
            del self.balances[other_user_id]

    def get_total_owed(self) -> float:

```

```

        return sum(v for v in self.balances.values() if v < 0)

    def get_total_owing(self) -> float:
        return sum(v for v in self.balances.values() if v > 0)

# ----- Expense -----
class Expense:
    _next_id = 0

    def __init__(self, description, amount, paid_by, splits, group_id=None):
        Expense._next_id += 1
        self.expense_id = f"expense{Expense._next_id}"
        self.description = description
        self.amount = amount
        self.paid_by = paid_by
        self.splits = splits
        self.group_id = group_id

# ----- Debt Simplifier (Greedy) -----
class DebtSimplifier:
    @staticmethod
    def simplify_data(group_balances: Dict[str, Dict[str, float]]) -> Dict[str, Dict[str, float]]:
        # Step 1: net per user
        net: Dict[str, float] = {}
        for creditor, inner in group_balances.items():
            for debtor, amount in inner.items():
                if amount > 0: # process once
                    net[creditor] = net.get(creditor, 0.0) + amount
                    net[debtor] = net.get(debtor, 0.0) - amount

        # Step 2: split creditors / debtors
        creditors = [(uid, v) for uid, v in net.items() if v > 0]
        debtors = [(uid, -v) for uid, v in net.items() if v < 0]

        # Step 3: sort descending
        creditors.sort(key=lambda x: -x[1])
        debtors.sort(key=lambda x: -x[1])

        # Step 4: greedy matching
        simplified = {uid: {} for uid in net}
        i = j = 0
        while i < len(debtors) and j < len(creditors):
            d_id, d_amt = debtors[i]
            c_id, c_amt = creditors[j]
            settled = min(d_amt, c_amt)

```

```

simplified[c_id][d_id] = simplified[c_id].get(d_id, 0.0) + settled
simplified[d_id][c_id] = simplified[d_id].get(c_id, 0.0) - settled

debtors[i] = (d_id, d_amt - settled)
creditors[j] = (c_id, c_amt - settled)
if debtors[i][1] < 0.01: i += 1
if creditors[j][1] < 0.01: j += 1

# cleanup
for uid in list(simplified.keys()):
    simplified[uid] = {k: v for k, v in simplified[uid].items() if abs(v) > 0.01}
return simplified

# ----- Group (Observable) -----
class Group:
    _next_id = 0

    def __init__(self, name: str):
        Group._next_id += 1
        self.group_id = f"group{Group._next_id}"
        self.name = name
        self.users: List[User] = []
        self.expenses: Dict[str, Expense] = {}
        self.balances: Dict[str, Dict[str, float]] = {} # uid -> {other_uid: amt}

    # observers
    def add_user(self, user: User):
        self.users.append(user)
        self.balances.setdefault(user.user_id, {})
        print(f"{user.name} added to group {self.name}")

    def remove_user(self, user_id: str) -> bool:
        if not self.can_user_leave_group(user_id):
            print("User is not allowed to leave the group without clearing expenses.")
            return False
        self.users = [u for u in self.users if u.user_id != user_id]
        self.balances.pop(user_id, None)
        for inner in self.balances.values():
            inner.pop(user_id, None)
        return True

    def is_member(self, user_id: str) -> bool:
        return user_id in self.balances

    def get_user_by_id(self, user_id: str) -> User | None:
        return next((u for u in self.users if u.user_id == user_id), None)

```

```

def notify_members(self, message: str):
    for u in self.users:
        u.update(message)

# balance ops
def update_group_balance(self, from_id: str, to_id: str, amount: float):
    self.balances[from_id][to_id] = self.balances[from_id].get(to_id, 0.0) + amount
    self.balances[to_id][from_id] = self.balances[to_id].get(from_id, 0.0) - amount

    if abs(self.balances[from_id].get(to_id, 0)) < 0.01:
        self.balances[from_id].pop(to_id, None)
    if abs(self.balances[to_id].get(from_id, 0)) < 0.01:
        self.balances[to_id].pop(from_id, None)

def can_user_leave_group(self, user_id: str) -> bool:
    if user_id not in self.balances:
        return False
    for amt in self.balances[user_id].values():
        if abs(amt) > 0.01:
            return False
    return True

# expense
def add_expense(self, description, amount, paid_by, involved, split_type, values):
    if not self.is_member(paid_by):
        raise RuntimeError("Payer not in group")
    for uid in involved:
        if not self.is_member(uid):
            raise RuntimeError(f"User {uid} not in group")

    strategy = SplitFactory.get_split_strategy(split_type)
    splits = strategy.calculate_split(amount, involved, values)

    exp = Expense(description, amount, paid_by, splits, self.group_id)
    self.expenses[exp.expense_id] = exp

    for s in splits:
        if s.user_id == paid_by:
            continue
        self.update_group_balance(s.user_id, paid_by, s.amount)

    self.notify_members(
        f"New expense: {description} of ₹{amount} paid by "
        f"{self.get_user_by_id(paid_by).name}")

def settle_payment(self, from_id, to_id, amount):
    if not self.is_member(from_id) or not self.is_member(to_id):

```

```

        raise RuntimeError("User not in group")
    self.update_group_balance(to_id, from_id, amount)
    self.notify_members(
        f"Settlement: {self.get_user_by_id(from_id).name} paid "
        f"₹{amount} to {self.get_user_by_id(to_id).name}")

# display
def show_group_balances(self):
    print(f"----- Group: {self.name} -----")
    for uid, inner in self.balances.items():
        u = self.get_user_by_id(uid)
        uname = u.name if u else uid
        print(f" {uname}'s balance sheet:")
        for other_uid, amt in inner.items():
            ou = self.get_user_by_id(other_uid)
            oname = ou.name if ou else other_uid
            print(f"    {oname}: ₹{amt}")

# simplify
def simplify_group_debts(self):
    self.balances = DebtSimplifier.simplify_data(self.balances)
    print("Group debts have been simplified.")

# ----- Splitwise (Facade + Singleton) -----
class Splitwise:
    _instance = None

    def __init__(self):
        self.users: Dict[str, User] = {}
        self.groups: Dict[str, Group] = {}
        self.expenses: Dict[str, Expense] = {}

    @classmethod
    def get_instance(cls) -> "Splitwise":
        if cls._instance is None:
            cls._instance = Splitwise()
        return cls._instance

    def create_user(self, name: str, email: str) -> User:
        u = User(name, email)
        self.users[u.user_id] = u
        return u

    def get_user(self, user_id: str) -> User | None:
        return self.users.get(user_id)

    def create_group(self, name: str) -> Group:

```

```

    g = Group(name)
    self.groups[g.group_id] = g
    return g

def get_group(self, group_id: str) -> Group | None:
    return self.groups.get(group_id)

def add_user_to_group(self, group_id: str, user_id: str):
    g = self.groups.get(group_id); u = self.users.get(user_id)
    if g and u: g.add_user(u)

def remove_user_from_group(self, group_id: str, user_id: str) -> bool:
    g = self.groups.get(group_id)
    return g.remove_user(user_id) if g else False

def add_expense_to_group(self, group_id, description, amount, paid_by,
                        involved, split_type, values):
    g = self.groups.get(group_id)
    if not g: print("Group not found"); return
    g.add_expense(description, amount, paid_by, involved, split_type, values)

def settle_payment_in_group(self, group_id, from_id, to_id, amount):
    g = self.groups.get(group_id)
    if g: g.settle_payment(from_id, to_id, amount)

def add_individual_expense(self, description, amount, paid_by, to_user_id,
                        split_type, values):
    strategy = SplitFactory.get_split_strategy(split_type)
    splits = strategy.calculate_split(amount, [paid_by, to_user_id], values)

    exp = Expense(description, amount, paid_by, splits, None)
    self.expenses[exp.expense_id] = exp

    payer = self.users[paid_by]
    other = self.users[to_user_id]
    half = amount / 2.0
    payer.update_balance(to_user_id, +half)
    other.update_balance(paid_by, -half)
    print(f"{payer.name} added individual expense with {other.name}")

def settle_individual_payment(self, from_id, to_id, amount):
    self.users[from_id].update_balance(to_id, amount)
    self.users[to_id].update_balance(from_id, -amount)

def show_user_balance(self, user_id):
    u = self.users[user_id]
    print(f"----- Balance for {u.name} -----")
    for other_id, amt in u.balances.items():

```

```

        other = self.users.get(other_id)
        oname = other.name if other else other_id
        print(f" {oname}: ₹{amt}")

def show_group_balances(self, group_id):
    g = self.groups.get(group_id)
    if g: g.show_group_balances()

def simplify_group_debts(self, group_id):
    g = self.groups.get(group_id)
    if g: g.simplify_group_debts()

# ----- Demo -----
if __name__ == "__main__":
    app = Splitwise.get_instance()

    aditya = app.create_user("Aditya", "aditya@x.com")
    rohit = app.create_user("Rohit", "rohit@x.com")
    manish = app.create_user("Manish", "manish@x.com")
    saurav = app.create_user("Saurav", "saurav@x.com")

    hostel = app.create_group("Hostel Expenses")
    for u in (aditya, rohit, manish, saurav):
        app.add_user_to_group(hostel.group_id, u.user_id)

    all_users = [aditya.user_id, rohit.user_id, manish.user_id, saurav.user_id]

    # Expense 1: Aditya ₹800 Lunch – Equal
    app.add_expense_to_group(hostel.group_id, "Lunch", 800, aditya.user_id,
                             all_users, SplitType.EQUAL, [])

    # Expense 2: Manish ₹700 Dinner – Exact
    dinner_users = [aditya.user_id, manish.user_id, saurav.user_id]
    dinner_amounts = [200.0, 300.0, 200.0]
    app.add_expense_to_group(hostel.group_id, "Dinner", 700, manish.user_id,
                             dinner_users, SplitType.EXACT, dinner_amounts)

    print("\n--- Balances BEFORE simplification ---")
    app.show_group_balances(hostel.group_id)

    app.simplify_group_debts(hostel.group_id)

    print("\n--- Balances AFTER simplification ---")
    app.show_group_balances(hostel.group_id)

    print()
    app.remove_user_from_group(hostel.group_id, rohit.user_id) # fails

```

```
app.settle_payment_in_group(hostel.group_id, rohit.user_id, manish.user_id, 200)
app.remove_user_from_group(hostel.group_id, rohit.user_id) # succeeds
```

## 9. Transaction Simplification Algorithm — Deep Explanation

### Problem

In a group of 4 friends with 2 expenses:

- Aditya paid ₹800 (Lunch) — split equally among all 4.
- Manish paid ₹700 (Dinner) — split 200/300/200 among Aditya/Manish/Saurav.

This creates **4 individual obligations**. Can we do it in **fewer transactions**?

### Step-by-Step

#### Step 1: Build the balance matrix

|        | Aditya | Rohit | Manish | Saurav |
|--------|--------|-------|--------|--------|
| Aditya | —      | +200  | -100   | 0      |
| Rohit  | -200   | —     | 0      | 0      |
| Manish | +100   | 0     | —      | +200   |
| Saurav | 0      | 0     | -200   | —      |

#### Step 2: Compute net balances

- Aditya:  $+200 - 100 = +100$ ? No wait — from the diagram in the lecture, it becomes:
  - Aditya: +400
  - Manish: +200
  - Rohit: -200
  - Saurav: -400

#### Step 3: Split into creditors and debtors

| Creditors | Amount | Debtors | Amount |
|-----------|--------|---------|--------|
| Aditya    | +400   | Saurav  | -400   |
| Manish    | +200   | Rohit   | -200   |

#### Step 4: Greedy matching (largest first)

- Saurav (-400) meets Aditya (+400) → **Saurav pays Aditya ₹400**. Both settle.
- Rohit (-200) meets Manish (+200) → **Rohit pays Manish ₹200**. Both settle.

**Result: 2 transactions instead of 3 (or more).**

## Why Greedy Works

At each step:

- We match the **biggest debtor** with the **biggest creditor**.
- The smaller of the two amounts is settled.
- One of the parties (or both) is fully settled, so we advance.
- Number of iterations  $\leq$  (number of debtors + creditors).

**Complexity:**  $O(N \log N)$  for sorting +  $O(N)$  for matching.

## 10. Key Design Decisions

| Decision                                                                  | Why                                            |
|---------------------------------------------------------------------------|------------------------------------------------|
| Store balances as <b>Map of Maps</b> in Group                             | Each user has their own balance sheet          |
| Use <b>Split class</b> instead of <code>Pair&lt;String, Double&gt;</code> | Named fields prevent confusion                 |
| <b>Composition</b> for DebtSimplifier in Group                            | Only Group needs it; can't exist independently |
| <b>Static methods</b> for Factory & DebtSimplifier                        | No state to maintain                           |
| <b>Singleton</b> for Splitwise                                            | One app instance                               |

| Decision                          | Why                                      |
|-----------------------------------|------------------------------------------|
| <b>Observer</b> for notifications | Group is observable; Users are observers |
| <b>Facade</b> for Splitwise       | Delegates to User, Group, Expense        |

## 11. Real-World Considerations

- **Concurrency:** Lock balance updates per group (in real app).
- **Persistence:** Use DB (PostgreSQL) with tables `users` , `groups` , `group_members` , `expenses` , `splits` , `balances` .
- **Payment integration:** Real Splitwise uses UPI/Paytm integration — out of scope here.
- **Precision:** Always compare doubles with epsilon (0.01) — avoid `== 0` .
- **IDs:** In real app, use UUIDs or DB auto-increment. Here we use counter because no DB.

## 12. Key Takeaways

1. **Group = Observable** — notifies all members on expense/settlement.
2. **User = Observer** — receives notifications.
3. **Split Strategy** — three variants (Equal, Percentage, Exact) chosen via factory.
4. **Cannot leave group with pending balances** — enforced via `canUserLeaveGroup()` .
5. **Two balance systems:**
  - **Group-level:** `Map<User, Map<User, Amount>>`
  - **User-level (individual):** `Map<User, Amount>`
6. **Transaction simplification** uses greedy — sort creditors/debtors descending, match greedily.
7. **Splitwise facade** orchestrates everything — this is standard in LLD.
8. **Always avoid** `Pair<A,B>` **in public APIs** — use named classes ( `Split` ).

## 13. Quick Cheatsheet

| Concept                              | Implementation                                                    |
|--------------------------------------|-------------------------------------------------------------------|
| Notification                         | Observer pattern (Group → Users)                                  |
| Split strategies                     | Strategy + Factory                                                |
| Orchestrator                         | Splitwise (Facade + Singleton)                                    |
| Can't leave group                    | Check <code>canUserLeaveGroup()</code>                            |
| Balance storage                      | <code>Map&lt;String, Map&lt;String, Double&gt;&gt;</code>         |
| Individual expense                   | Direct balance update between two users                           |
| Simplify debts                       | Greedy: sort creditors/debtors, match largest                     |
| Why composition with DebtSimplifier? | Exists only for Group                                             |
| Auto IDs                             | Static counters ( <code>user1</code> , <code>group1</code> , ...) |
| Zeros cleanup                        | Remove entries with <code> amount  &lt; 0.01</code>               |

Would you like me to also produce:

- A **Mermaid class diagram** for the Splitwise clone?
- A **production-ready Spring Boot version** with REST endpoints?
- A **multi-file project structure** for the Java code?



हां जी तो करें स्टार्ट। सो वेलक...  
TXT 209.44KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

## LLD: State Design Pattern — Complete Deep Notes

### 1. Introduction

The **State Design Pattern** lets an object **alter its behavior when its internal state changes**. The object appears to change its class.

It's a **behavioral pattern** used when:

- An object can exist in a **limited number of states**
- A **limited number of operations** can be performed on it
- Each operation can cause the object to **transition to a new state**

The lecture uses the classic **Vending Machine** example — a standard interview favorite.

Other popular uses: **ATM machine**, **Document Editor**

(Draft/Published/Archived/Deleted), **Order tracking** (Placed/Shipped/Delivered), **Traffic light**.

## 2. Why State Pattern?

Without it, you end up writing **massive if-else chains** inside each method:

java

```
void insertCoin(Coin c) {  
    if (state == NO_COIN) {  
        // accept coin  
    } else if (state == HAS_COIN) {  
        // accept another coin  
    } else if (state == DISPENSE) {  
        // reject  
    } else if (state == SOLD_OUT) {  
        // reject  
    }  
}
```

Now multiply that by 5 operations × 4 states = **20 branches** in one class. Ugly, hard to maintain, violates Open/Closed.

With State Pattern:

- Each state becomes a **class**.
- Each class implements only the behavior valid for that state.
- Adding a new state = adding a new class, not modifying existing ones.

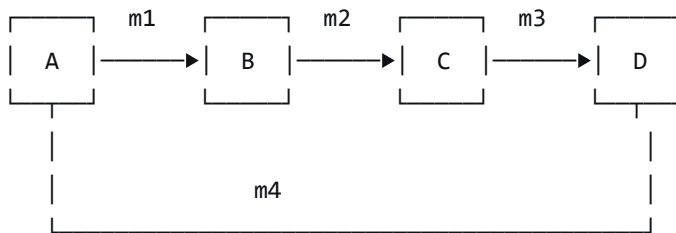
### 3. State Machine Diagram (SMD)

A **State Machine Diagram** (from Theory of Computation) shows:

- **States** (circles / boxes)
- **Transitions** (arrows) labeled with the **operation** that causes them

#### 3.1 Generic SMD

text



- States: A, B, C, D
- Operations: m1, m2, m3, m4
- Each operation on a state either **moves to a new state** or **stays in the same state**

#### 3.2 Vending Machine SMD

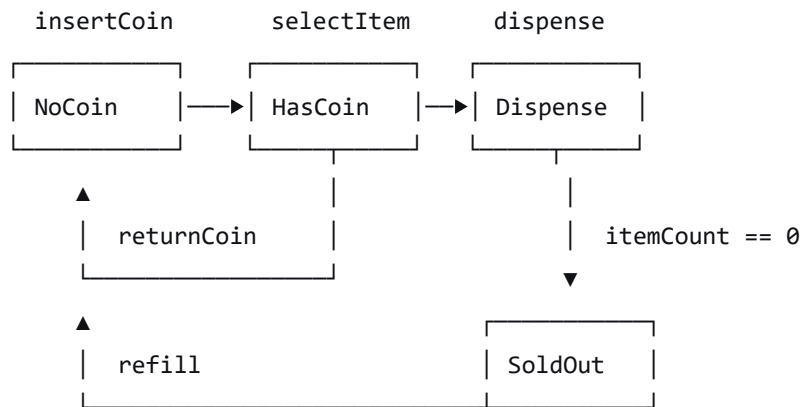
States:

- **NoCoinState** — idle, waiting for coin
- **HasCoinState** — has coin(s), can select item
- **DispenseState** — dispensing item
- **SoldOutState** — out of stock

Operations:

- `insertCoin` — insert money
- `selectItem` — choose item
- `dispense` — actually dispense
- `returnCoin` — user cancels, money returned
- `refill` — restock machine

text



### 3.3 State Transition Table

| State    | insertCoin | selectItem | dispense            | returnCoin | refill    |
|----------|------------|------------|---------------------|------------|-----------|
| NoCoin   | → HasCoin  | no change  | no change           | no change  | no change |
| HasCoin  | no change  | → Dispense | no change           | → NoCoin   | no change |
| Dispense | no change  | no change  | → NoCoin OR SoldOut | no change  | no change |
| SoldOut  | no change  | no change  | no change           | no change  | → NoCoin  |

**Key insight:** Each state has exactly **one or two operations** that change state; the rest stay in the same state (and typically throw an error).

## 4. Participants in the Pattern

| Role             | Responsibility                                                                                     |
|------------------|----------------------------------------------------------------------------------------------------|
| Context          | Holds a <code>currentState</code> reference. Delegates all operations to <code>currentState</code> |
| State (abstract) | Interface/abstract class declaring all operations                                                  |
| ConcreteState    | Implements operations; returns a <b>new state reference</b> to Context                             |

The Context is a **dumb object** — it doesn't know what state does; it just calls `state.method()` and updates its reference.

## 5. Java Implementation

### 5.1 Enums and Coin

java

```
public enum Coin {  
    ONE_RUPEE(1), FIVE_RUPEES(5), TEN_RUPEES(10), TWENTY_RUPEES(20);  
  
    private final int value;  
    Coin(int value) { this.value = value; }  
    public int getValue() { return value; }  
}
```

### 5.2 State (Abstract)

java

```
public interface VendingState {  
    VendingState insertCoin(VendingMachine machine, Coin coin);  
    VendingState selectItem(VendingMachine machine, String itemCode);  
    VendingState dispense(VendingMachine machine);  
    VendingState returnCoin(VendingMachine machine);  
    VendingState refill(VendingMachine machine, int quantity);  
    String getStateName();  
}
```

### 5.3 Concrete States

java

```
// ----- NoCoinState -----  
class NoCoinState implements VendingState {  
  
    @Override  
    public VendingState insertCoin(VendingMachine machine, Coin coin) {  
        machine.setInsertedCoins(machine.getInsertedCoins() + coin.getValue());  
        System.out.println("Coin inserted. Current balance: ₹" + machine.getInsertedCoins());  
    }  
}
```

```

        return machine.getHasCoinState();
    }

    @Override
    public VendingState selectItem(VendingMachine machine, String itemCode) {
        System.out.println("Please insert coin first.");
        return this;
    }

    @Override
    public VendingState dispense(VendingMachine machine) {
        System.out.println("Please insert coin and select item first.");
        return this;
    }

    @Override
    public VendingState returnCoin(VendingMachine machine) {
        System.out.println("No coin to return.");
        return this;
    }

    @Override
    public VendingState refill(VendingMachine machine, int quantity) {
        machine.incrementItemCount(quantity);
        System.out.println("Refilled " + quantity + " items.");
        return this;
    }

    @Override
    public String getStateName() { return "NO_COIN"; }
}

// ----- HasCoinState -----
class HasCoinState implements VendingState {

    @Override
    public VendingState insertCoin(VendingMachine machine, Coin coin) {
        machine.setInsertedCoins(machine.getInsertedCoins() + coin.getValue());
        System.out.println("Additional coin inserted. Balance: ₹" + machine.getInsertedCoins());
        return this;
    }

    @Override
    public VendingState selectItem(VendingMachine machine, String itemCode) {
        if (machine.getInsertedCoins() < machine.getItemPrice()) {
            int need = machine.getItemPrice() - machine.getInsertedCoins();
            System.out.println("Insufficient funds. Need ₹" + need + " more.");
        }
    }
}

```

```

        return this;
    }
    System.out.println("Item " + itemCode + " selected. Dispensing soon.");
    return machine.getDispenseState();
}

@Override
public VendingState dispense(VendingMachine machine) {
    System.out.println("Please select an item first.");
    return this;
}

@Override
public VendingState returnCoin(VendingMachine machine) {
    System.out.println("Returning ₹" + machine.getInsertedCoins());
    machine.setInsertedCoins(0);
    return machine.getNoCoinState();
}

@Override
public VendingState refill(VendingMachine machine, int quantity) {
    System.out.println("Cannot refill while a transaction is in progress.");
    return this;
}

@Override
public String getStateName() { return "HAS_COIN"; }
}

// ----- DispenseState -----
class DispenseState implements VendingState {

    @Override
    public VendingState insertCoin(VendingMachine machine, Coin coin) {
        System.out.println("Please wait, already dispensing an item.");
        return this;
    }

    @Override
    public VendingState selectItem(VendingMachine machine, String itemCode) {
        System.out.println("Already dispensing an item.");
        return this;
    }

    @Override
    public VendingState dispense(VendingMachine machine) {
        int change = machine.getInsertedCoins() - machine.getItemPrice();
        if (change > 0) System.out.println("Change returned: ₹" + change);
    }
}

```

```
        machine.setInsertedCoins(0);
        machine.decrementItemCount();

        System.out.println("Item dispensed.");
        if (machine.getItemCount() > 0) return machine.getNoCoinState();
        else return machine.getSoldOutState();
    }

    @Override
    public VendingState returnCoin(VendingMachine machine) {
        System.out.println("Cannot return coin while dispensing.");
        return this;
    }

    @Override
    public VendingState refill(VendingMachine machine, int quantity) {
        System.out.println("Cannot refill while dispensing.");
        return this;
    }

    @Override
    public String getStateName() { return "DISPENSE"; }
}

// ----- SoldOutState -----
class SoldOutState implements VendingState {

    @Override
    public VendingState insertCoin(VendingMachine machine, Coin coin) {
        System.out.println("Machine sold out. Cannot accept coin.");
        return this;
    }

    @Override
    public VendingState selectItem(VendingMachine machine, String itemCode) {
        System.out.println("Machine sold out.");
        return this;
    }

    @Override
    public VendingState dispense(VendingMachine machine) {
        System.out.println("Machine sold out.");
        return this;
    }

    @Override
    public VendingState returnCoin(VendingMachine machine) {
        System.out.println("No coin inserted.");
    }
}
```

```

        return this;
    }

    @Override
    public VendingState refill(VendingMachine machine, int quantity) {
        machine.incrementItemCount(quantity);
        System.out.println("Machine refilled with " + quantity + " items.");
        return machine.getNoCoinState();
    }

    @Override
    public String getStateName() { return "SOLD_OUT"; }
}

```

## 5.4 Context — VendingMachine

java

```

public class VendingMachine {
    private VendingState currentState;
    private final VendingState noCoinState = new NoCoinState();
    private final VendingState hasCoinState = new HasCoinState();
    private final VendingState dispenseState = new DispenseState();
    private final VendingState soldOutState = new SoldOutState();

    private int itemCount;
    private int itemPrice;
    private int insertedCoins;

    public VendingMachine(int itemCount, int itemPrice) {
        this.itemCount = itemCount;
        this.itemPrice = itemPrice;
        this.insertedCoins = 0;
        this.currentState = itemCount > 0 ? noCoinState : soldOutState;
    }

    // ---- Delegating operations ----
    public void insertCoin(Coin coin) {
        currentState = currentState.insertCoin(this, coin);
    }

    public void selectItem(String itemCode) {
        currentState = currentState.selectItem(this, itemCode);
    }

    public void dispense() {
        currentState = currentState.dispense(this);
    }

    public void returnCoin() {

```

```

        currentState = currentState.returnCoin(this);
    }
    public void refill(int quantity) {
        currentState = currentState.refill(this, quantity);
    }

    public void printStatus() {
        System.out.println("Items: " + itemCount +
            ", Balance: ₹" + insertedCoins +
            ", State: " + currentState.getStateName());
    }

    // ---- Getters/Setters used by States ----
    public VendingState getNoCoinState() { return noCoinState; }
    public VendingState getHasCoinState() { return hasCoinState; }
    public VendingState getDispenseState() { return dispenseState; }
    public VendingState getSoldOutState() { return soldOutState; }

    public int getItemCount() { return itemCount; }
    public void incrementItemCount(int n) { itemCount += n; }
    public void decrementItemCount() { if (itemCount > 0) itemCount--; }

    public int getItemPrice() { return itemPrice; }
    public int getInsertedCoins() { return insertedCoins; }
    public void setInsertedCoins(int v) { insertedCoins = v; }
}

```

## 5.5 Main (Demo)

java

```

public class Main {
    public static void main(String[] args) {
        VendingMachine machine = new VendingMachine(2, 20); // 2 items, ₹20 each
        machine.printStatus();

        System.out.println("\n--- Try selecting item without coin ---");
        machine.selectItem("WATER");
        machine.printStatus();

        System.out.println("\n--- Insert ₹10 ---");
        machine.insertCoin(Coin.TEN_RUPEES);
        machine.printStatus();

        System.out.println("\n--- Try selecting with insufficient funds ---");
        machine.selectItem("WATER");
        machine.printStatus();
    }
}

```

```

        System.out.println("\n--- Insert another ₹10 ---");
        machine.insertCoin(Coin.TEN_RUPEES);
        machine.printStatus();

        System.out.println("\n--- Select item ---");
        machine.selectItem("WATER");
        machine.printStatus();

        System.out.println("\n--- Dispense ---");
        machine.dispense();
        machine.printStatus();

        System.out.println("\n--- Buy remaining item ---");
        machine.insertCoin(Coin.TWENTY_RUPEES);
        machine.selectItem("WATER");
        machine.dispense();
        machine.printStatus();

        System.out.println("\n--- Try to buy when sold out ---");
        machine.insertCoin(Coin.TEN_RUPEES);
        machine.printStatus();

        System.out.println("\n--- Refill ---");
        machine.refill(5);
        machine.printStatus();
    }
}

```

## 6. Python Implementation

python

```

from abc import ABC, abstractmethod
from enum import Enum

# ----- Coin -----
class Coin(Enum):
    ONE = 1
    FIVE = 5
    TEN = 10
    TWENTY = 20

    def value(self):

```

```

        return self.value

# ----- State (ABC) -----
class VendingState(ABC):
    @abstractmethod
    def insert_coin(self, machine, coin): ...
    @abstractmethod
    def select_item(self, machine, item_code): ...
    @abstractmethod
    def dispense(self, machine): ...
    @abstractmethod
    def return_coin(self, machine): ...
    @abstractmethod
    def refill(self, machine, quantity): ...
    @abstractmethod
    def get_state_name(self): ...

# ----- NoCoinState -----
class NoCoinState(VendingState):
    def insert_coin(self, machine, coin):
        machine.inserted_coins += coin.value
        print(f"Coin inserted. Balance: ₹{machine.inserted_coins}")
        return machine.has_coin_state

    def select_item(self, machine, item_code):
        print("Please insert coin first.")
        return self

    def dispense(self, machine):
        print("Please insert coin and select item first.")
        return self

    def return_coin(self, machine):
        print("No coin to return.")
        return self

    def refill(self, machine, quantity):
        machine.item_count += quantity
        print(f"Refilled {quantity} items.")
        return self

    def get_state_name(self): return "NO_COIN"

# ----- HasCoinState -----
class HasCoinState(VendingState):

```

```

def insert_coin(self, machine, coin):
    machine.inserted_coins += coin.value
    print(f"Additional coin inserted. Balance: ₹{machine.inserted_coins}")
    return self

def select_item(self, machine, item_code):
    if machine.inserted_coins < machine.item_price:
        need = machine.item_price - machine.inserted_coins
        print(f"Insufficient funds. Need ₹{need} more.")
        return self
    print(f"Item {item_code} selected.")
    return machine.dispense_state

def dispense(self, machine):
    print("Please select an item first.")
    return self

def return_coin(self, machine):
    print(f"Returning ₹{machine.inserted_coins}")
    machine.inserted_coins = 0
    return machine.no_coin_state

def refill(self, machine, quantity):
    print("Cannot refill while transaction in progress.")
    return self

def get_state_name(self): return "HAS_COIN"

# ----- DispenseState -----
class DispenseState(VendingState):
    def insert_coin(self, machine, coin):
        print("Please wait, already dispensing.")
        return self

    def select_item(self, machine, item_code):
        print("Already dispensing.")
        return self

    def dispense(self, machine):
        change = machine.inserted_coins - machine.item_price
        if change > 0:
            print(f"Change returned: ₹{change}")
        machine.inserted_coins = 0
        machine.item_count -= 1
        print("Item dispensed.")
        if machine.item_count > 0:
            return machine.no_coin_state

```

```

        return machine.sold_out_state

    def return_coin(self, machine):
        print("Cannot return coin while dispensing.")
        return self

    def refill(self, machine, quantity):
        print("Cannot refill while dispensing.")
        return self

    def get_state_name(self): return "DISPENSE"

# ----- SoldOutState -----
class SoldOutState(VendingState):
    def insert_coin(self, machine, coin):
        print("Machine sold out.")
        return self

    def select_item(self, machine, item_code):
        print("Machine sold out.")
        return self

    def dispense(self, machine):
        print("Machine sold out.")
        return self

    def return_coin(self, machine):
        print("No coin inserted.")
        return self

    def refill(self, machine, quantity):
        machine.item_count += quantity
        print(f"Machine refilled with {quantity} items.")
        return machine.no_coin_state

    def get_state_name(self): return "SOLD_OUT"

# ----- Context - VendingMachine -----
class VendingMachine:
    def __init__(self, item_count: int, item_price: int):
        self.item_count = item_count
        self.item_price = item_price
        self.inserted_coins = 0

        self.no_coin_state = NoCoinState()
        self.has_coin_state = HasCoinState()

```

```

        self.dispense_state = DispenseState()
        self.sold_out_state = SoldOutState()

        self.current_state = self.no_coin_state if item_count > 0 else self.sold_out_state
    e

    # Delegating operations
    def insert_coin(self, coin): self.current_state = self.current_state.insert_coin(self, coin)
    def select_item(self, code): self.current_state = self.current_state.select_item(self, code)
    def dispense(self): self.current_state = self.current_state.dispense(self)
    def return_coin(self): self.current_state = self.current_state.return_coin(self)
    def refill(self, q): self.current_state = self.current_state.refill(self, q)

    def print_status(self):
        print(f"Items: {self.item_count}, Balance: ₹{self.inserted_coins}, "
              f"State: {self.current_state.get_state_name()}")

# ----- Demo -----
if __name__ == "__main__":
    m = VendingMachine(2, 20)
    m.print_status()

    print("\n--- Select without coin ---")
    m.select_item("WATER"); m.print_status()

    print("\n--- Insert ₹10 ---")
    m.insert_coin(Coin.TEN); m.print_status()

    print("\n--- Try select with insufficient funds ---")
    m.select_item("WATER"); m.print_status()

    print("\n--- Insert another ₹10 ---")
    m.insert_coin(Coin.TEN); m.print_status()

    print("\n--- Select item ---")
    m.select_item("WATER"); m.print_status()

    print("\n--- Dispense ---")
    m.dispense(); m.print_status()

    print("\n--- Buy remaining item ---")
    m.insert_coin(Coin.TWENTY)
    m.select_item("WATER")
    m.dispense(); m.print_status()

```

```

print("\n--- Try buy when sold out ---")
m.insert_coin(Coin.TEN); m.print_status()

print("\n--- Refill ---")
m.refill(5); m.print_status()

```

## Sample Output:

text

```

Items: 2, Balance: ₹0, State: NO_COIN

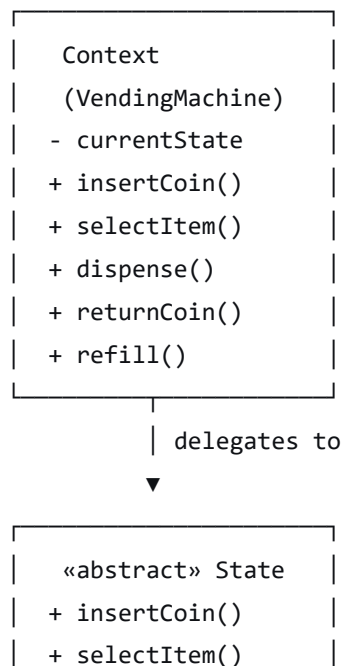
--- Select without coin ---
Please insert coin first.
Items: 2, Balance: ₹0, State: NO_COIN
...
--- Buy remaining item ---
Item dispensed.
Items: 0, Balance: ₹0, State: SOLD_OUT

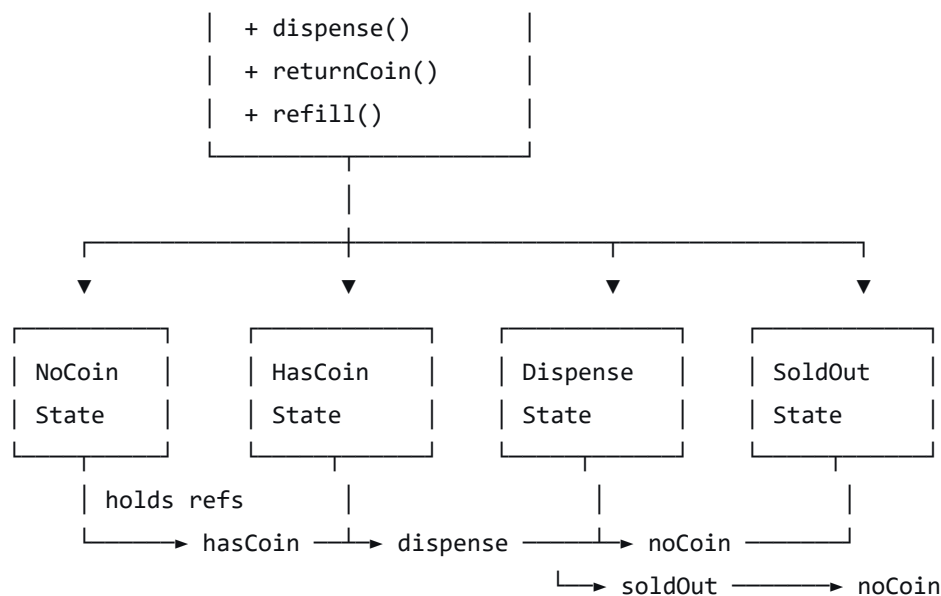
--- Refill ---
Machine refilled with 5 items.
Items: 5, Balance: ₹0, State: NO_COIN

```

## 7. Standard UML Diagram (Text Form)

text





**Key relationship:** Each ConcreteState holds **references to the other states** (or fetches them via Context) so it can return the next state.

## 8. Standard Definition

**State Pattern** allows an object to **alter its behavior when its internal state changes**. The object will appear to **change its class**.

**Key phrase:** "Appear to change its class" — because the Context's public methods stay the same, but the internal delegate changes → behavior changes.

## 9. Why State Pattern Matters

### Without State Pattern

Every Context method would have a giant if-else:

java

```

void insertCoin(Coin c) {
    if (currentState == NO_COIN) { ... }
    else if (currentState == HAS_COIN) { ... }
    else if (currentState == DISPENSE) { ... }
    else if (currentState == SOLD_OUT) { ... }
}
  
```

Multiply by 5 operations = **~20 branch combos**. Adding a new state = touching all methods. Violates Open/Closed.

## With State Pattern

- Each state is a **class** → SRP satisfied.
- New state = new class, no changes to existing.
- Context just delegates.
- State transitions are explicit in each state class.
- Extensible & testable.

## 10. Real-World Use Cases

| Domain          | States                                                |
|-----------------|-------------------------------------------------------|
| Vending machine | NoCoin, HasCoin, Dispense, SoldOut                    |
| ATM             | Idle, CardInserted, PinEntered, TransactionInProgress |
| Document editor | Draft, Published, Archived, Deleted                   |
| Order tracking  | Placed, Shipped, OutForDelivery, Delivered, Returned  |
| Traffic signal  | Red, Yellow, Green                                    |
| TCP connection  | Closed, Listen, SynSent, Established, ...             |
| Media player    | Stopped, Playing, Paused                              |
| Elevator        | Idle, MovingUp, MovingDown, DoorOpen                  |

## 11. Comparison with Related Patterns

| Aspect | State                                   | Strategy                    |
|--------|-----------------------------------------|-----------------------------|
| Intent | Change behavior based on internal state | Choose algorithm at runtime |

| Aspect                    | State                                    | Strategy                           |
|---------------------------|------------------------------------------|------------------------------------|
| Who drives change?        | States trigger their own transitions     | Client sets strategy               |
| State knows about others? | Yes — concrete states return next states | Strategies don't know each other   |
| Client awareness          | Client often unaware of state changes    | Client explicitly chooses strategy |
| UML                       | States reference each other              | Strategies are independent         |

| Aspect            | State                                    | Chain of Responsibility                 |
|-------------------|------------------------------------------|-----------------------------------------|
| Reference pattern | Concrete states hold refs to next states | Handlers hold refs to next handler      |
| Flow              | State transition per operation           | Request passes down chain until handled |
| Return value      | Next state                               | Handler result or None                  |

**Note from lecture:** The State pattern's structure resembles CoR — a concrete class holds a reference to another concrete class — but intent is **completely different**.

## 12. Key Takeaways

1. **When to use:** Object with **finite states**, **finite operations**, each op **transitions state**.
2. **SMD first:** Draw the state machine diagram before coding. It clarifies everything.
3. **Three building blocks:**
  - **Context** — holds state ref; delegates operations.
  - **State interface** — declares all operations.
  - **ConcreteStates** — implement ops; return next state.
4. **Context is dumb:** It just calls `state.op()` and updates its reference.
5. **State handles its own transition:** Each concrete state decides what the next state will be.

6. **Pre-instantiate states in Context constructor** so they're reused (avoids new object creation on every transition).
7. **Default state:** Based on initial conditions (e.g., if itemCount == 0 → SoldOut, else NoCoin).
8. **Eliminates if-else spaghetti** — each state class encapsulates its own rules.
9. **Standard definition:** *Allows an object to alter its behavior when its internal state changes; it appears to change its class.*
10. **Popular interview questions:** Vending Machine, ATM, Document Editor, Traffic Light, Elevator.

## 13. Quick Cheatsheet

| Question                        | Answer                                                                      |
|---------------------------------|-----------------------------------------------------------------------------|
| What problem does it solve?     | Object behavior varies by state, with many possible transitions             |
| Main participants               | Context, State, ConcreteState                                               |
| How does context know behavior? | It delegates to <code>currentState</code>                                   |
| Who triggers state change?      | Concrete states return the next state                                       |
| How to avoid if-else?           | Move logic into state classes                                               |
| Structure similar to?           | Chain of Responsibility (but intent differs)                                |
| Where is state stored?          | In a field of Context ( <code>currentState</code> )                         |
| How are transitions drawn?      | State machine diagram (SMD)                                                 |
| Default state?                  | Decided by initial conditions in Context constructor                        |
| Real-world examples             | Vending machine, ATM, Document editor, Traffic light                        |
| Standard definition             | Object alters behavior when internal state changes, appears to change class |

Would you like me to also produce:

- A **Mermaid class + state diagram** for the vending machine?
- A **multi-file project structure** in Java?
- An **ATM machine LLD** using the same State pattern?



हां जी तो करें स्टार्ट। वेलकम बै...  
TXT 144.22KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

## LLD: Tic-Tac-Toe Game — Complete Deep Notes

### 1. Introduction

Tic-Tac-Toe LLD is a **classic interview question** (often paired with Snake & Ladders). It's simple but reveals how well you handle:

- Object decomposition (Board, Cell, Symbol, Player, Rules)
- Extensibility ( $n \times n$  board, custom rules)
- Design patterns (Strategy, Factory, Observer, Facade)
- Turn management (Deque / round-robin)

The lecture follows a **Top-Down approach** — start with `Game` , then discover `Board` , `Symbol` , `Player` , `Rules` , etc.

### 2. Functional Requirements

| # | Requirement                                                                                               |
|---|-----------------------------------------------------------------------------------------------------------|
| 1 | <b>Board size should be scalable</b> — $3 \times 3$ , $4 \times 4$ , or $n \times n$                      |
| 2 | <b>Standard game rules + should be extensible</b> — e.g., diagonal wins disallowed, or minimum-4-in-a-row |

| # | Requirement                                                                                      |
|---|--------------------------------------------------------------------------------------------------|
| 3 | <b>On-app notifications</b> — when a player makes a move or wins, both players see it in the app |

**Non-Functional:** Same as always — loosely coupled, SOLID, extensible.

**On-app notifications** = notifications shown **inside the app** (like in [Chess.com](https://www.chess.com/), Ludo King) — different from email/SMS/OTP notifications which require a third-party service.

### 3. Why Top-Down Approach?

For games (Tic-Tac-Toe, Snake & Ladders):

- There are many **small objects** (Cell, Symbol, Player, Rule, etc.)
- Starting from `Game` (biggest class) helps identify what's needed
- We discovered `Board`, then `Grid`, then `Symbol`, then `Empty Symbol` — a natural discovery chain

## 4. Class-by-Class Design

### 4.1 Game (Orchestrator + Observable)

**Fields:**

- `board` : `Board` (Composition)
- `players` : `Deque<Player>` — round-robin turn order
- `rules` : `TicTacToeRules` (Composition)
- `observers` : `List<IObserver>` (Observer Pattern)
- `gameOver` : `boolean`

**Methods:**

- `addPlayer(Player)`
- `addObserver(IObserver) / notify(message)`

- `setRules(TicTacToeRules)`
- `play()` — main loop

**Why Deque?** For 2+ players, using a `List` with index tracking is messy. A `Deque` naturally does **round-robin**:

- Pop from front → current player plays
- Push to back → goes to end of queue
- Next front is the next player

**Why Composition with Rules?** Rules can't exist independently of a Game. If Game is destroyed, Rules is destroyed too.

## 4.2 Board (Dumb Object)

**Fields:**

- `grid : List<List<Symbol>>` — 2D grid
- `size : int`
- `emptySymbol : Symbol` (with mark `_`)

**Why emptySymbol instead of null?**

- `null` requires null checks everywhere → risk of NPE
- An `EmptySymbol` object marks empty cells cleanly

**Methods:**

- `isEmptyCell(row, col) → boolean`
- `getCell(row, col) → Symbol`
- `placeMark(row, col, symbol) → boolean`
- `display()`

**SRP:** Board only manages **cells/symbols** — knows nothing about win conditions.

## 4.3 Symbol

**Fields:**

- `mark : char` (X, O, `_`, etc.)

**Methods:** getMark()

**Extensible:** Could add displayStyle() later.

## 4.4 Player

**Fields:**

- playerId : int
- name : String
- symbol : Symbol
- score : int

**Methods:** getters/setters, incrementScore()

## 4.5 TicTacToeRules (Abstract + Strategy Pattern)

**Methods (abstract):**

- isValidMove(board, row, col) → boolean
- checkWinCondition(board, symbol) → boolean
- checkDrawCondition(board) → boolean

**StandardTicTacToeRules (Concrete):**

- Valid move → board.isCellEmpty(row, col)
- Win check → iterate rows, columns, main diagonal, anti-diagonal
- Draw check → all cells filled (no empty cell found)

**Why Strategy?** Different rules for different game modes:

- Standard (3-in-a-row any direction)
- No-diagonal-wins mode
- Min-4-in-a-row for bigger boards

## 4.6 IObserver (Observer Pattern)

java

```
public interface IObserver {  
    void update(String message);  
}
```

}

**ConsoleNotifier (Concrete Observer):**

- `update(message)` prints: `[NOTIFICATION] <message>`

**Key insight:** The observer is **NOT** the Player class. It's an **in-app notifier** because notifications appear inside the app UI, not on a player's phone/email. This is different from the general Observer pattern where the User is often the observer.

## 4.7 GameFactory

**Methods:**

- `createGame(GameType type, int boardSize) → Game`

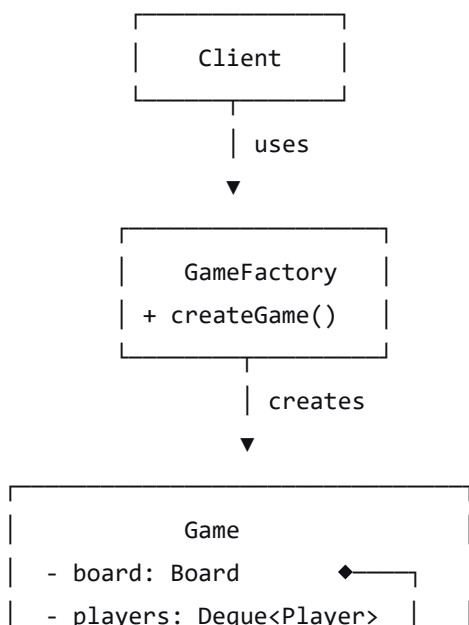
**GameType (Enum):** STANDARD

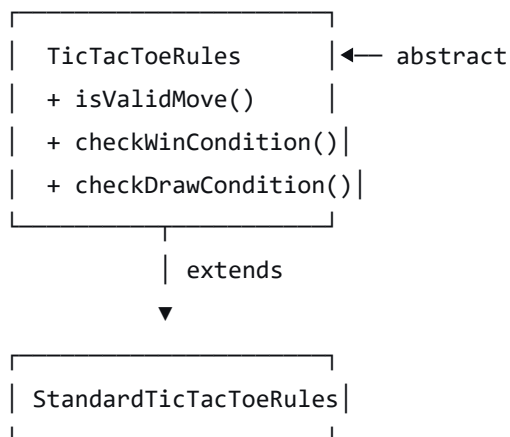
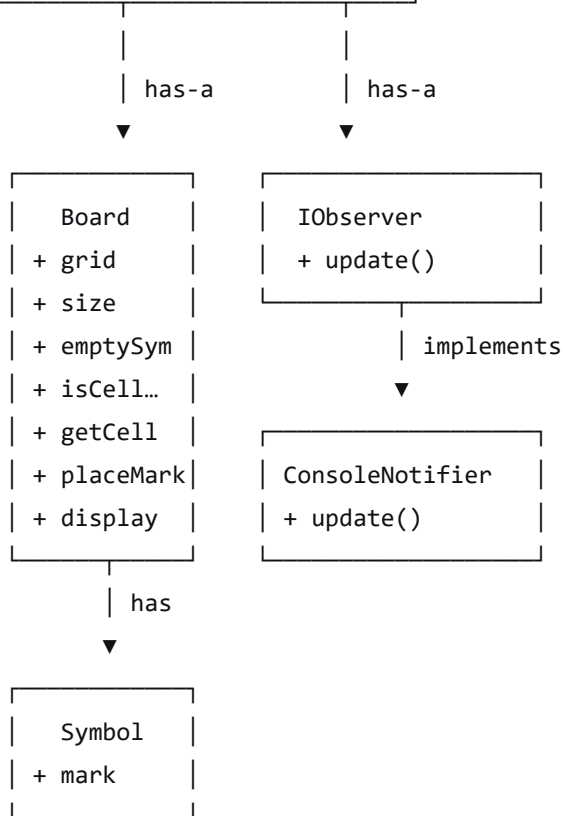
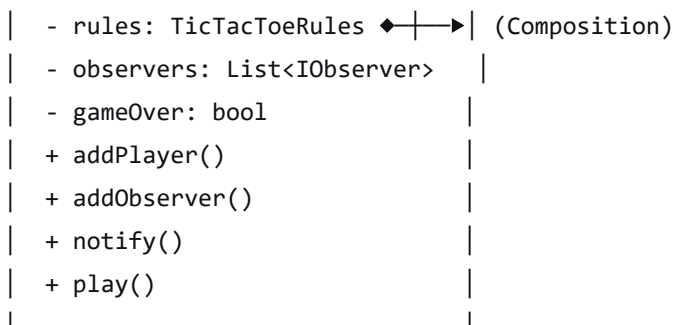
**Why Factory?**

- Encapsulates game creation
- Easy to add new game types
- Impresses interviewers with pattern knowledge

## 5. UML Relationships

text





## 6. Game Play Flow

1. **Client** calls `gameFactory.createGame(STANDARD, boardSize)` .
2. Factory instantiates `Game` with default rules.

3. Client adds two `Player`s and a `ConsoleNotifier` .

4. Client calls `game.play()` .

**Inside `play()` :**

text

```
while (!gameOver):
    currentPlayer = players.pop_front()
    read row, col from currentPlayer
    if (!rules.isValidMove(board, row, col)):
        print("Invalid move, try again")
        players.push_back(currentPlayer) // same player retries
        continue
    board.placeMark(row, col, currentPlayer.symbol)
    notify("Player X placed at (r,c)")
    if (rules.checkWinCondition(board, currentPlayer.symbol)):
        board.display()
        currentPlayer.incrementScore()
        notify(currentPlayer.name + " wins!")
        gameOver = true
        break
    if (rules.checkDrawCondition(board)):
        board.display()
        notify("Match drawn!")
        gameOver = true
        break
    // No win, no draw → rotate player
    players.push_back(currentPlayer)
```

**Note:** The player who just moved goes to the **back** of the queue so the next player gets the front.

## 7. Java Implementation

### 7.1 Symbol

java

```
public class Symbol {
    private final char mark;

    public Symbol(char mark) { this.mark = mark; }
```

```
    public char getMark() { return mark; }  
}
```

## 7.2 Board

java

```
import java.util.*;  
  
public class Board {  
    private final List<List<Symbol>> grid;  
    private final int size;  
    private final Symbol emptySymbol;  
  
    public Board(int size) {  
        this.size = size;  
        this.emptySymbol = new Symbol('_');  
        this.grid = new ArrayList<>();  
        for (int i = 0; i < size; i++) {  
            List<Symbol> row = new ArrayList<>();  
            for (int j = 0; j < size; j++) row.add(emptySymbol);  
            grid.add(row);  
        }  
    }  
  
    public int getSize() { return size; }  
    public Symbol getEmptySymbol() { return emptySymbol; }  
  
    public boolean isValid(int r, int c) {  
        return r >= 0 && r < size && c >= 0 && c < size;  
    }  
  
    public boolean isEmptyCell(int r, int c) {  
        if (!isValid(r, c)) return false;  
        return grid.get(r).get(c) == emptySymbol;  
    }  
  
    public boolean placeMark(int r, int c, Symbol s) {  
        if (!isValid(r, c)) return false;  
        if (!isEmptyCell(r, c)) return false;  
        grid.get(r).set(c, s);  
        return true;  
    }  
  
    public Symbol getCell(int r, int c) {  
        if (!isValid(r, c)) return null;  
        return grid.get(r).get(c);  
    }  
}
```

```

    }

    public void display() {
        System.out.println();
        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size; j++) {
                System.out.print(grid.get(i).get(j).getMark() + " ");
            }
            System.out.println();
        }
        System.out.println();
    }
}

```

## 7.3 Player

java

```

public class Player {
    private final int playerId;
    private final String name;
    private final Symbol symbol;
    private int score;

    public Player(int playerId, String name, Symbol symbol) {
        this.playerId = playerId;
        this.name = name;
        this.symbol = symbol;
        this.score = 0;
    }

    public int getPlayerId() { return playerId; }
    public String getName() { return name; }
    public Symbol getSymbol() { return symbol; }
    public int getScore() { return score; }
    public void incrementScore() { score++; }
}

```

## 7.4 Rules (Strategy Pattern)

java

```

public interface TicTacToeRules {
    boolean isValidMove(Board board, int row, int col);
    boolean checkWinCondition(Board board, Symbol symbol);
}

```

```
        boolean checkDrawCondition(Board board);
    }

    class StandardTicTacToeRules implements TicTacToeRules {

        @Override
        public boolean isValidMove(Board board, int row, int col) {
            return board.isCellEmpty(row, col);
        }

        @Override
        public boolean checkWinCondition(Board board, Symbol symbol) {
            int n = board.getSize();

            // Check rows
            for (int i = 0; i < n; i++) {
                boolean win = true;
                for (int j = 0; j < n; j++)
                    if (board.getCell(i, j) != symbol) { win = false; break; }
                if (win) return true;
            }

            // Check columns
            for (int j = 0; j < n; j++) {
                boolean win = true;
                for (int i = 0; i < n; i++)
                    if (board.getCell(i, j) != symbol) { win = false; break; }
                if (win) return true;
            }

            // Main diagonal
            boolean winMain = true;
            for (int i = 0; i < n; i++)
                if (board.getCell(i, i) != symbol) { winMain = false; break; }
            if (winMain) return true;

            // Anti-diagonal
            boolean winAnti = true;
            for (int i = 0; i < n; i++)
                if (board.getCell(i, n - 1 - i) != symbol) { winAnti = false; break; }
            return winAnti;
        }

        @Override
        public boolean checkDrawCondition(Board board) {
            int n = board.getSize();
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++)
```

```

        if (board.isEmpty(i, j)) return false;
        return true;
    }
}

```

## 7.5 Observer + ConsoleNotifier

java

```

public interface IObserver {
    void update(String message);
}

class ConsoleNotifier implements IObserver {
    @Override
    public void update(String message) {
        System.out.println("[NOTIFICATION] " + message);
    }
}

```

## 7.6 Game

java

```

import java.util.*;

public class Game {
    private final Board board;
    private final Deque<Player> players = new ArrayDeque<>();
    private TicTacToeRules rules;
    private final List<IObserver> observers = new ArrayList<>();
    private boolean gameOver;

    public Game(int boardSize) {
        this.board = new Board(boardSize);
        this.gameOver = false;
    }

    public void setRules(TicTacToeRules rules) { this.rules = rules; }

    public void addPlayer(Player p) { players.addLast(p); }
    public void addObserver(IObserver o) { observers.add(o); }

    public void notifyObservers(String msg) {
        for (IObserver o : observers) o.update(msg);
    }
}

```

```
}

public void play() {
    if (players.size() < 2) {
        System.out.println("At least 2 players required");
        return;
    }

    notifyObservers("Tic-Tac-Toe started!");
    Scanner sc = new Scanner(System.in);

    while (!gameOver) {
        Player current = players.pollFirst();
        board.display();
        System.out.print(current.getName() + "'s turn (symbol " +
            current.getSymbol().getMark() + "). Enter row and col: ");

        int r = sc.nextInt();
        int c = sc.nextInt();

        if (!rules.isValidMove(board, r, c)) {
            System.out.println("Invalid move, try again.");
            players.addFirst(current);    // put back at front
            continue;
        }

        board.placeMark(r, c, current.getSymbol());
        notifyObservers(current.getName() + " placed " +
            current.getSymbol().getMark() + " at (" + r + ", " + c + ")");

        if (rules.checkWinCondition(board, current.getSymbol())) {
            board.display();
            current.incrementScore();
            notifyObservers(current.getName() + " wins!");
            gameOver = true;
        } else if (rules.checkDrawCondition(board)) {
            board.display();
            notifyObservers("Match drawn!");
            gameOver = true;
        } else {
            players.addLast(current);    // rotate
        }
    }
    System.out.println("Game Over.");
}
```

## 7.7 GameType + Factory

```
java

public enum GameType { STANDARD }

public class GameFactory {
    public static Game createGame(GameType type, int boardSize) {
        Game game = new Game(boardSize);
        if (type == GameType.STANDARD) {
            game.setRules(new StandardTicTacToeRules());
            return game;
        }
        return null;
    }
}
```

## 7.8 Main (Demo)

```
java

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter board size: ");
        int boardSize = sc.nextInt();

        Game game = GameFactory.createGame(GameType.STANDARD, boardSize);

        Player p1 = new Player(1, "Aditya", new Symbol('X'));
        Player p2 = new Player(2, "Harshita", new Symbol('O'));

        game.addPlayer(p1);
        game.addPlayer(p2);
        game.addObserver(new ConsoleNotifier());

        game.play();
    }
}
```

## 8. Python Implementation

python

```
from abc import ABC, abstractmethod
from collections import deque
from enum import Enum
```

```
# ----- Symbol -----
```

```
class Symbol:
```

```
    def __init__(self, mark: str):
        self._mark = mark
    def get_mark(self):
        return self._mark
```

```
# ----- Board -----
```

```
class Board:
```

```
    def __init__(self, size: int):
        self.size = size
        self.empty_symbol = Symbol('_')
        self.grid = [[self.empty_symbol for _ in range(size)] for _ in range(size)]
```

```
    def is_valid(self, r, c):
        return 0 <= r < self.size and 0 <= c < self.size
```

```
    def is_cell_empty(self, r, c):
        return self.is_valid(r, c) and self.grid[r][c] is self.empty_symbol
```

```
    def place_mark(self, r, c, symbol):
        if not self.is_cell_empty(r, c):
            return False
        self.grid[r][c] = symbol
        return True
```

```
    def get_cell(self, r, c):
        if not self.is_valid(r, c):
            return None
        return self.grid[r][c]
```

```
    def display(self):
        print()
        for row in self.grid:
            print(" ".join(s.get_mark() for s in row))
        print()
```

```
# ----- Player -----
```

```
class Player:
```

```

def __init__(self, pid, name, symbol):
    self.player_id = pid
    self.name = name
    self.symbol = symbol
    self.score = 0

def increment_score(self):
    self.score += 1

# ----- Rules -----
class TicTacToeRules(ABC):
    @abstractmethod
    def is_valid_move(self, board, row, col): ...
    @abstractmethod
    def check_win_condition(self, board, symbol): ...
    @abstractmethod
    def check_draw_condition(self, board): ...

class StandardTicTacToeRules(TicTacToeRules):
    def is_valid_move(self, board, row, col):
        return board.is_cell_empty(row, col)

    def check_win_condition(self, board, symbol):
        n = board.size
        # rows
        for i in range(n):
            if all(board.get_cell(i, j) is symbol for j in range(n)):
                return True
        # cols
        for j in range(n):
            if all(board.get_cell(i, j) is symbol for i in range(n)):
                return True
        # main diag
        if all(board.get_cell(i, i) is symbol for i in range(n)):
            return True
        # anti diag
        if all(board.get_cell(i, n-1-i) is symbol for i in range(n)):
            return True
        return False

    def check_draw_condition(self, board):
        return all(not board.is_cell_empty(i, j)
                    for i in range(board.size)
                    for j in range(board.size))

```

```

# ----- Observer -----
class IObservable(ABC):
    @abstractmethod
    def update(self, message): ...

class ConsoleNotifier(IObservable):
    def update(self, message):
        print(f"[NOTIFICATION] {message}")

# ----- Game -----
class Game:
    def __init__(self, board_size):
        self.board = Board(board_size)
        self.players = deque()
        self.rules: TicTacToeRules | None = None
        self.observers = []
        self.game_over = False

    def set_rules(self, rules): self.rules = rules
    def add_player(self, p): self.players.append(p)
    def add_observer(self, o): self.observers.append(o)

    def notify(self, msg):
        for o in self.observers:
            o.update(msg)

    def play(self):
        if len(self.players) < 2:
            print("At least 2 players required")
            return

        self.notify("Tic-Tac-Toe started!")
        while not self.game_over:
            current = self.players.popleft()
            self.board.display()
            print(f"{current.name}'s turn ({current.symbol.get_mark()})")
            r = int(input("Enter row: "))
            c = int(input("Enter col: "))

            if not self.rules.is_valid_move(self.board, r, c):
                print("Invalid move, try again.")
                self.players.appendleft(current)
                continue

            self.board.place_mark(r, c, current.symbol)
            self.notify(f"{current.name} placed {current.symbol.get_mark()} at ({r},

```

```
{c})")
```

```

        if self.rules.check_win_condition(self.board, current.symbol):
            self.board.display()
            current.increment_score()
            self.notify(f"{current.name} wins!")
            self.game_over = True
        elif self.rules.check_draw_condition(self.board):
            self.board.display()
            self.notify("Match drawn!")
            self.game_over = True
        else:
            self.players.append(current)

    print("Game Over.")

```

```
# ----- GameType + Factory -----
```

```

class GameType(Enum):
    STANDARD = "STANDARD"

```

```

class GameFactory:
    @staticmethod
    def create_game(game_type: GameType, board_size: int) -> Game:
        game = Game(board_size)
        if game_type == GameType.STANDARD:
            game.set_rules(StandardTicTacToeRules())
            return game
        return None

```

```
# ----- Demo -----
```

```

if __name__ == "__main__":
    size = int(input("Enter board size: "))
    game = GameFactory.create_game(GameType.STANDARD, size)

    p1 = Player(1, "Aditya", Symbol('X'))
    p2 = Player(2, "Harshita", Symbol('O'))

    game.add_player(p1)
    game.add_player(p2)
    game.add_observer(ConsoleNotifier())

    game.play()

```

## 9. Design Patterns Used

| Pattern                    | Where                                                                                        | Why                                           |
|----------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------|
| <b>Strategy</b>            | TicTacToeRules hierarchy                                                                     | Swap rules without touching Game              |
| <b>Observer</b>            | IObserver + Game                                                                             | Notify both players of every move/win         |
| <b>Factory</b>             | GameFactory                                                                                  | Centralized game creation                     |
| <b>Composition</b>         | Game  Rules | Rules can't exist without Game                |
| <b>Facade/Orchestrator</b> | Game                                                                                         | Delegates to Board, Rules, Observers, Players |

## 10. Key Design Decisions

| Decision                               | Why                                                  |
|----------------------------------------|------------------------------------------------------|
| Deque for players                      | Natural round-robin without index tracking           |
| EmptySymbol instead of null            | Avoids NPE; cleaner cell checks                      |
| Symbol class (not char)                | Extensible; could add display style, color, etc.     |
| Board knows nothing about rules        | SRP — Board manages cells, Rules decide outcomes     |
| Observer = ConsoleNotifier, NOT Player | Notifications are <b>in-app</b> , not email/SMS      |
| Rules via Composition with Game        | Rules strictly bound to game lifecycle               |
| Factory returns Game                   | Interviewers love patterns; production code benefits |
| Board size user-controlled             | Scalability requirement                              |

## 11. Real-World Interview Insights

The same design philosophy applies to:

- **Snake & Ladders** — same Board/Symbol pattern, different Rules
- **Chess** — Board/Symbol pattern + more complex Rules
- **Ludo** — Deque for players, Board with cells
- **Connect Four** — Different rule for winning (min 4-in-a-row); rules extensible

Whenever you see a **game**, ask:

1. What's the Board? → Board class
2. What's the piece? → Symbol/Player
3. What are the rules? → Rule interface + concrete rule
4. Turn order? → Deque
5. What notifications? → Observer

## 12. Sample Run (Output)

text

Enter board size: 3

[NOTIFICATION] Tic-Tac-Toe started!

```
-- --
-- --
-- --
```

Aditya's turn (X)

Enter row: 0

Enter col: 0

[NOTIFICATION] Aditya placed X at (0,0)

```
X _ _
-- --
-- --
```

Harshita's turn (O)

Enter row: 0

Enter col: 2

[NOTIFICATION] Harshita placed O at (0,2)

...

[NOTIFICATION] Aditya wins!  
Game Over.

## 13. Key Takeaways

1. **Top-Down works well for games** — start with Game, discover smaller objects.
2. **Board = dumb object** — only manages cells. Knows nothing about rules.
3. **Rules = Strategy Pattern** — plug-and-play different rule sets.
4. **Player turn rotation = Deque** — natural round-robin (works for 2, 3, 4+ players).
5. **Notification = Observer Pattern** — but observer is a **console notifier**, not a Player.
6. **Use Empty Symbol instead of null** — avoid null checks / NPE.
7. **Board size is a constructor parameter** — makes the game  $n \times n$  scalable.
8. **Factory for creation** — impresses interviewers, eases future extensions.
9. **Game is a Facade/Orchestrator** — delegates to Board/Rules/Observers/Players.
10. **Composition between Game and Rules** — Rules cannot exist without Game.

## 14. Quick Cheatsheet

| Concept             | Implementation                                                                   |
|---------------------|----------------------------------------------------------------------------------|
| Board size          | Constructor parameter, $n \times n$                                              |
| Empty cell          | <code>EmptySymbol('_')</code> object, not null                                   |
| Turn management     | <code>Deque&lt;Player&gt;</code> — <code>pollFirst</code> , <code>addLast</code> |
| Win check           | Rows + Columns + 2 Diagonals                                                     |
| Draw check          | All cells filled                                                                 |
| Rules extensibility | <code>TicTacToeRules</code> interface + <code>Standard...</code> class           |
| Notifications       | <code>IObserver</code> + <code>ConsoleNotifier</code>                            |
| Game creation       | <code>GameFactory.createGame(type, size)</code>                                  |

| Concept                    | Implementation                                    |
|----------------------------|---------------------------------------------------|
| Rule ↔ Game relation       | Composition (Game ◆—— Rules)                      |
| Observer ↔ Game relation   | Aggregation (list of observers)                   |
| Popular interview variants | Tic-Tac-Toe, Snake & Ladders, Chess, Connect Four |

Would you like me to also produce:

- A **Mermaid class diagram** for Tic-Tac-Toe?
- A **Snake & Ladders LLD** using the same pattern?
- A **multi-file Java project structure** for the code?



हां जी तो करें स्टार्ट। वेलकम बै...  
TXT 182.12KB

make notes out of this transcript for me , explain everything deeply , if there is any code then explain code also give code in java and python also

## LLD: Snake & Ladders Game — Complete Deep Notes

### 1. Introduction

Snake & Ladders LLD is a **classic interview question** (fresh-level favorite, paired with Tic-Tac-Toe). It shares the same structure as the Tic-Tac-Toe LLD but with different rules and classes.

**Key learning from the lecture:** Board setup is not fixed — we need **Strategy Pattern** for multiple setup modes (Standard, Random with difficulty, Custom).

**Top-Down Approach** is used again — start with `Game` , discover `Board` , `Snake` , `Ladder` , `Dice` , `Player` , etc.

## 2. Functional Requirements

| # | Requirement                                                                         |
|---|-------------------------------------------------------------------------------------|
| 1 | <b>Board size should be scalable</b> — 100 cells, 1000 cells, etc.                  |
| 2 | <b>Standard game rules + should be extensible</b>                                   |
| 3 | <b>Multiple game setup strategies</b> — Standard, Random (Easy/Medium/Hard), Custom |
| 4 | <b>On-app notifications</b> (Console notifier, as before)                           |

## Random Difficulty Levels

| Level  | Snakes | Ladders |
|--------|--------|---------|
| Easy   | Fewer  | More    |
| Medium | Equal  | Equal   |
| Hard   | More   | Fewer   |

## Custom Setup Options

- Fixed number of snakes/ladders + random placement
- Fixed number of snakes/ladders + **exact positions** specified by user

## 3. Key Insight — Board Representation

The 10×10 visual grid is just a **presentation**. Internally:

- The board is a **1D sequence** 1, 2, 3, ..., n
- We **don't even need a 1D array** — we only need:
  - `size` (int) — max cell number
  - `entities` (List<BoardEntity>) — all snakes + ladders
  - `entitiesMap` (Map<Integer, BoardEntity>) — quick lookup by cell index

**Why Map?** O(1) lookup: "is there a snake/ladder at cell X?" instead of O(N) list scan.

**Player position** is stored inside the `Player` object itself.

## 4. Snake vs Ladder — Same Abstract Concept

|        | Start (Head) | End (Tail)             |
|--------|--------------|------------------------|
| Snake  | Higher cell  | Lower cell (goes down) |
| Ladder | Lower cell   | Higher cell (goes up)  |

Both are **cell movements** → abstract them as `BoardEntity` .

**Important rule:** Only the **Start** matters in the entities map. When a player lands on a snake's head or a ladder's foot, they teleport to End.

## 5. Class-by-Class Design

### 5.1 BoardEntity (Abstract)

Fields:

- `start : int`
- `end : int`

Methods:

- `getName()` → "Snake" or "Ladder"
- `display()` → abstract

### 5.2 Snake (extends BoardEntity)

Constructor Validation:

```
java
    if (end >= start) throw new RuntimeException("Invalid Snake");
```

Override:

- `display()` → prints "Snake: start → end"
- `getName()` → "Snake"

## 5.3 Ladder (extends BoardEntity)

### Constructor Validation:

java

```
if (end <= start) throw new RuntimeException("Invalid Ladder");
```

### Override:

- `display()` → prints "Ladder: start → end"
- `getName()` → "Ladder"

## 5.4 Board

### Fields:

- `size` : int
- `entities` : List<BoardEntity>
- `entitiesMap` : Map<Integer, BoardEntity>
- `setupStrategy` : BoardSetupStrategy

### Methods:

- `canAddEntity(position)` → boolean (checks if cell is free)
- `addEntity(entity)` → adds to list + map (keyed by `entity.start` )
- `getEntity(position)` → lookup
- `display()`
- `setupBoard(strategy)` — delegates to strategy

## 5.5 BoardSetupStrategy (Abstract — Strategy Pattern)

**Method:** `setupBoard(Board board)`

### Concrete implementations:

1. **StandardBoardSetupStrategy** — pre-defined fixed positions for 10×10

2. **RandomBoardSetupStrategy** — takes Difficulty in constructor; generates snakes/ladders randomly based on difficulty
3. **CustomCountBoardSetupStrategy** — user provides count + optional positions

## 5.6 Difficulty (Enum — nested in RandomBoardSetupStrategy)

java

```
enum Difficulty { EASY, MEDIUM, HARD }
```

## 5.7 Dice

**Fields:**

- faces : int

**Methods:**

- roll() → random number between 1 and faces

## 5.8 Player

**Fields:**

- id : int
- name : String
- position : int
- score : int

**Methods:** getters/setters, incrementScore()

## 5.9 SnakeAndLadderRules (Abstract — Strategy)

**Methods:**

- isValidMove(position, diceValue, boardSize) → boolean
- calculateNewPosition(position, diceValue, board) → int
- checkWinCondition(position, boardSize) → boolean

## 5.10 StandardSnakeAndLadderRules

**isValidMove:** position + diceValue <= boardSize

**calculateNewPosition:**

java

```
int next = position + diceValue;
BoardEntity entity = board.getEntity(next);
return (entity == null) ? next : entity.getEnd();
```

**checkWinCondition:** position == boardSize

## 5.11 Game (Orchestrator + Observable)

**Fields:**

- board : Board (Composition)
- dice : Dice
- players : Deque<Player>
- rules : SnakeAndLadderRules (Composition)
- observers : List<IObserver>
- gameOver : boolean

**Methods:**

- addPlayer() , addObserver() , notify(message)
- displayPlayerPosition()
- play()

## 5.12 IObserver / ConsoleNotifier

Same as Tic-Tac-Toe.

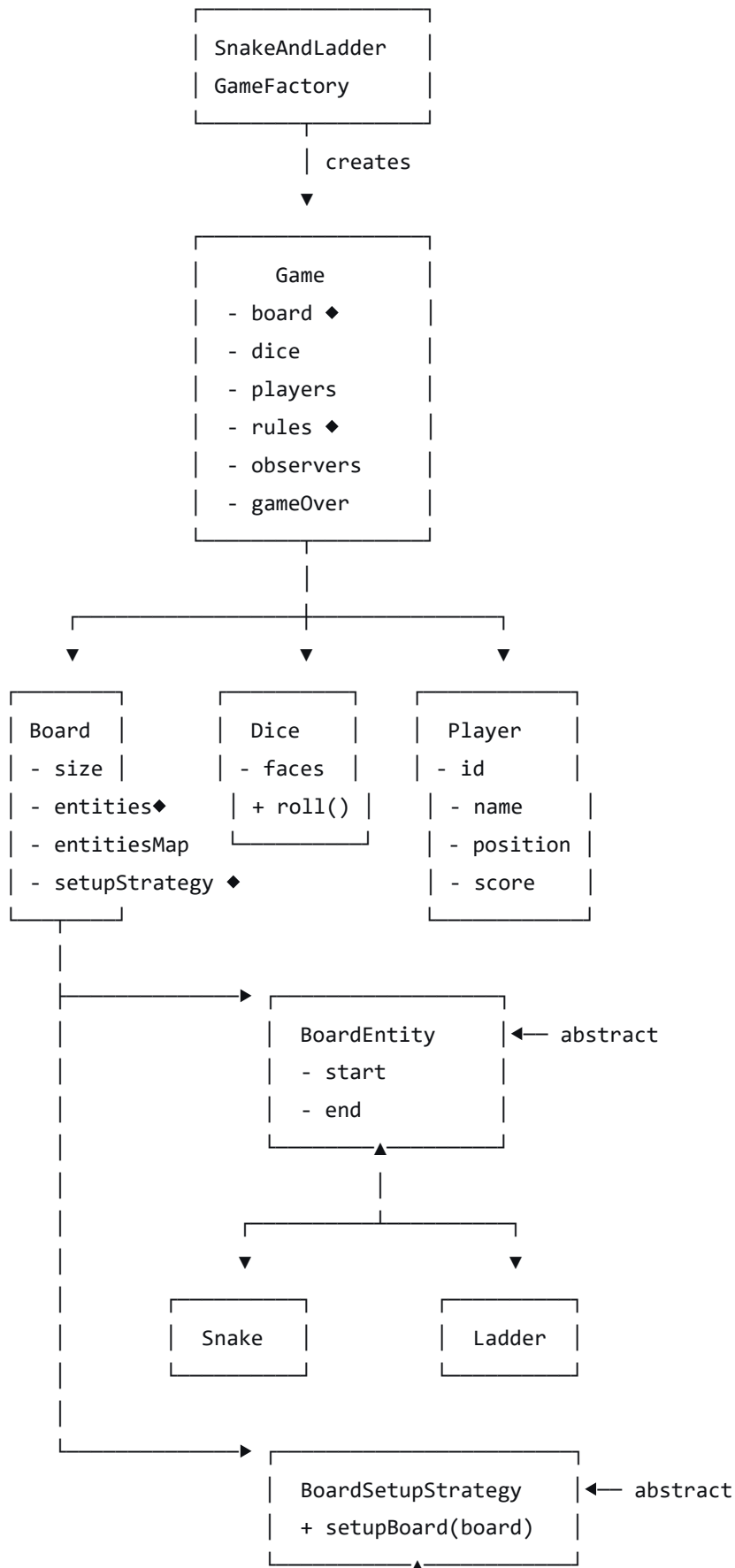
## 5.13 SnakeAndLadderGameFactory

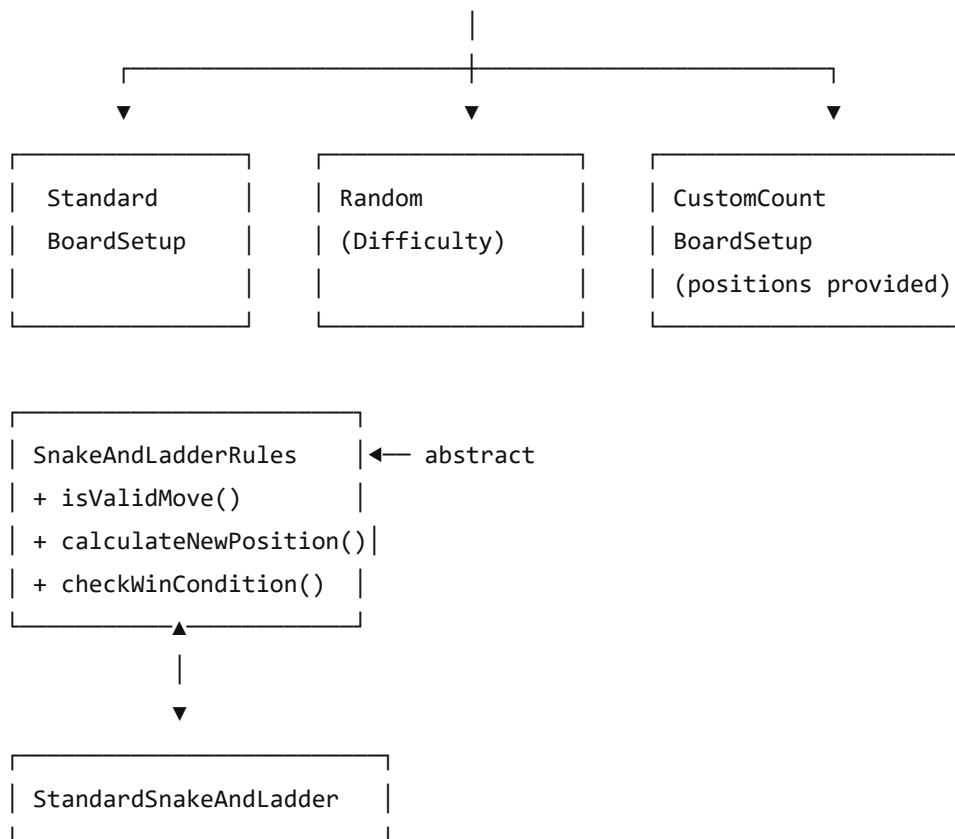
Three methods:

- createStandardGame()
- createRandomGame(boardSize, Difficulty)
- createCustomGame(boardSize, CustomCountBoardSetupStrategy)

## 6. UML Relationships

text





Composition (◆): Game - Rules, Game - Board, Board - BoardEntity

Aggregation: Game - Players, Game - Observers

## 7. Game Play Flow

text

```

play():
    if players < 2: error
    notify("Game started!")
    board.display()
    while (!gameOver):
        player = players.pollFirst()
        ask user to press Enter
        diceValue = dice.roll()
        currentPos = player.position
        if (!rules.isValidMove(currentPos, diceValue, board.size)):
            print("Invalid move, waiting for correct dice roll")
            players.addLast(player) // rotate
            continue
        intermediate = currentPos + diceValue
        newPos = rules.calculateNewPosition(currentPos, diceValue, board)
        player.position = newPos
        // notify if snake/ladder
  
```

```

entity = board.getEntity(intermediate)
if (entity != null):
    if entity.name == "Snake": notify("Snake encountered!")
    else: notify("Ladder climbed!")
notify(player.name + " now at " + player.position)
board.display()
if (rules.checkWinCondition(player.position, board.size)):
    print(player.name + " wins!")
    gameOver = true
else:
    players.addLast(player)

```

## 8. Java Implementation

### 8.1 BoardEntity (Abstract)

java

```

public abstract class BoardEntity {
    protected int start;
    protected int end;

    public BoardEntity(int start, int end) {
        this.start = start;
        this.end = end;
    }

    public int getStart() { return start; }
    public int getEnd() { return end; }

    public abstract String getName();
    public abstract void display();
}

```

### 8.2 Snake & Ladder

java

```

public class Snake extends BoardEntity {
    public Snake(int start, int end) {
        super(start, end);
        if (end >= start) throw new RuntimeException("Invalid Snake: end must be less than start");
    }
}

```

```

    }

    @Override public String getName() { return "Snake"; }

    @Override
    public void display() {
        System.out.println("Snake: " + start + " -> " + end);
    }
}

public class Ladder extends BoardEntity {
    public Ladder(int start, int end) {
        super(start, end);
        if (end <= start) throw new RuntimeException("Invalid Ladder: end must be greater
than start");
    }

    @Override public String getName() { return "Ladder"; }

    @Override
    public void display() {
        System.out.println("Ladder: " + start + " -> " + end);
    }
}

```

## 8.3 BoardSetupStrategy + Implementations

```

java

import java.util.*;

public interface BoardSetupStrategy {
    void setupBoard(Board board);
}

// ----- Standard -----
class StandardBoardSetupStrategy implements BoardSetupStrategy {
    @Override
    public void setupBoard(Board board) {
        if (board.getSize() != 100) {
            System.out.println("Standard setup only works for 10x10 board");
            return;
        }
        int[][] snakes = {{99, 54}, {95, 75}, {93, 73}, {87, 24}, {64, 60},
                           {62, 19}, {17, 7}, {54, 34}, {48, 26}, {16, 6}};
        int[][] ladders = {{1, 38}, {4, 14}, {9, 31}, {21, 42}, {28, 84},
                           {36, 44}, {51, 67}, {71, 91}, {80, 100}};
    }
}

```

```

        for (int[] s : snakes) board.addEntity(new Snake(s[0], s[1]));
        for (int[] l : ladders) board.addEntity(new Ladder(l[0], l[1]));
    }
}

// ----- Random with Difficulty -----
class RandomBoardSetupStrategy implements BoardSetupStrategy {
    public enum Difficulty { EASY, MEDIUM, HARD }

    private final Difficulty difficulty;
    private final Random random = new Random();

    public RandomBoardSetupStrategy(Difficulty d) { this.difficulty = d; }

    @Override
    public void setupBoard(Board board) {
        int totalEntities = board.getSize() / 10;
        double snakeProbability;
        switch (difficulty) {
            case EASY: snakeProbability = 0.3; break;
            case HARD: snakeProbability = 0.7; break;
            default: snakeProbability = 0.5;
        }
        setupWithProbability(board, totalEntities, snakeProbability);
    }

    private void setupWithProbability(Board board, int total, double snakeProb) {
        int snakesAdded = 0, laddersAdded = 0;
        for (int i = 0; i < total; i++) {
            if (Math.random() < snakeProb) {
                if (addRandomSnake(board)) snakesAdded++;
            } else {
                if (addRandomLadder(board)) laddersAdded++;
            }
        }
        System.out.println("Random setup: snakes=" + snakesAdded + ", ladders=" + ladders
Added);
    }

    private boolean addRandomSnake(Board board) {
        for (int attempts = 0; attempts < 50; attempts++) {
            int start = 2 + random.nextInt(board.getSize() - 2);
            int end = 1 + random.nextInt(start - 1);
            if (board.canAddEntity(start)) {
                board.addEntity(new Snake(start, end));
                return true;
            }
        }
    }
}

```

```

    }
    return false;
}

private boolean addRandomLadder(Board board) {
    for (int attempts = 0; attempts < 50; attempts++) {
        int start = 1 + random.nextInt(board.getSize() - 2);
        int end = start + 1 + random.nextInt(board.getSize() - start);
        if (end > board.getSize()) continue;
        if (board.canAddEntity(start)) {
            board.addEntity(new Ladder(start, end));
            return true;
        }
    }
    return false;
}
}

// ----- Custom Count -----
class CustomCountBoardSetupStrategy implements BoardSetupStrategy {
    private final int numSnakes, numLadders;
    private final boolean randomPlacement;
    private final List<int[]> snakePositions = new ArrayList<>();
    private final List<int[]> ladderPositions = new ArrayList<>();
    private final Random random = new Random();

    public CustomCountBoardSetupStrategy(int numSnakes, int numLadders, boolean randomPlacement) {
        this.numSnakes = numSnakes;
        this.numLadders = numLadders;
        this.randomPlacement = randomPlacement;
    }

    public void addSnakePosition(int start, int end) { snakePositions.add(new int[]{start, end}); }
    public void addLadderPosition(int start, int end) { ladderPositions.add(new int[]{start, end}); }

    @Override
    public void setupBoard(Board board) {
        if (randomPlacement) {
            placeRandom(board);
        } else {
            for (int[] s : snakePositions)
                if (board.canAddEntity(s[0])) board.addEntity(new Snake(s[0], s[1]));
            for (int[] l : ladderPositions)
                if (board.canAddEntity(l[0])) board.addEntity(new Ladder(l[0], l[1]));
        }
    }
}

```

```

    }

    private void placeRandom(Board board) {
        int snakesAdded = 0;
        while (snakesAdded < numSnakes) {
            int start = 2 + random.nextInt(board.getSize() - 2);
            int end = 1 + random.nextInt(start - 1);
            if (board.canAddEntity(start)) {
                board.addEntity(new Snake(start, end));
                snakesAdded++;
            }
        }
        int laddersAdded = 0;
        while (laddersAdded < numLadders) {
            int start = 1 + random.nextInt(board.getSize() - 2);
            int end = start + 1 + random.nextInt(board.getSize() - start);
            if (end > board.getSize()) continue;
            if (board.canAddEntity(start)) {
                board.addEntity(new Ladder(start, end));
                laddersAdded++;
            }
        }
    }
}

```

## 8.4 Board

java

```

import java.util.*;

public class Board {
    private final int size;
    private final List<BoardEntity> entities = new ArrayList<>();
    private final Map<Integer, BoardEntity> entitiesMap = new HashMap<>();

    public Board(int size) { this.size = size; }

    public int getSize() { return size; }
    public List<BoardEntity> getEntities() { return entities; }

    public boolean canAddEntity(int position) {
        return !entitiesMap.containsKey(position)
            && position > 0 && position < size;
    }

    public void addEntity(BoardEntity entity) {

```

```

        if (canAddEntity(entity.getStart())) {
            entities.add(entity);
            entitiesMap.put(entity.getStart(), entity);
        }
    }

    public BoardEntity getEntity(int position) {
        return entitiesMap.get(position);
    }

    public void setupBoard(BoardSetupStrategy strategy) {
        strategy.setupBoard(this);
    }

    public void display() {
        System.out.println("----- Board (size " + size + ") -----");
        for (BoardEntity e : entities) e.display();
        System.out.println("-----");
    }
}

```

## 8.5 Dice

java

```

import java.util.Random;

public class Dice {
    private final int faces;
    private final Random random = new Random();

    public Dice(int faces) { this.faces = faces; }

    public int roll() { return 1 + random.nextInt(faces); }
}

```

## 8.6 Player

java

```

public class Player {
    private final int id;
    private final String name;
    private int position;
    private int score;
}

```

```

public Player(int id, String name) {
    this.id = id;
    this.name = name;
    this.position = 0;
    this.score = 0;
}

public int getId() { return id; }
public String getName() { return name; }
public int getPosition() { return position; }
public void setPosition(int p) { this.position = p; }
public int getScore() { return score; }
public void incrementScore() { score++; }
}

```

## 8.7 Rules (Strategy)

java

```

public interface SnakeAndLadderRules {
    boolean isValidMove(int position, int diceValue, int boardSize);
    int calculateNewPosition(int position, int diceValue, Board board);
    boolean checkWinCondition(int position, int boardSize);
}

class StandardSnakeAndLadderRules implements SnakeAndLadderRules {

    @Override
    public boolean isValidMove(int position, int diceValue, int boardSize) {
        return position + diceValue <= boardSize;
    }

    @Override
    public int calculateNewPosition(int position, int diceValue, Board board) {
        int next = position + diceValue;
        if (next > board.getSize()) return position;
        BoardEntity entity = board.getEntity(next);
        return (entity == null) ? next : entity.getEnd();
    }

    @Override
    public boolean checkWinCondition(int position, int boardSize) {
        return position == boardSize;
    }
}

```

## 8.8 Observer + ConsoleNotifier

java

```
public interface IObserver {
    void update(String message);
}

class SnakeAndLadderConsoleNotifier implements IObserver {
    @Override
    public void update(String message) {
        System.out.println("[NOTIFICATION] " + message);
    }
}
```

## 8.9 Game

java

```
import java.util.*;

public class Game {
    private final Board board;
    private final Dice dice;
    private final Deque<Player> players = new ArrayDeque<>();
    private final SnakeAndLadderRules rules;
    private final List<IObserver> observers = new ArrayList<>();
    private boolean gameOver;

    public Game(Board board, Dice dice) {
        this.board = board;
        this.dice = dice;
        this.rules = new StandardSnakeAndLadderRules();
        this.gameOver = false;
    }

    public void addPlayer(Player p) { players.addLast(p); }
    public void addObserver(IObserver o) { observers.add(o); }

    public void notifyObservers(String msg) {
        for (IObserver o : observers) o.update(msg);
    }

    public void displayPlayerPosition() {
        for (Player p : players)
            System.out.println(p.getName() + " -> " + p.getPosition());
    }
}
```

```

public void play() {
    if (players.size() < 2) {
        System.out.println("At least 2 players required");
        return;
    }

    notifyObservers("Snake & Ladders started!");
    board.display();

    Scanner sc = new Scanner(System.in);

    while (!gameOver) {
        Player current = players.pollFirst();
        System.out.print(current.getName() + "'s turn. Press Enter to roll the dic
e...");
        sc.nextLine();

        int diceValue = dice.roll();
        System.out.println("Dice rolled: " + diceValue);

        int currentPos = current.getPosition();

        if (!rules.isValidMove(currentPos, diceValue, board.getSize())) {
            System.out.println("Invalid move. Need exactly " +
                (board.getSize() - currentPos) + " to win.");
            players.addLast(current);
            continue;
        }

        int intermediate = currentPos + diceValue;
        int newPos = rules.calculateNewPosition(currentPos, diceValue, board);
        current.setPosition(newPos);

        // Notify snake/ladder
        BoardEntity entity = board.getEntity(intermediate);
        if (entity != null) {
            if ("Snake".equals(entity.getName()))
                notifyObservers(current.getName() + " hit a Snake at " + intermediate
+
                    " and went down to " + newPos);
            else
                notifyObservers(current.getName() + " climbed a Ladder at " + interme
diate +
                    " and went up to " + newPos);
        }

        notifyObservers(current.getName() + " moved to " + newPos);
    }
}

```

```

board.display();
displayPlayerPosition();

if (rules.checkWinCondition(current.getPosition(), board.getSize())) {
    current.incrementScore();
    System.out.println(current.getName() + " wins!");
    notifyObservers(current.getName() + " wins!");
    gameOver = true;
} else {
    players.addLast(current);
}
}
}
}

```

## 8.10 Factory

java

```

public class SnakeAndLadderGameFactory {

    public static Game createStandardGame() {
        Board board = new Board(100);
        board.setupBoard(new StandardBoardSetupStrategy());
        Dice dice = new Dice(6);
        return new Game(board, dice);
    }

    public static Game createRandomGame(int boardSize,
                                         RandomBoardSetupStrategy.Difficulty difficulty) {
        Board board = new Board(boardSize);
        board.setupBoard(new RandomBoardSetupStrategy(difficulty));
        Dice dice = new Dice(6);
        return new Game(board, dice);
    }

    public static Game createCustomGame(int boardSize, CustomCountBoardSetupStrategy strategy) {
        Board board = new Board(boardSize);
        board.setupBoard(strategy);
        Dice dice = new Dice(6);
        return new Game(board, dice);
    }
}

```

## 8.11 Main (Demo)

```

java

import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Choose game type:");
        System.out.println("1. Standard");
        System.out.println("2. Random");
        System.out.println("3. Custom");
        int choice = sc.nextInt();

        Game game = null;

        switch (choice) {
            case 1:
                game = SnakeAndLadderGameFactory.createStandardGame();
                break;
            case 2:
                System.out.print("Enter board size: ");
                int size = sc.nextInt();
                System.out.print("Difficulty (1-Easy, 2-Medium, 3-Hard): ");
                int d = sc.nextInt();
                RandomBoardSetupStrategy.Difficulty diff =
                    d == 1 ? RandomBoardSetupStrategy.Difficulty.EASY :
                    d == 3 ? RandomBoardSetupStrategy.Difficulty.HARD :
                    RandomBoardSetupStrategy.Difficulty.MEDIUM;
                game = SnakeAndLadderGameFactory.createRandomGame(size, diff);
                break;
            case 3:
                System.out.print("Enter board size: ");
                int cSize = sc.nextInt();
                System.out.print("Number of snakes: ");
                int nSnakes = sc.nextInt();
                System.out.print("Number of ladders: ");
                int nLadders = sc.nextInt();
                System.out.print("Random placement? (1=Yes, 0=No): ");
                boolean randomPlace = sc.nextInt() == 1;

                CustomCountBoardSetupStrategy custom =
                    new CustomCountBoardSetupStrategy(nSnakes, nLadders, randomPlac

e);

                if (!randomPlace) {

```

```

        for (int i = 0; i < nSnakes; i++) {
            System.out.print("Snake " + (i+1) + " (start end): ");
            custom.addSnakePosition(sc.nextInt(), sc.nextInt());
        }
        for (int i = 0; i < nLadders; i++) {
            System.out.print("Ladder " + (i+1) + " (start end): ");
            custom.addLadderPosition(sc.nextInt(), sc.nextInt());
        }
    }
    game = SnakeAndLadderGameFactory.createCustomGame(cSize, custom);
    break;
default:
    System.out.println("Invalid choice");
    return;
}

game.addObserver(new SnakeAndLadderConsoleNotifier());

System.out.print("How many players? ");
int numPlayers = sc.nextInt();
sc.nextLine();

for (int i = 1; i <= numPlayers; i++) {
    System.out.print("Player " + i + " name: ");
    String name = sc.nextLine();
    game.addPlayer(new Player(i, name));
}

game.play();
}
}

```

## 9. Python Implementation

python

```

from abc import ABC, abstractmethod
from collections import deque
from enum import Enum
import random

# ===== BoardEntity =====
class BoardEntity(ABC):
    def __init__(self, start: int, end: int):

```

```

        self.start = start
        self.end = end

    @abstractmethod
    def get_name(self) -> str: ...

    @abstractmethod
    def display(self): ...

class Snake(BoardEntity):
    def __init__(self, start, end):
        if end >= start:
            raise ValueError("Invalid Snake: end must be < start")
        super().__init__(start, end)

    def get_name(self): return "Snake"
    def display(self): print(f"Snake: {self.start} -> {self.end}")

class Ladder(BoardEntity):
    def __init__(self, start, end):
        if end <= start:
            raise ValueError("Invalid Ladder: end must be > start")
        super().__init__(start, end)

    def get_name(self): return "Ladder"
    def display(self): print(f"Ladder: {self.start} -> {self.end}")

# ===== Setup Strategies =====
class BoardSetupStrategy(ABC):
    @abstractmethod
    def setup_board(self, board): ...

class StandardBoardSetupStrategy(BoardSetupStrategy):
    def setup_board(self, board):
        if board.size != 100:
            print("Standard setup only works for 10x10 board")
            return
        snakes = [(99,54),(95,75),(93,73),(87,24),(64,60),
                  (62,19),(17,7),(54,34),(48,26),(16,6)]
        ladders = [(1,38),(4,14),(9,31),(21,42),(28,84),
                  (36,44),(51,67),(71,91),(80,100)]
        for s, e in snakes: board.add_entity(Snake(s, e))
        for s, e in ladders: board.add_entity(Ladder(s, e))

```

```

class RandomBoardSetupStrategy(BoardSetupStrategy):
    class Difficulty(Enum):
        EASY = 1
        MEDIUM = 2
        HARD = 3

    def __init__(self, difficulty: "RandomBoardSetupStrategy.Difficulty"):
        self.difficulty = difficulty

    def setup_board(self, board):
        total = board.size // 10
        if self.difficulty == self.Difficulty.EASY:    prob = 0.3
        elif self.difficulty == self.Difficulty.HARD: prob = 0.7
        else:                                          prob = 0.5

        snakes_added = ladders_added = 0
        for _ in range(total):
            if random.random() < prob:
                if self._add_random_snake(board): snakes_added += 1
            else:
                if self._add_random_ladder(board): ladders_added += 1

        print(f"Random setup: snakes={snakes_added}, ladders={ladders_added}")

    def _add_random_snake(self, board):
        for _ in range(50):
            start = random.randint(2, board.size - 1)
            end = random.randint(1, start - 1)
            if board.can_add_entity(start):
                board.add_entity(Snake(start, end))
                return True
        return False

    def _add_random_ladder(self, board):
        for _ in range(50):
            start = random.randint(1, board.size - 2)
            end = random.randint(start + 1, board.size)
            if board.can_add_entity(start):
                board.add_entity(Ladder(start, end))
                return True
        return False

class CustomCountBoardSetupStrategy(BoardSetupStrategy):
    def __init__(self, num_snakes, num_ladders, random_placement):
        self.num_snakes = num_snakes
        self.num_ladders = num_ladders

```

```

self.random_placement = random_placement
self.snake_positions = []
self.ladder_positions = []

def add_snake_position(self, s, e): self.snake_positions.append((s, e))
def add_ladder_position(self, s, e): self.ladder_positions.append((s, e))

def setup_board(self, board):
    if self.random_placement:
        sa = 0
        while sa < self.num_snakes:
            s = random.randint(2, board.size - 1)
            e = random.randint(1, s - 1)
            if board.can_add_entity(s):
                board.add_entity(Snake(s, e)); sa += 1
        la = 0
        while la < self.num_ladders:
            s = random.randint(1, board.size - 2)
            e = random.randint(s + 1, board.size)
            if board.can_add_entity(s):
                board.add_entity(Ladder(s, e)); la += 1
    else:
        for s, e in self.snake_positions:
            if board.can_add_entity(s): board.add_entity(Snake(s, e))
        for s, e in self.ladder_positions:
            if board.can_add_entity(s): board.add_entity(Ladder(s, e))

# ===== Board =====
class Board:
    def __init__(self, size):
        self.size = size
        self.entities = []
        self.entities_map = {}

    def can_add_entity(self, pos):
        return 0 < pos < self.size and pos not in self.entities_map

    def add_entity(self, entity):
        if self.can_add_entity(entity.start):
            self.entities.append(entity)
            self.entities_map[entity.start] = entity

    def get_entity(self, pos):
        return self.entities_map.get(pos)

    def setup_board(self, strategy: BoardSetupStrategy):
        strategy.setup_board(self)

```

```

def display(self):
    print(f"----- Board (size {self.size}) -----")
    for e in self.entities: e.display()
    print("-----")

# ===== Dice =====
class Dice:
    def __init__(self, faces): self.faces = faces
    def roll(self): return random.randint(1, self.faces)

# ===== Player =====
class Player:
    def __init__(self, pid, name):
        self.id = pid
        self.name = name
        self.position = 0
        self.score = 0

    def increment_score(self): self.score += 1

# ===== Rules =====
class SnakeAndLadderRules(ABC):
    @abstractmethod
    def is_valid_move(self, pos, dice, board_size): ...
    @abstractmethod
    def calculate_new_position(self, pos, dice, board): ...
    @abstractmethod
    def check_win_condition(self, pos, board_size): ...

class StandardSnakeAndLadderRules(SnakeAndLadderRules):
    def is_valid_move(self, pos, dice, board_size):
        return pos + dice <= board_size

    def calculate_new_position(self, pos, dice, board):
        nxt = pos + dice
        if nxt > board.size: return pos
        entity = board.get_entity(nxt)
        return entity.end if entity else nxt

    def check_win_condition(self, pos, board_size):
        return pos == board_size

```

```

# ===== Observer =====
class IObserver(ABC):
    @abstractmethod
    def update(self, message): ...

class ConsoleNotifier(IObserver):
    def update(self, message):
        print(f"[NOTIFICATION] {message}")

# ===== Game =====
class Game:
    def __init__(self, board, dice):
        self.board = board
        self.dice = dice
        self.players = deque()
        self.rules = StandardSnakeAndLadderRules()
        self.observers = []
        self.game_over = False

    def add_player(self, p): self.players.append(p)
    def add_observer(self, o): self.observers.append(o)

    def notify(self, msg):
        for o in self.observers: o.update(msg)

    def display_player_position(self):
        for p in self.players:
            print(f"{p.name} -> {p.position}")

    def play(self):
        if len(self.players) < 2:
            print("At least 2 players required")
            return

        self.notify("Snake & Ladders started!")
        self.board.display()

        while not self.game_over:
            current = self.players.popleft()
            input(f"{current.name}'s turn. Press Enter to roll...")
            dice_value = self.dice.roll()
            print(f"Dice: {dice_value}")

            pos = current.position

            if not self.rules.is_valid_move(pos, dice_value, self.board.size):

```

```

        need = self.board.size - pos
        print(f"Invalid move. Need exactly {need} to win.")
        self.players.append(current)
        continue

    intermediate = pos + dice_value
    new_pos = self.rules.calculate_new_position(pos, dice_value, self.board)
    current.position = new_pos

    entity = self.board.get_entity(intermediate)
    if entity:
        if entity.get_name() == "Snake":
            self.notify(f"{current.name} hit a Snake at {intermediate}, went down
to {new_pos}")
        else:
            self.notify(f"{current.name} climbed a Ladder at {intermediate}, went
up to {new_pos}")

    self.notify(f"{current.name} moved to {new_pos}")
    self.board.display()
    self.display_player_position()

    if self.rules.check_win_condition(current.position, self.board.size):
        current.increment_score()
        print(f"{current.name} wins!")
        self.notify(f"{current.name} wins!")
        self.game_over = True
    else:
        self.players.append(current)

# ===== Factory =====
class GameFactory:
    @staticmethod
    def create_standard_game():
        board = Board(100)
        board.setup_board(StandardBoardSetupStrategy())
        return Game(board, Dice(6))

    @staticmethod
    def create_random_game(board_size, difficulty):
        board = Board(board_size)
        board.setup_board(RandomBoardSetupStrategy(difficulty))
        return Game(board, Dice(6))

    @staticmethod
    def create_custom_game(board_size, strategy):
        board = Board(board_size)

```

```

board.setup_board(strategy)
return Game(board, Dice(6))

# ===== Demo =====
if __name__ == "__main__":
    print("1. Standard 2. Random 3. Custom")
    choice = int(input("Choose game type: "))

    if choice == 1:
        game = GameFactory.create_standard_game()
    elif choice == 2:
        size = int(input("Board size: "))
        d = int(input("Difficulty (1-Easy, 2-Medium, 3-Hard): "))
        diff = (RandomBoardSetupStrategy.Difficulty.EASY if d == 1 else
                RandomBoardSetupStrategy.Difficulty.HARD if d == 3 else
                RandomBoardSetupStrategy.Difficulty.MEDIUM)
        game = GameFactory.create_random_game(size, diff)
    else:
        size = int(input("Board size: "))
        ns = int(input("Number of snakes: "))
        nl = int(input("Number of ladders: "))
        rp = input("Random placement (y/n)? ").lower() == "y"
        custom = CustomCountBoardSetupStrategy(ns, nl, rp)
        if not rp:
            for i in range(ns):
                s, e = map(int, input(f"Snake {i+1} (start end): ").split())
                custom.add_snake_position(s, e)
            for i in range(nl):
                s, e = map(int, input(f"Ladder {i+1} (start end): ").split())
                custom.add_ladder_position(s, e)
        game = GameFactory.create_custom_game(size, custom)

    game.add_observer(ConsoleNotifier())
    n = int(input("How many players? "))
    for i in range(1, n + 1):
        name = input(f"Player {i} name: ")
        game.add_player(Player(i, name))

    game.play()

```

## 10. Design Patterns Used

| Pattern            | Where                                              | Why                                                 |
|--------------------|----------------------------------------------------|-----------------------------------------------------|
| <b>Strategy</b>    | BoardSetupStrategy (Standard, Random, Custom)      | Different ways to set up board                      |
| <b>Strategy</b>    | SnakeAndLadderRules                                | Extensible rules                                    |
| <b>Observer</b>    | IObserver + Game                                   | On-app notifications                                |
| <b>Factory</b>     | SnakeAndLadderGameFactory                          | Centralized game creation                           |
| <b>Composition</b> | Game ◆— Board, Board ◆— BoardEntity, Game ◆— Rules | Strong lifecycle binding                            |
| <b>Facade</b>      | Game                                               | Orchestrates Board, Dice, Players, Rules, Observers |

## 11. Key Design Decisions

| Decision                                          | Why                                                |
|---------------------------------------------------|----------------------------------------------------|
| <b>BoardEntity abstraction</b> for Snake & Ladder | Same movement mechanism; polymorphism              |
| <b>Map&lt;Integer, BoardEntity&gt;</b> in Board   | O(1) lookup at any cell                            |
| <b>No 1D/2D array</b> in Board                    | Only need a map + list; saves memory               |
| <b>Only store Start</b> in map                    | Start is the trigger; End is retrieved from entity |
| <b>Deque&lt;Player&gt;</b>                        | Round-robin turn order for 2+ players              |
| <b>Difficulty enum nested in Random Strategy</b>  | Scoped only to that strategy                       |
| <b>RandomBoardSetupStrategy</b> with probability  | Easy = fewer snakes, Hard = more snakes            |
| <b>Composition</b> Game ◆— Rules / Board          | Rules/Entities strictly bound to game              |
| <b>Factory with 3 methods</b>                     | Standard, Random, Custom templates                 |
| <b>Dice.faces configurable</b>                    | 6-face standard, 10-face variant possible          |

| Decision                                | Why                      |
|-----------------------------------------|--------------------------|
| <b>Player.position stored on Player</b> | No redundant board state |

## 12. Real-World Insights

Same design philosophy extends to:

- **Ludo** — same Board/Player/Dice; different rules
- **Monopoly** — Dice + Board + special cells (chance, jail)
- **Snakes & Ladders variants** — custom setup strategies

### Interview Tips:

1. Draw the game on paper first.
2. Identify fixed vs. extensible parts.
3. Use **Strategy** for anything that might change (rules, setup).
4. Use **Observer** for "notify participants" requirements.
5. Use **Factory** to hide creation details.
6. **Top-Down** approach: Game → Board → BoardEntity → Snake / Ladder .

## 13. Sample Run Output

text

```
Choose game type: 1
How many players? 2
Player 1 name: Aditya
Player 2 name: Harshita

[NOTIFICATION] Snake & Ladders started!
----- Board (size 100) -----
Snake: 99 -> 54
Snake: 95 -> 75
...
Ladder: 80 -> 100
-----

Aditya's turn. Press Enter to roll...
```

```
Dice: 1
[NOTIFICATION] Aditya moved to 1
...
Harshita's turn. Press Enter to roll...
Dice: 4
[NOTIFICATION] Harshita moved to 4
...
Aditya's turn. Press Enter to roll...
Dice: 6
[NOTIFICATION] Aditya hit a Snake at 22, went down to 4
...
[NOTIFICATION] Aditya wins!
Aditya wins!
```

## 14. Key Takeaways

1. **Snake & Ladders shares structure with Tic-Tac-Toe** — Game, Board, Player, Rules, Observer, Factory.
2. **BoardSetupStrategy** is the star pattern here — 3 implementations (Standard, Random, Custom).
3. **Difficulty enum** scopes to RandomBoardSetupStrategy — allows Easy/Medium/Hard probabilities.
4. **BoardEntity abstraction** unifies Snake and Ladder (both move a player from Start to End).
5. **Map keyed by Start** gives  $O(1)$  "is there a snake/ladder at cell X?".
6. **Deque for players** — round-robin works for 2, 3, 4+ players.
7. **Validation in constructors** — Snake end < start; Ladder end > start.
8. **No 1D/2D array** needed — just size + list + map.
9. **Composition with Game** — Rules/Board/Entities cannot exist without a Game.
10. **Top-Down approach** works best for games — start with the orchestrator class.

## 15. Quick Cheatsheet

### Concept

### Implementation

Snake representation

Snake extends BoardEntity (end < start)

| Concept                    | Implementation                                                 |
|----------------------------|----------------------------------------------------------------|
| Ladder representation      | Ladder extends BoardEntity (end > start)                       |
| Board storage              | size + List<BoardEntity> +<br>Map<Integer, BoardEntity>        |
| Cell lookup                | Map.get(position) — O(1)                                       |
| Setup strategies           | Standard, Random (with Difficulty), Custom                     |
| Difficulty probabilities   | Easy 30% snakes, Medium 50%, Hard 70%                          |
| Dice                       | faces configurable (default 6)                                 |
| Turn rotation              | Deque<Player>                                                  |
| Win condition              | position == boardSize                                          |
| Move validation            | position + dice <= boardSize                                   |
| New position               | Check board entity at intermediate → teleport to<br>entity.end |
| Notification               | Observer Pattern with ConsoleNotifier                          |
| Game creation              | SnakeAndLadderGameFactory (3 methods)                          |
| Composition (◆)            | Game–Rules, Game–Board, Board–BoardEntity                      |
| Popular interview variants | Snake & Ladders, Ludo, Monopoly                                |

Would you like me to also produce:

- A **Mermaid class diagram** for Snake & Ladders?
- A **Ludo LLD** using the same design philosophy?
- A **multi-file Java project structure** for the code?

*This response is AI-generated, for reference only.*